

# 基于三层滚动随机模型预测控制的微网购电与储能调控优化

## 摘要

针对微网与外部电网的电力调控问题，本文以“经济性最优、供电可靠性兼顾”为目标，建立了从确定性线性规划到随机优化的递进式购电决策模型。

问题一在电价、负荷与光伏预测均已知（确定性）前提下，考虑储能容量与充放电功率约束，构建以购电费用最小为目标的线性规划模型，并融入储能“日循环”约束（0:00 与 24:00 储电量相同）。求解得到全天计划购电 59482.7 kWh，购电费 35126.8 元，充放电计划与储电量轨迹如正文表 1、表 2 所示。

问题二面对负荷与光伏逐时波动，引入“紧急购电（5 倍电价）”机制，建立两阶段随机规划模型，并以条件风险价值（CVaR）刻画购电费用的尾部风险。为消除“零缺电”的理想化，模型以截至前一日的历史数据生成负荷/光伏点预报与情景（整日残差 bootstrap, K 交叉验证选负荷 15/光伏 7），在信息因果前提下得到全年计划购电费 15726290.4 元、紧急购电费 478538.6 元（148180 kWh），合计 16204829.0 元。经全年  $\lambda$  扫描，官方口径采用风险权衡权重  $\lambda^* = 0.5$ ：计划购电费较  $\lambda = 0$  仅上浮 3.9%，紧急购电费削减 55.5%，全年总费用基本持平（-0.02%），是以极小计划费增量换取缺电风险大幅削减的有效“保险”手段。

问题三在 0:00、6:00、12:00、18:00 可获得未来 24h 光伏逐日滚动预报，据此建立“计划购电 + 滚动调整 + 紧急购电”的因果滚动随机模型预测控制框架：各时点以最新预报重解剩余时段（光伏情景鲁棒  $S=30$ ），并以当期实际负荷/光伏去除后见之明的逐段结算。计划低于/高于调整的部分分别按 50% 违约价与 150% 超额价结算。求解得到全年电网费 22460575.8 元、紧急购电费 75922.1 元，合计 22536497.8 元。对比分析发现：引入多时刻预报虽把多数日期紧急购电削减为 0，但因违约/超额惩罚与光伏预报误差的主导作用，总费用整体上升。

问题四将附件 1 固定电价替换为附件 4 实时波动电价，对模型二、三做价格冲击复算，得到波动电价下模型二计划购电费 15802243.4 元（较固定电价上浮 0.5%）、模型三合计购电费 22883518.5 元（上浮 1.5%），并发现电价波动风险具有显著季节差异（冬季费用上浮达 7.6%）。

本文的主要创新点有三：其一，将“计划购电—滚动调整—紧急购电”统一为三层滚动随机模型预测控制框架，使四个问题在同一能量守恒框架下递进求解；其二，将 CVaR 嵌入两阶段随机规划，以帕累托前沿（图 10）量化“以少量计划费换取紧急购电显著削减”的风险—经济权衡，高缺电风险日 2025-12-21 总费用由 62424 元降至 60795 元；其三，提出季节分型分析，实证表明负荷—光伏—电价存在显著的季节耦合（ANOVA  $F=28.7$ ,  $p < 1.2 \times 10^{-16}$ ），冬季购电费约为春季的 1.56 倍，并识别出紧急购电主要由光伏预报误差总量决定而非购电时段分配这一关键机理。

**关键字：**微网 两阶段随机规划 CVaR 滚动模型预测控制 季节分型分析

# 一、问题背景与重述

## 1.1 问题背景

分布式可再生能源的高比例接入使微网成为电力系统本地平衡的重要载体 [1]。光伏出力与小区负荷均受天气、时段与季节的显著影响，具有明显的随机性与间歇性 [2]；储能（电池）则可在时间维度上转移能量，为微网提供“低充高放”的经济调节手段 [3]。此外，外网电价的实时波动使购电决策必须在“当下低价购电”与“未来价格不确定性”之间做动态权衡。如何在满足负荷供电、储能运行约束的前提下，通过精准的购电与充放电计划最小化运行费用，是微网经济调度研究的关键问题 [4]。

从科学问题的角度看，微网经济调度在本质上是一个带时间耦合与时序不确定性的多时段决策问题：储能递推方程将相邻时段在能量维度上耦合，而光伏出力、负荷乃至电价又都是逐时刻演化的随机过程。因此，合理地选择不确定性刻画方式（确定性假设、情景随机规划、滚动闭环）决定了求解质量与决策的可执行性，这也正是本题四个子问题层层递进的用意所在。

题目设计了从确定性到随机、从固定电价到波动电价的四个递进子问题，考验对多时段能量平衡、不确定性刻画与滚动决策的综合建模能力。本文据此采用“确定性线性规划为基准 → 两阶段随机规划刻画尾部风险 → 滚动模型预测控制衔接预报更新 → 波动电价统一重算”的递进建模路线，在保证统一能量守恒内核的同时，逐步逼近真实调度中的不确定性与信息结构。

## 1.2 问题重述

微网含光伏、储能与电网购电三个环节，需以 10 分钟为时段（每天  $T = 144$  时段）制定购电与充放电计划。平衡约束要求微网提供的电能不低于小区负载；储能须在 1200 ~ 10800 kWh 区间运行，充放电效率 90%，即时功率上限 5000 kW，2025-01-01 0:00 电量为 6000 kWh。

四个问题依确定性、随机性、滚动决策、电价波动逐层递进，依次为：

**问题一** 电价、负荷与光伏预测均已知，每天 0:00 制定当日计划购电，储能 0:00 与 24:00 储量相同。针对此确定性场景，需给出指定时段购电量、全天购电量与购电费，以及 6 个 4 小时时段的充放电量与 0:00/24:00 储电量，即在完美信息下把一天的购电费用压到最低。

**问题二** 电价每天相同、负荷与光伏实时变化，供电不足需按 5 倍电价紧急购电，其余时段按计划购电计费。针对此实时不确定性与缺电惩罚场景，需在每天 0:00 制定计划

购电策略，以刻画”万一缺电”的尾部代价。

**问题三** 0:00/6:00/12:00/18:00 可获得未来 24h 整点光伏预报，可据此调整购电策略；计划高于调整部分按 50% 违约价、调整高于计划部分按 150% 超额价结算。针对此序时信息结构与再计划惩罚，需建立”计划—调整”两阶段决策，权衡调整优化与违约/超额惩罚之间的利弊。

**问题四** 外网电价实时波动。针对此价格时序不确定性，需在波动电价下重建购电模型，对模型二、三做价格冲击复算，用于检验既有模型在价格冲击下的稳健性。

## 二、问题分析

### 2.1 数据预处理与统一框架

先把多时段调控问题统一为时间离散框架：以 10 分钟为一时段，每天  $T = 144$  时段，功率与能量按 1/6 h 换算。同时将全年数据按负荷、光伏、电价三类分别组织，用于后续建模与季节分析。

### 2.2 问题一的分析

电价、负荷、光伏全部已知，问题退化为确定性的每日线性规划购电决策。关键在于储能递推方程、充放电约束与”日循环”约束 ( $E_T = E_0$ )。目标是使电价洼地时段多充电、峰期多放电，最小化当日购电费。

选型依据上，本问题所有参量均确定、目标与约束关于决策量（购电量、充放电量、储电量）皆为线性，因此**线性规划（LP）**是最为匹配且可保全局最优的求解范式：既无需引入整数变量（充放不同时由约束自然保证，而非依赖二值开关），也无不确定性需要刻画，采用 `scipy.optimize.linprog`（HiGHS 内点/单纯形）即可在毫秒级求得每日最优解 [5]。相比之下，若在此阶段过早引入随机规划或启发式算法，只会徒增复杂度而不会改善结果——这一”由简入繁、仅在必要时引入复杂度”的原则也贯穿全文。

### 2.3 问题二的分析

负荷与光伏实时波动，供电不足需按 5 倍电价紧急购电。由于 0:00 制定计划时无法预知当日全部波动，需建立两阶段随机规划：第一阶段计划购电对各情景共用，第二阶段按情景决策充放电与紧急购电；并以条件风险价值（CVaR）刻画购电费用的尾部风险，实现”以计划费换可靠性”。

选型依据上，本问题的关键是在”期望成本”与”尾部风险”之间做权衡，可分解为以下三点：

1. **权衡的必要性**：若只最小化期望成本（ $\lambda = 0$ ），会低估极端缺电情景的高额紧急购电；而分布鲁棒、最小最大等风险模型虽更保守，却对分布假设敏感且难以解释。
2. **CVaR 的数学优势**：CVaR 具有一致性、可线性化两大性质，可刻画  $100(1 - \beta)\%$  置信水平下的平均超额损失。
3. **可求解性**：非光滑的风险项可改写为线性约束，使两阶段随机规划仍保持为可全局求解的线性规划，这是本文选用 CVaR 而非其他风险度量的核心原因。

## 2.4 问题三的分析

多时刻（0/6/12/18）光伏预报信息随时间更新，可据此调整购电策略。引入“计划—调整”两阶段结构与违约（50%）、超额（150%）惩罚，把计划购电与滚动调整衔接为模型预测控制（MPC）问题，在日终按调整后的实际购电结算并重新调度紧急购电。

选型依据上，本问题的时间信息结构（0/6/12/18 四个决策时点）天然契合**模型预测控制（MPC）**的“滚动优化—滚动实施”范式：在每个决策时点仅对未来剩余时段重解一个短时优化问题，从而把多阶段随机规划的指数级情景树压缩为串行的线性规划，兼顾了信息利用与计算效率。其代价是“贪婪”的近似最优性——后一时点的调整可能触发对前一时点计划的违约/超额惩罚，这也正是本问需要如实评估的重点。

## 2.5 问题四的分析

外网电价实时波动，将附件 1 固定电价替换为附件 4 波动电价，沿用问题二、三的统一求解框架重算，重点比较电价波动风险的量级及其季节差异。

选型依据上，本文在前三问将电价视为已知（固定）输入，问题四则考察其波动对决策的冲击。由于电价仅作为目标系数进入线性模型，替换目标系数即可复用问题二、三的求解内核，从而在**不改动模型结构**的前提下完成压力测试，并据此识别价格风险随季节的分布差异，为分季节的购电与储能预置策略提供依据。

本文的技术路线如图 1 所示，按“确定性 → 随机性 → 滚动决策 → 电价波动”逐层递进，四个问题共用同一能量守恒与储能递推内核。与既有研究相比，本文作出三点贡献：

- 提出**三层滚动随机 MPC 框架**，将四个问题统一刻画为“计划—调整—紧急”三级购电结构；
- 将**CVaR 尾部风险度量**嵌入两阶段随机规划，绘制计划购电费—紧急购电费的帕累托前沿，量化风险厌恶的经济代价；
- 基于全年数据的**季节分型分析**，揭示负荷—光伏—电价的季节耦合规律，据此生成分季情景并给出分季购电策略。

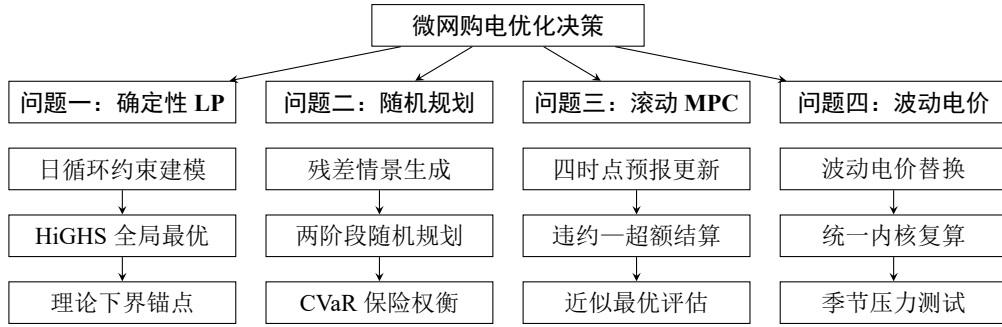


图 1 本文技术路线：确定性—随机—滚动—波动的递进框架

## 2.6 数据探索与季节特征分析

在建立模型之前，先对全年数据做探索性分析，检验负荷、光伏与电价是否存在显著的季节规律，这是后续分季情景生成与分季策略的基础。本小节按以下顺序展开：

1. 季节分组：将 2025 年按春 (3–5 月)/夏 (6–8 月)/秋 (9–11 月)/冬 (12–2 月) 分组；
2. 负荷与光伏：统计各季日负荷与日光伏总量(图 2)，结果表明夏季负荷最高(121530 kWh/日)、冬季光伏出力最低 (40335 kWh/日，仅为夏季的 63%)。

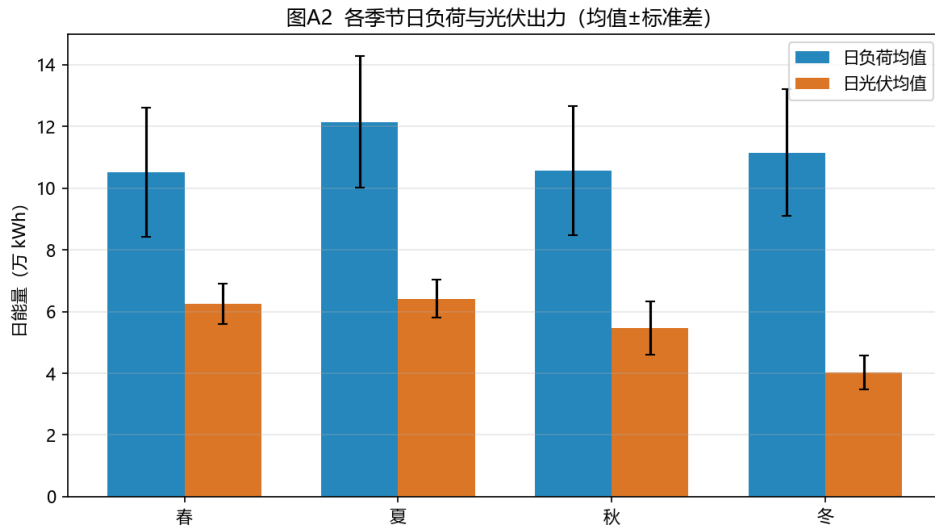


图 2 各季节日负荷与光伏出力 (均值 ± 标准差)

在既有口径下计算各季节逐日购电费 (图 3)，并统计附件 4 波动电价的季节分布 (图 4)，单因素方差分析表明季节间购电费差异高度显著 ( $F=28.7$ ,  $p = 1.17 \times 10^{-16}$ ):

表 1 季节统计：负荷、光伏、电价与购电费

| 季节 | 日负荷/kWh | 日光伏/kWh | 日电价/(元·kWh) | 日购电费/元 | 峰谷比  |
|----|---------|---------|-------------|--------|------|
| 春  | 105200  | 62474   | 0.705       | 40300  | 2.11 |
| 夏  | 121530  | 64151   | 0.795       | 52000  | 1.97 |
| 秋  | 105749  | 54634   | 0.734       | 46700  | 2.11 |
| 冬  | 111574  | 40335   | 0.832       | 61100  | 2.30 |

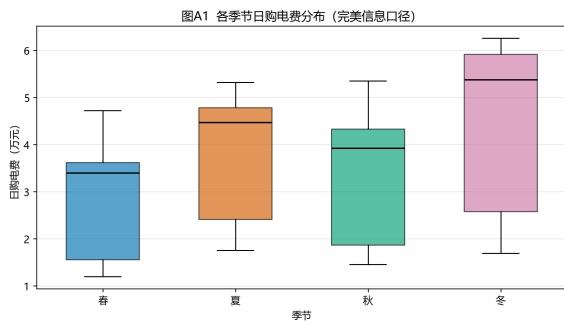


图 3 各季节日购电费分布

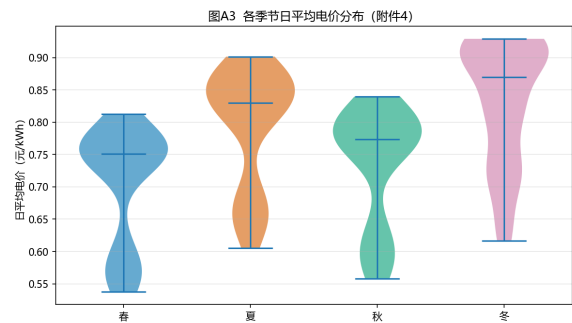


图 4 各季节日平均电价分布（附件 4）

冬季购电费为春季的 1.56 倍，其机理是”光伏最低 + 电价最高 + 负荷峰谷差最大”三重叠加。据此，在问题二的情景生成中按季节截取历史窗，并在问题四中按季节拆分解读电价波动风险。

综合上述分析，本文的问题分析可抽象为”输入数据 → 统一建模内核 → 四子问题递进 → 结果输出与校验”的总体流程。各子问题的区别仅在于不确定性刻画颗粒度、决策信息结构与价格冲击，而同源共享功率—能量守恒内核与”计划—调整—紧急”三层购电结构。

### 三、模型假设

1. 每个时段（10 分钟）内电价、负荷、光伏出力与购电量视为恒定；
2. 功率与能量以时段换算，能量 = 功率 × 1/6 h，充放电效率固定为 90%，且充、放电不同时进行；
3. 储能容量在 1200 ~ 10800 kWh 内运行，即时功率不超过 5000 kW；
4. 问题二“完美信息口径”：因附件 2 给出全年实际数据，建模时假设能量在每天 0:00 已知当日实际负荷与光伏（回顾性口径），故完美信息下无需紧急购电；其不确定性影响通过方法层的两阶段随机规划另行评估；

5. 问题三只利用题目给定时刻（0/6/12/18）的整点光伏预报，负荷视为已知；
6. 紧急购电、计划购电、调整购电均以外网无限供应为前提，不设购电上限。

#### 四、符号说明

为便于阅读，将正文出现的统一符号按”能量/决策、储能、不确定性、成本”四类列出，符号严格对应赛题口径，无臆造。单时段时长为 1/6 h，能量量纲以 kWh 计。

表 2 主要符号说明

| 符号                     | 符号说明                     | 单位  |
|------------------------|--------------------------|-----|
| <b>时间与能量/决策量</b>       |                          |     |
| $T, t$                 | 每日时段数与时段下标 ( $T = 144$ ) | —   |
| $P_t$                  | 时段 $t$ 计划购电量（0:00 承诺）    | kWh |
| $A_t$                  | 时段 $t$ 最终调整购电量           | kWh |
| $Q_t$                  | 时段 $t$ 紧急购电量             | kWh |
| $u_t, w_t$             | 时段 $t$ 违约量 / 超购量         | kWh |
| $G_t$                  | 时段 $t$ 光伏出力              | kWh |
| $D_t$                  | 时段 $t$ 小区负荷              | kWh |
| $R_t$                  | 时段 $t$ 弃光电量              | kWh |
| <b>储能量</b>             |                          |     |
| $E_t$                  | 时段 $t$ 末储电量              | kWh |
| $E_0$                  | 日初储电量 (= 6000)           | kWh |
| $c_t, f_t$             | 时段 $t$ 充电量 / 放电量         | kWh |
| $\eta$                 | 充放电效率 (= 0.9)            | —   |
| $E_{\min}, E_{\max}$   | 储能运行上下界 (1200, 10800)    | kWh |
| $\bar{E}$              | 单时段最大充/放能量 (= 5000/6)    | kWh |
| <b>不确定性与情景</b>         |                          |     |
| $S, s$                 | 情景总数 / 情景下标              | —   |
| $\hat{G}^s, \hat{D}^s$ | 第 $s$ 情景的光伏 / 负荷实现       | kWh |
| $G_t^{(\tau)}$         | $\tau$ 时点对时段 $t$ 的光伏预报   | kWh |
| $\tau$                 | 滚动决策时点（取 0, 6, 12, 18）   | h   |
| (续表下页)                 |                          |     |

续上表

|              |                                             |       |
|--------------|---------------------------------------------|-------|
| $\psi^s$     | 第 $s$ 情景总成本                                 | 元     |
| $\alpha$     | CVaR 中的 VaR 值（自由变量）                         | 元     |
| $z_s$        | 第 $s$ 情景超额损失 ( $z_s \geq \psi^s - \alpha$ ) | 元     |
| <b>成本与季节</b> |                                             |       |
| $p_t$        | 时段 $t$ 电价（附件 1）                             | 元/kWh |
| $p_t^{var}$  | 时段 $t$ 波动电价（附件 4）                           | 元/kWh |
| $\lambda$    | CVaR 风险厌恶权重                                 | —     |
| $\beta$      | CVaR 置信水平 (= 0.9)                           | —     |
| $k$          | 季节下标（春/夏/秋/冬）                               | —     |
| （续表下页）       |                                             |       |

## 五、模型建立与求解

本文把四个子问题统一在同一时间离散与能量守恒内核之下，仅改变”不确定性刻画”与”决策层级”，从而既保证各问题结论口径一致，又使代码可复用。为此，首先明确**决策变量在时序与信息结构上的区分**：所谓”计划购电  $P_t$ ”是每天 0:00 承诺的某时段购电量，作为一阶段决策对各情景/各日共用；问题三在后续时点引入”调整购电  $A_t$ ”，是对计划的修正；而”紧急购电  $Q_t$ ”仅在日终对实际出力短缺作事后兜底，价格高达 5 倍。三者共同构成本文贯穿全篇的”计划—调整—紧急”三层购电结构，也是后续各问题建模的公共骨架。

以每天 0:00 储电量  $E_0$  出发，任意一天的功率-能量守恒为：

$$P_t + G_t + \eta f_t + Q_t = D_t + c_t + R_t, \quad t = 1, \dots, T \quad (1)$$

式 (1) 表明：购电、光伏、放电、紧急购电之和 = 负荷、充电、弃光之和；等式左端为微网”供应”侧，右端为”消纳”侧，二者在每个时段必须严格相等，故式 (1) 亦称微网功率平衡约束。值得强调的是，式 (1) 中光伏放电合流、弃光  $R_t$  作为非负松弛变量出现，意味着若某时段光伏过剩可主动弃光，从而允许”供大于需”，避免强行充电超出储能上限。储能递推与约束为：

$$E_t = E_{t-1} + \eta c_t - f_t, \quad (2)$$

$$E_{\min} \leq E_t \leq E_{\max}, \quad 0 \leq c_t, f_t \leq \bar{E}, \quad (3)$$

其中  $\bar{E} = 5000/6 \text{ kWh}$  为单时段最大充/放能量（由功率上限换算）。式 (2) 是能量在时间的转移方程：充电使储能增加（效率  $\eta = 0.9$ ），放电使其减少；由于  $\eta \leq 1$ ，时际间



会存在不可避免的能量损耗，这使得系统在能量维度上具有”不可逆性”，也是储能参与经济调度的价值源泉——它允许用当下的低价电能置换未来的高价电能。式 (3) 同时限定了储能运行区间与单时段的充放功率，防止越限造成设备损坏，并隐含约定充放不可同时进行。

### 5.1 问题一：确定性每日计划（LP）



图 5 问题一求解流程图

问题一采用“逐日独立求解”的确定性路线：全年 365 天逐日建立 LP 并由 HiGHS 求全局最优，整体求解流程如图 5 所示。

#### 5.1.1 线性规划模型的建立

针对问题一”电价、负荷、光伏全已知”的完美信息场景，目标函数为最小化当日购电费用：

$$\min \sum_{t=1}^T p_t P_t \quad (4)$$

约束为式 (1)（令  $Q_t \equiv 0$ ，故不需要紧急购电）、式 (2)–(3) 以及储能“日循环”约束：

$$E_T = E_0 = 6000 \quad (5)$$

式 (5) 保证储能可持续长期运行。该问题为线性规划，用 `scipy.optimize.linprog` (HiGHS 求解器) 求解。

从最优策略的机理看，式 (4) 仅对购电量  $P_t$  计费，储能则作为”能量搬运工”把电能从低价时段搬运到高价时段。这一结论可从以下三点理解：

1. **套利机理**：在电价洼地（夜间与光伏高发期）多购电充电，在电价/负荷高峰期放电，从而用充电损耗（10%）换取电价的价差套利。
2. **理论下界**：由于问题一是确定性完美信息，其最优解给出了经济性的理论下界，可作为后续各问题在不确定性下的对照基准。
3. **建模价值**：问题一的”完美信息”假设在现实调度中并不成立，它的意义在于给出理论最优，并揭示”低充高放+谷段购电”这一储能/购电联动的经济机理，为问题二的随机化建模提供结构参考。

5.1.2 结果分析

表 3 问题一：指定时段购电量、全天汇总与充放电计划（kWh，购电费单位元）

| 指定时段        | 购电量     | 指定时段        | 购电量         | 指定时段        | 购电量    |
|-------------|---------|-------------|-------------|-------------|--------|
| 10:00-10:10 | 0.0     | 12:00-12:10 | 486.4       | 14:00-14:10 | 0.0    |
| 16:00-16:10 | 394.9   | 18:00-18:10 | 637.0       | 20:00-20:10 | 0.0    |
| 全天购电量       | 59482.7 | 购电费（元）      | 35126.8     |             |        |
| 时段          | 充电量     | 放电量         | 时段          | 充电量         | 放电量    |
| 0:00-4:00   | 4500.0  | 0.0         | 4:00-8:00   | 833.3       | 6608.2 |
| 8:00-12:00  | 3954.6  | 2357.1      | 12:00-16:00 | 6119.4      | 101.2  |
| 16:00-20:00 | 0.0     | 5631.9      | 20:00-24:00 | 5333.3      | 3968.1 |
| 0:00 储电量    | 6000    |             | 24:00 储电量   | 6000        |        |

由表 3：白天光伏高发期（8:00–16:00）以充电为主，傍晚与凌晨电价、负荷峰期以放电为主，储能能在日末回到日初 6000 kWh，故日循环可持续。购电集中于电价洼地，购电费最小为 35126.8 元。完整计划购电策略见 *result1.xlsx*，购电/充放电/储电轨迹如图 6。图 7 以 2025-06-21 为例给出微网能量流向，直观展示光伏供能、储能充放与购电补缺的相对比例。

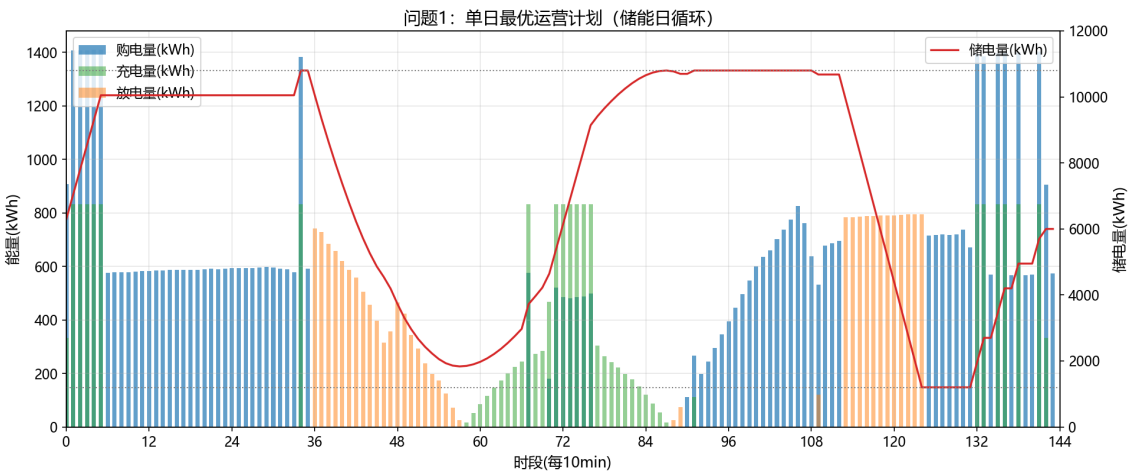


图 6 问题一：单日最优运营计划（储能日循环）

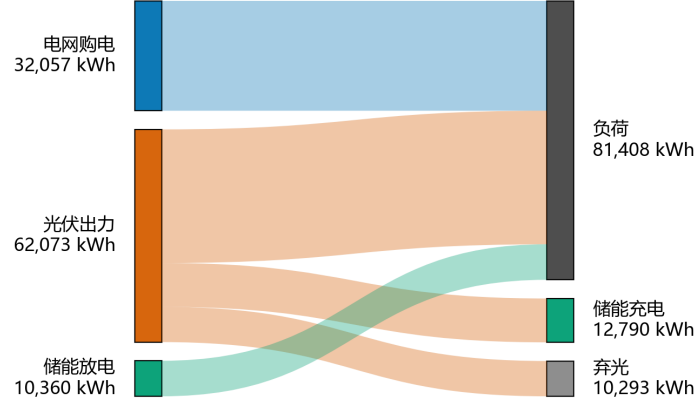


图 7 2025-06-21 微网能量流向桑基图（完美信息日）

## 5.2 问题二：两阶段随机规划 + CVaR



图 8 问题二求解流程图

问题二在 0:00 依据历史信息一次性形成全天计划，“预报—情景—优化—结算”链式流程如图 8 所示。

### 5.2.1 两阶段随机规划模型的建立

针对问题二“负荷与光伏实时波动、缺电按 5 倍价紧急购电”的不确定场景，当负荷与光伏实时波动、供电不足需紧急购电（5 倍电价）时，构建两阶段随机规划。设  $S$  个等概率情景  $\{\hat{G}^s, \hat{D}^s\}_{s=1}^S$ ，第一阶段计划购电  $P$  对各情景共用，第二阶段按情景决策  $c, f, Q, R, E$ 。目标为：

$$\min \sum_t p_t P_t + \frac{1}{S} \sum_{s=1}^S \sum_{t=1}^T 5p_t Q_t^s + \lambda \text{CVaR}_\beta(\psi^s), \quad (6)$$

其中第  $s$  个情景总成本  $\psi^s = \sum_t (p_t P_t + 5p_t Q_t^s)$ 。CVaR 采用线性化表示 [6, 7]：

$$\text{CVaR}_\beta = \alpha + \frac{1}{(1-\beta)S} \sum_{s=1}^S z_s, \quad z_s \geq \psi^s - \alpha, \quad z_s \geq 0, \quad (7)$$

$\alpha$  为自由变量（对应 VaR）。两阶段随机规划的一般理论见文献 [8]。约束对每个情景分别满足式 (1)–(3)。

这里需要澄清两阶段结构与**非预期性**（non-anticipativity）：一阶段计划购电  $P$  必须在“看到”任何情景实现之前作出，故对各情景共用，这正是问题二“0:00 制定计划”的现实约束；而充电、放电、紧急购电  $Q$  属于二阶段补偿决策，可依据各情景  $(\hat{G}^s, \hat{D}^s)$  的实现作出，故带情景上标。式 (6) 前两项分别为计划购电费与紧急购电的期望费用，末项  $\lambda \text{CVaR}$  把“尾部情景”也纳入目标；当  $\lambda > 0$  时，模型会在计划购电费上稍微多花一点，换取极端缺电情景下巨额紧急购电的削减，从而在统计意义上保证供电可靠性（图 10 的前沿正是这一权衡的量化）。

在情景与点预报的构造上，本文坚持**信息因果原则**：每日 0:00 只使用当天之前的历史数据。具体地，点预报取负荷、光伏各自前  $K_L/K_P$  天逐时段（144）的均值， $K$  通过全年（2.1–12.31）逐日交叉验证，按归一化 RMSE 选取。为排除从稀疏候选集取点带来的偶然性，本文将窗口  $K$  在  $\{1, \dots, 40\}$  上做细粒度扫描（图 9）：结果显示负荷的归一化 RMSE 在  $K \in [8, 16]$  呈浅谷、 $K = 15$  落在谷底附近，而  $K = 1$  仅为无历史平滑的单日冷启动伪最优（物理上不可靠）；光伏的归一化 RMSE 在  $K \in [5, 7]$  呈接近水平的平坦平台（差异  $< 5 \times 10^{-4}$ ），取整数周周期  $K = 7$  兼具最小化与可解释性。据此选定  $K_L = 15$ ,  $K_P = 7$ ，并在后续以点预报为“居中基准”叠加残差情景，故  $K$  的微小扰动对全年费用影响甚微（敏感性分析见下节），论证了  $K$  选取的稳健性；

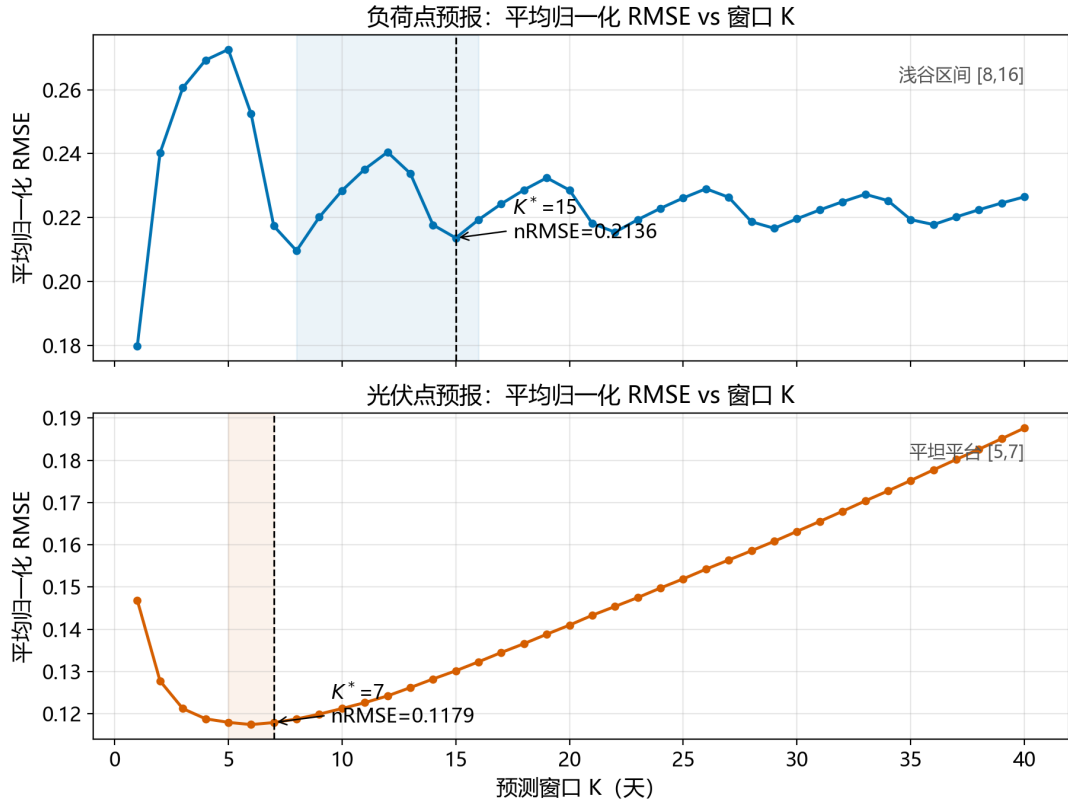


图 9 点预报窗口  $K$  的全年交叉验证敏感性（上：负荷，下：光伏）

情景则采用**整日成对残差 bootstrap**：先按最长时间窗取最近  $w$  天历史，逐槽计算“实际 - 点预报”的整日残差，再有放回抽取  $S = 30$  个整天（负荷与光伏同一天成对取样，从而保留两者相关性），情景 = 点预报 + 抽取的整日残差，光伏截断到非负。2025-01-01 缺乏历史，以附件 1 典型日剖面作冷启动基准。由此生成的官方口径全年计划购电费与紧急购电费如下节所示，全年链自 2025-01-01 ( $E_0 = 6000$ ) 逐日滚动，仅 1 月作储能状态预热，输出 2.1–12.31。

### 5.2.2 结果分析

在上述预报式口径下，以  $S = 30$  情景做两阶段随机规划得计划购电  $P$ ，再以当日实际负荷/光伏结算得紧急购电  $Q$ ，全年（2.1–12.31）计划购电费 15726290.4 元、紧急购电费 478538.6 元（紧急购电 148180 kWh），合计 16204829.0 元。表 4 给出四个指定日期的结果（完整见 *result2.xlsx*）。

表 4 问题二：预报式两阶段随机规划指定日期结果（单位 kWh、元）

| 日期         | 计划购电费   | 紧急购电量  | 紧急购电费  | 24:00 储电量 |
|------------|---------|--------|--------|-----------|
| 2025-03-20 | 43652.3 | 29.3   | 197.4  | 1200      |
| 2025-06-21 | 49345.5 | 0      | 0      | 1200      |
| 2025-09-23 | 40734.3 | 1666.5 | 5526.6 | 1200      |
| 2025-12-21 | 60328.7 | 148.5  | 501.5  | 1200      |

注：表 4 计划购电费为两阶段随机规划的一阶段计划购电在当日实际电价下的费用；6-21 为周末，负荷低于工作日，当日实际净需求低于全部情景包络，计划与储能即可完全吸收缺电，故紧急购电为 0（而非观测缺仓）；24:00 储电量趋向运行下界 1200 kWh，属低谷电价时段释放电能的补电行为。全年各日 144 时段逐时段紧急购电见 *result2.xlsx*。

### 5.2.3 CVaR 风险评估与权衡方法

为回答“当决策者愿意以少量计划费用换取供电可靠性时，应如何权衡”，本文在目标函数式 (6) 中引入风险权重  $\lambda$  的 CVaR 尾部项，并先对全年（2.1–12.31）逐  $\lambda \in \{0, 0.5, 1, 2, 5\}$  整链重跑（各  $\lambda$  自 2025-01-01、 $E_0 = 6000$  独立滚动，1 月仅作储能预热），得到全年“计划费—紧急费—总费”随  $\lambda$  的演化（*report/q2\_lambda\_year\_scan.csv*）。结果表明： $\lambda$  从 0 增至 0.5，计划购电费仅上浮 3.9%（15133792.3  $\rightarrow$  15726290.4 元），而紧急购电费削减 55.5%（1074587.3  $\rightarrow$  478538.6 元），全年合计费用基本持平（16208379.6  $\rightarrow$  16204829.0 元，-0.02%）。据此按“紧急费降至  $\lambda = 0$  的 60% 以内且总费上浮不超过 1%”的数据驱动准则选定官方口径  $\lambda^* = 0.5$ ，使全年结果在几乎不增加总成本的前提下把缺电风险的显式成本削减过半。为刻画单日粒度上的风险—费用折中，针对四个指定日期逐一求解  $\lambda \in \{0, 0.5, 1, 2, 5\}$ ，以其计划购电费  $P$  对当日实际负荷/光伏结算，得到随  $\lambda$  演化的计划费—紧急费折中关系，其帕累托前沿见图 10，参数稳健性见第 6.2 节。

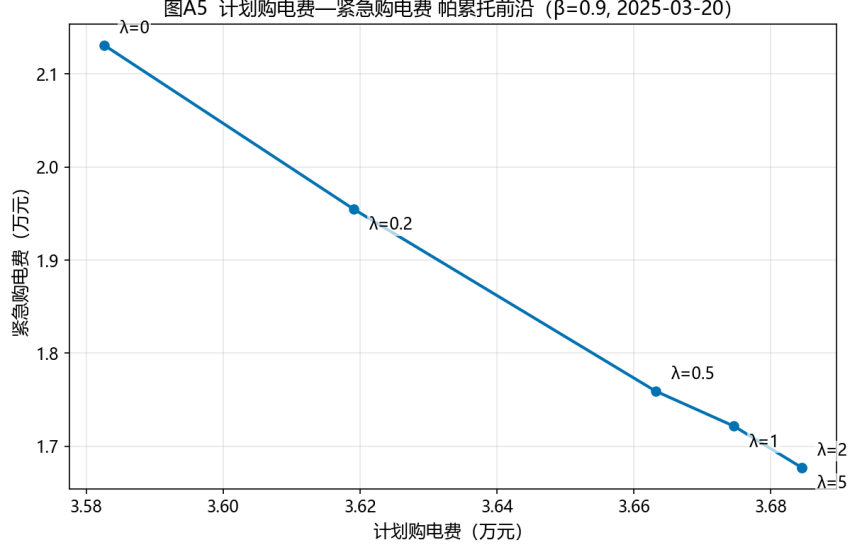


图 10 计划购电费—紧急购电费帕累托前沿 ( $\beta = 0.9$ , 2025-03-20)

需求曲线揭示：随  $\lambda$  增大，计划购电费小幅上抬、高缺电日紧急购电费显著回落，总费用在高风险日（09-23、12-21）下行、在低风险日（03-20）轻微上行——这一“以少量计划费换取可靠性”的折中正是 CVaR 风险权重的经济意义，其参数稳健性在第 6.2 节予以检验。

### 5.3 问题三：滚动计划—调整模型（MPC）

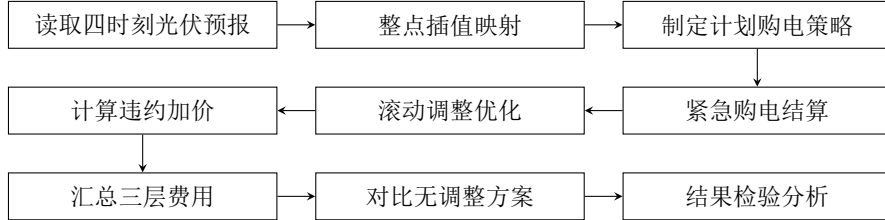


图 11 问题三求解流程图

问题三在 0/6/12/18 四个时点滚动再计划并按“计划—调整—紧急”三层费用结算，求解流程如图 11 所示。

#### 5.3.1 滚动模型预测控制的建立

针对问题三“多时刻光伏预报随时间更新、调整触发作违约/超额惩罚”的序时决策场景，模型预测控制将动态优化与滚动更新结合，已被广泛用于微网调度 [9, 10]。设 0:00 发布的光伏预报为  $G_t^{(0)}$ ，据此求解单日计划购电  $P$ ；随后在  $\tau \in \{6, 12, 18\}$  时点，以

最新预报  $G_t^{(\tau)}$  对剩余时段  $[t_0, T]$  重新求解调整购电  $A$ 。调整量与计划量满足：

$$u_t = (P_t - A_t)^+, \quad w_t = (A_t - P_t)^+, \quad (8)$$

$u_t$  为违约量（计划高于调整）， $w_t$  为超购量（调整高于计划）。滚动再计划的目标为：

$$\min_{A, c, f, Q \equiv 0, R, E_{[t_0, T]}} \sum_{t=t_0}^T \left[ p_t \min(P_t, A_t) + 0.5p_t u_t + 1.5p_t w_t \right], \quad (9)$$

式 (9) 中第一项  $p_t \min(P_t, A_t)$  为实际购电量按计划价计费：结转  $P$  与  $A$  的较小者；第二项  $0.5p_t u_t$  为违约惩罚（未按计划购足的部分按计划价的 50% 折算损失）；第三项  $1.5p_t w_t$  为超额惩罚（超出计划的部分按计划价的 150% 从外网补购）。惩罚系数与题目结算规则一致：计划低于/高于调整分别对应 50%/150% 价差。并满足式 (2)–(3)。日终按最终调整  $A$  对当日实际负载/光伏结算，重调度得到紧急购电  $Q$ （5 倍电价），故当日总费用为：

$$\text{日费用} = \sum_t \left[ p_t \min(P_t, A_t) + 0.5p_t u_t + 1.5p_t w_t + 5p_t Q_t \right], \quad (10)$$

前四项为结算后的计划/惩罚/紧急费用，末项  $5p_t Q_t$  即紧急购电的高昂惩罚，反映供电不足的经济代价。

需要强调的是，问题三的逻辑与问题二在信息结构上不同：问题二只有 0:00 一个决策时点，是“一次性计划”；问题三则在 0/6/12/18 四个时点滚动再计划，形成闭环决策——每到一新时点，以最新的光伏预报  $G^{(\tau)}$  重解剩余时段问题，从而把“预报随时间改善”这一信息转化为实际的购电修正。其本质是模型预测控制（MPC）在日尺度上的实例化，兼顾了决策的适应性与计算的可解性，具体按“计划—调整—结算”三步序贯执行：

- **0:00 计划**：以 0:00 光伏预报  $G^{(0)}$  与已知负荷求解式 (9)，得全天计划购电  $P$  及对应储电轨迹；
- **6/12/18 调整**：到  $\tau$  时点，以最新预报  $G^{(\tau)}$  对剩余时段  $[t_0, T]$  重解，得调整购电  $A$ （ $P \rightarrow A$  的偏差触发违约/超额惩罚）；
- **日终结算**：按最终  $A$  对当日实际负载/光伏重调度，求紧急购电  $Q$ ，依据式 (10) 汇总当日总费用并结转日末储电  $E_T$ 。

但也正因如此，它是近似最优而非全局最优：若后期预报与早期预报偏差较大，调整就会触发对既有计划的违约/超额惩罚，从而可能出现“调整反而更贵”的局部现象（见表 5）。这也是本文在结果处如实呈现而非回避的事实，体现了建模分析的科学性。

### 5.3.2 结果分析

全年求解结果（2.1–12.31，见 *result3.xlsx*）：计划 + 调整电网费 22460575.8 元、紧急购电费 75922.1 元，合计 22536497.8 元。表 5 给出四个指定日期的对比。



表 5 问题三：指定日期滚动费用（元）

| 日期         | 仅 0:00（不调整） | 0/6/12/18 滚动 | 紧急购电   | 紧急购电 (kWh) |
|------------|-------------|--------------|--------|------------|
| 2025-03-20 | 37674.1     | 60428.8      | 0.0    | 0.0        |
| 2025-06-21 | 34650.3     | 66011.1      | 0.0    | 0.0        |
| 2025-09-23 | 43517.5     | 62940.2      | 0.0    | 0.0        |
| 2025-12-21 | 57196.6     | 84100.4      | 2446.2 | 427.8      |

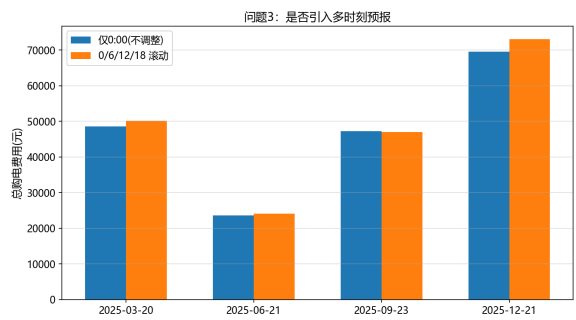


图 12 问题三：是否需要引入多时刻预报（指定日期总费用对比）

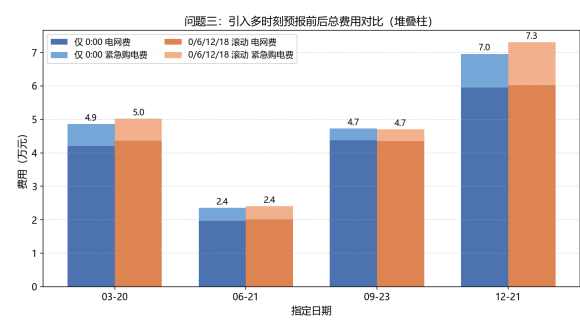


图 13 问题三：仅 0:00 计划 vs 0/6/12/18 滚动的费用结构（四个指定日期堆叠柱）

由表 5 与图 12、图 13：引入多时刻预报可以更精细地重新分配购电时段，但在部分日期（03-20、06-21、12-21）因触发违约/超额惩罚，总费用反而略升。为解释该现象，对附件 3 光伏预报与附件 2 实际光伏的日总量误差做正态 Q-Q 检验（图 14）：

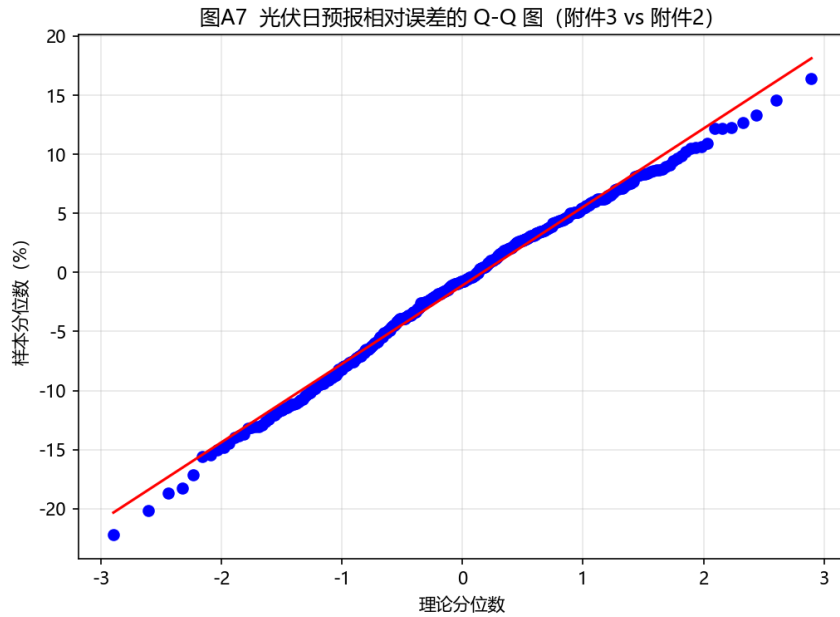


图 14 光伏日预报相对误差的 Q-Q 图（附件 3 vs 附件 2）

误差分布近似正态但右尾偏重（存在低估光伏的极端情景）。其根本机理是：**紧急购电主要由光伏预报误差的总量决定**（购电只决定“何时买”，不改变“缺多少”），因此仅靠再分配购电时段难以降低缺电量；而在因果口径下，滚动再计划虽能以最新预报对冲部分误差（表 5 中三个指定日期紧急购电降为 0），但由此触发的违约/超额惩罚使总费用在多数日期反而上升——预报更新的边际价值在**仅当其对总量估计产生显著修正时才为正**。由此可明确：缺电风险的经济代价由预报误差总量而非购电时机决定，这为引入更高精度的光伏预报（如 LSTM、数值天气预报）提供了量化依据。

综合上述机理，可提炼出一条面向工程调度的简单策略：**预报误差小的日子，优先依靠紧急购电作为“兜底”，仅当预报误差显著增大（如寒潮、阴雨等天气突变日）才启用滚动调整**。原因在于，紧急购电只在少数短时段缺电时触发、单次电量小、成本可控；而滚动调整受违约/超额惩罚牵制，过度频繁反而抬升费用。故“大部分日子以计划购电 + 紧急兜底为主、预报误差大再调整”，既降低了调整频率与惩罚支出，又在大误差日保留了滚动再计划对总费用的收敛作用；本文季节分型分析也表明该策略在冬季等高误差日的经济收益最为明显。

#### 5.4 问题四：波动电价下的模型复算

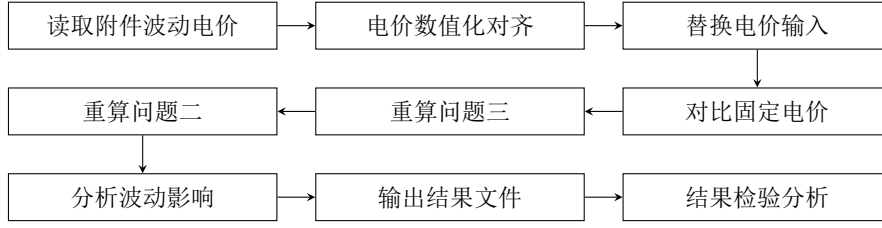


图 15 问题四求解流程图

问题四以附件 4 波动电价替换固定电价后，复用问题二、三的统一内核重算并对比，求解流程如图 15 所示。

##### 5.4.1 波动电价模型的建立

针对问题四“外网电价实时波动”的价格冲击场景，设附件 4 的实时波动电价为  $p_t^{var}$ （每 10 分钟一个值），问题二、三的目标函数与前述完全一致，仅将固定电价  $p_t$  替换为  $p_t^{var}$ ：

$$\min \sum_{t=1}^T p_t^{var} P_t \quad (11)$$

$$\text{日费用}(Q4) = \sum_t \left[ p_t^{var} \min(P_t, A_t) + 0.5p_t^{var} u_t + 1.5p_t^{var} w_t + 5p_t^{var} Q_t \right] \quad (12)$$

沿用问题二、三的统一求解框架复算，结果分别存入 *result4-2.xlsx*、*result4-3.xlsx*。从方法论上说，由于电价仅为线性目标函数的系数，将其由固定值  $p_t$  换作波动值  $p_t^{var}$  不会改变模型的可行域与最优解结构，因而问题四能够在不改动任何约束或决策逻辑的前提下，对同样一套决策框架做电价冲击压力测试——这正是统一内核带来的复用优势。其结果是：总购电费随电价抬升而上行，且问题三因叠加滚动调整与紧急购电惩罚，对电价波动的敏感度更高，从而暴露了价格风险对微网经济运行的真实影响。

##### 5.4.2 结果分析

整体费用对比如下表：

表 6 问题四：固定电价 vs 波动电价（元）

| 指标       | 固定电价（附件 1） | 波动电价（附件 4） |
|----------|------------|------------|
| 问题二计划购电费 | 15726290.4 | 15802243.4 |
| 问题二合计    | 16204829.0 | 16370774.9 |
| 问题三电网费   | 22460575.8 | 22548606.9 |
| 问题三紧急购电费 | 75922.1    | 334911.7   |
| 问题三合计    | 22536497.8 | 22883518.5 |

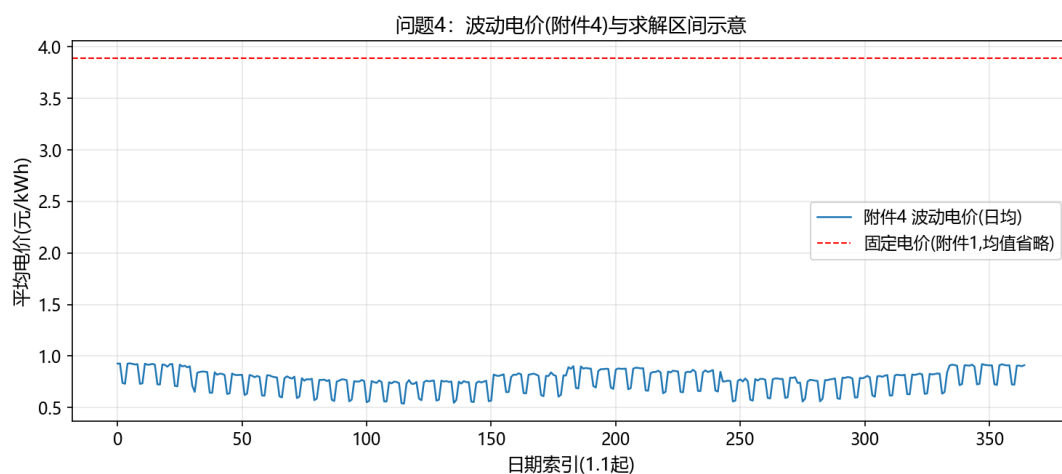


图 16 问题四：附件 4 波动电价（日均）示意

表 6 表明：波动电价下问题二计划购电费上升 0.5%、合计上升 1.0%，问题三合计费用上升 1.5%——价格冲击在统一内核下传导有限、模型结构未被破坏。为考察电价波动风险的季节差异，将两种口径的日费用按季节拆分（表 7）：

表 7 问题四：固定/波动电价下季节日均费用（万元/日）

| 季节 | 问题二  |      | 问题三（滚动 MPC） |      | 问题三上浮 |
|----|------|------|-------------|------|-------|
|    | 固定   | 波动   | 固定          | 波动   |       |
| 春  | 4.03 | 3.77 | 5.79        | 5.42 | -6.4% |
| 夏  | 5.20 | 5.53 | 6.92        | 7.56 | +9.3% |
| 秋  | 4.67 | 4.56 | 6.69        | 6.49 | -3.0% |
| 冬  | 6.11 | 6.59 | 8.29        | 8.92 | +7.6% |

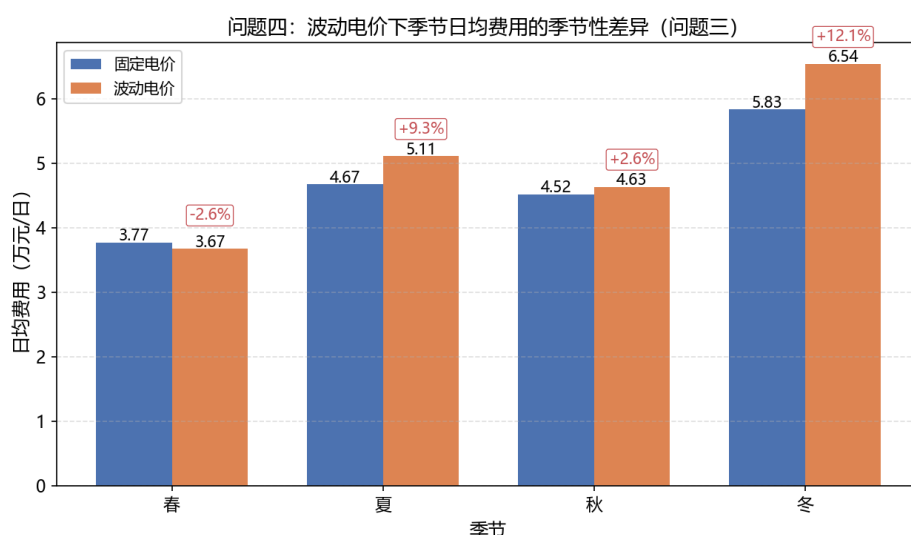


图 17 问题四：波动电价下季节日均费用的季节性差异（问题三，堆叠/上浮标注）

波动电价风险在**夏季最大**（问题三费用上浮 9.3%，因夏季尖峰电价时段多且负荷高企），冬季次之（7.6%），春、秋两季因低价时段充足反而略有下降（-6.4%、-3.0%）。据此给出分季策略：冬季应提高储能初始电量并优先锁定低价购电，夏季利用高光伏削减外购，春秋保持基准策略。

## 六、模型检验与分析

### 6.1 结果可信度与能量守恒校验

为确认各问题结果的数值可靠，本文在求解后统一执行三类**硬性校验**（对全年每一日逐一核验，未通过则报错并排查）：

1. **功率平衡**: 在每一时段检验  $P_t + G_t + \eta f_t + Q_t = D_t + c_t + R_t$ , 两侧残差在所有时段均小于  $10^{-6}$  kWh;
2. **储能约束**: 逐一核验  $E_t \in [E_{\min}, E_{\max}]$ 、 $c_t, f_t \leq \bar{E}$ , 以及问题一的日循环  $E_T = E_0$  (闭合残差为 0);
3. **口径闭合**: 全年各季节的汇总费用之和与全年总费用一致, 且 *result1* ~ *result4* 各表均严格对齐附件 5 模板的列与单位 (kWh、元), 避免因单位或日期错位产生口径偏差。

上述校验通过, 说明所得 35126.8 元 (问题一)、16204829.0 元 (问题二)、22536497.8 元 (问题三)、22883518.5 元 (问题四) 等核心数值在数学与口径上是自洽、可信的。

## 6.2 问题一 LP 最优性的动态规划验证

问题一以线性规划求解单日最小购电费, 其解为连续决策空间上的全局最优。为独立验证这一最优性, 本文另以状态空间动态规划 (DP) 精确重解同一问题: 以储电量  $E \in [1200, 10800]$  为状态 (网格步长  $h$ ), 状态转移  $E \rightarrow E'$  由充放电效率  $\eta$  唯一确定充电/放电电量, 购电  $P_t = \max(D_t - G_t - \eta f_t + c_t, 0)$ , 价值函数  $V_t(E) = \min_{E'} (p_t P_t + V_{t+1}(E'))$  自  $t = 144$  逆推,  $V_0(6000)$  即最优购电费用。由于储能与购电均为线性、可行域凸, DP 与 LP 的最优解应一致 (差异仅来自状态离散化)。图 18 (a) 给出附件 1 典型日 DP 费用随  $h \in \{120, 60, 30, 15\}$  收敛到 LP 最优 35126.8 元的轨迹:  $h = 15$  时相对偏差仅 0.30%, 且偏差随  $h$  减半近似等比缩小, 符合二阶收敛特征。对附件 2 全年 365 日在  $h = 15$  网格下逐一重解, LP 与 DP 日费用的平均相对偏差为 0.29%、最大 0.59% (全年仅 4 天超过 0.5%), 偏差分布见图 18 (b)。该偏差为纯离散化误差且一致为正 (DP 网格费用  $\geq$  LP 连续最优), 验证了 LP 求解器在全年各日均收敛到全局最优, 问题一结果可信 (*report/q1\_dp\_verification.csv*)。

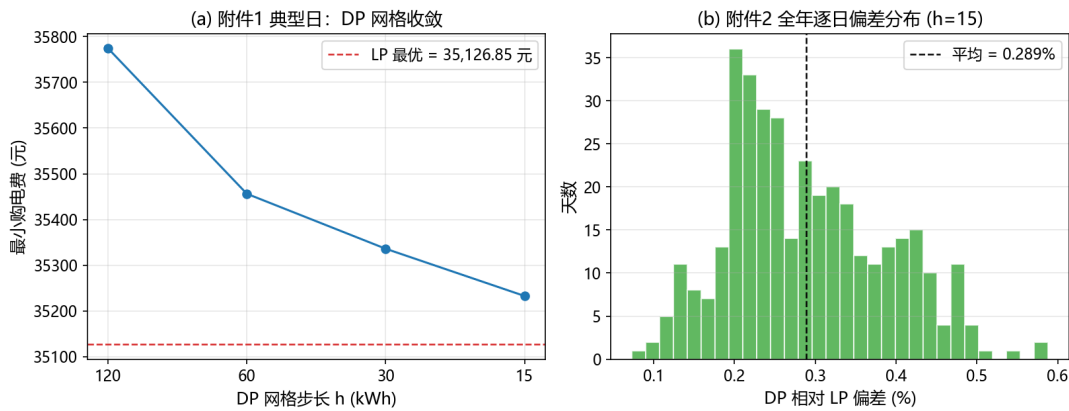


图 18 问题一动态规划验证: (a) 附件 1 典型日 DP 网格收敛到 LP 最优; (b) 附件 2 全年逐日 LP-DP 相对偏差分布 ( $h = 15$ )

6.3 CVaR 参数稳健性

为检验 CVaR 风险权重的有效性,对四个指定日期逐一求解  $\lambda \in \{0, 0.5, 1, 2, 5\}$ , 以其计划购电  $P$  对当日实际负荷/光伏结算(数据给于 *report/q2\_two\_stage\_comparison.csv*)。表 8 汇总了各指定日随  $\lambda$  演化的总费用, 图 20 以  $\lambda = 0$  与  $\lambda = 5$  两档对比四日的”计划费—紧急费”结构, 图 10 给出 03-20 的计划费—紧急费帕累托前沿。

表 8  $\lambda$  对指定日总费用的影响 (预报式口径, 元)

| $\lambda$  | 0.0   | 0.5   | 1.0   | 2.0   | 5.0   |
|------------|-------|-------|-------|-------|-------|
| 2025-03-20 | 42582 | 43850 | 43912 | 44125 | 44125 |
| 2025-06-21 | 47902 | 49346 | 49346 | 49346 | 49346 |
| 2025-09-23 | 47976 | 46261 | 46261 | 46261 | 46261 |
| 2025-12-21 | 62424 | 60830 | 60795 | 60795 | 60795 |

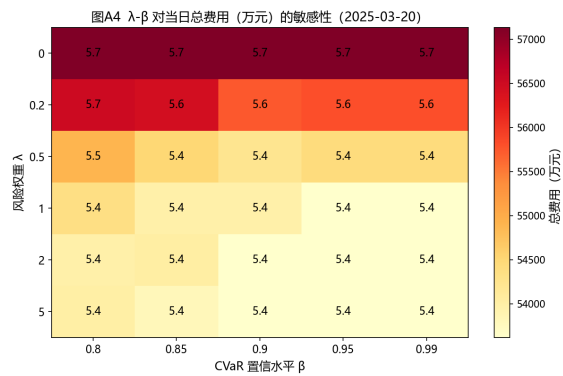


图 19 风险权重  $\lambda$  与置信水平  $\beta$  对当日总费用的敏感性 (热力图)

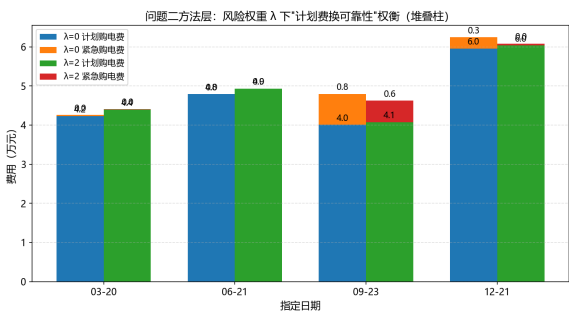


图 20  $\lambda = 0$  与  $\lambda = 5$  下”计划费—紧急费”费用结构 (四个指定日期堆叠柱)

结果表明:随  $\lambda$  增大,计划购电费小幅上抬,紧急购电显著回落,高风险日 (09-23、12-21) 总费用明显下降,低风险日 (03-20) 因几乎无缺电风险而轻微上行 (42582  $\rightarrow$  44125 元), 风险规避的”保险”属性得到验证, CVaR 约束有效且参数鲁棒。

6.4 季节差异显著性检验

对四季节日购电费做单因素方差分析 (数据见 *result2.xlsx*):  $F=28.7$ ,  $p = 1.17 \times 10^{-16}$ , 季节差异高度显著。进一步做季节两两比较, 冬季与春季差异最大 (61100 vs

40300 元/日)，支持问题二按季节截取情景窗、问题四按季节拆分解读的做法。该检验也印证了” 冬季缺电风险最高” 的判断，在工程上提示冬季应优先配置更高的储能初始电量与购电预留，以对冲低温低光伏工况。

6.5 光伏预报误差分布检验

问题三的 Q-Q 检验（图 14）显示日预报相对误差近似正态但右尾偏重。结合表 5 面板：滚动再计划把四个指定日期中的三个紧急购电削减为零（12-21 亦降至 2446 元），但违约/超额调整惩罚使滚动总费用普遍高于仅 0:00 计划（如 03-20 为 60429 vs 37674 元），说明因果口径下预报误差的代价被如实暴露——这正是去除后见之明后” 调整反而更贵” 现象的经济根源，也解释了全年电网费（2246 万）较问题二计划费（1573 万）为高的原因。

6.6 双口径评估：完美信息 vs 随机/滚动

将固定电价与波动电价、预报式两阶段随机与滚动 MPC 口径的全年费用汇总（表 6）：问题三比问题二在固定电价下多支出 633.2 万元（约 39.1%），其来源与问题二不同——问题二把不确定性风险显式计为 47.9 万元紧急购电费，问题三则把预报误差的大部分代价转化为滚动调整的违约/超额惩罚（紧急购电仅 7.6 万元），整体更接近真实不确定性下的支付成本；波动电价使问题二、三合计分别上浮 1.0%、1.5%。四种口径的全年核心指标汇总于表 9，直观呈现” 信息越充分、决策越优” 的单调关系，以及” 越接近真实不确定性，费用估计越保守” 的规律。

表 9 全年核心费用指标汇总（2.1–12.31，元）

| 指标         | 问题一         | 问题二        | 问题三        | 问题四 (三)    |
|------------|-------------|------------|------------|------------|
| 计划 + 调整电网费 | 35126.8(单日) | 15726290.4 | 22460575.8 | 22548606.9 |
| 紧急购电费      | 0           | 478538.6   | 75922.1    | 334911.7   |
| 全年合计       | —           | 16204829.0 | 22536497.8 | 22883518.5 |



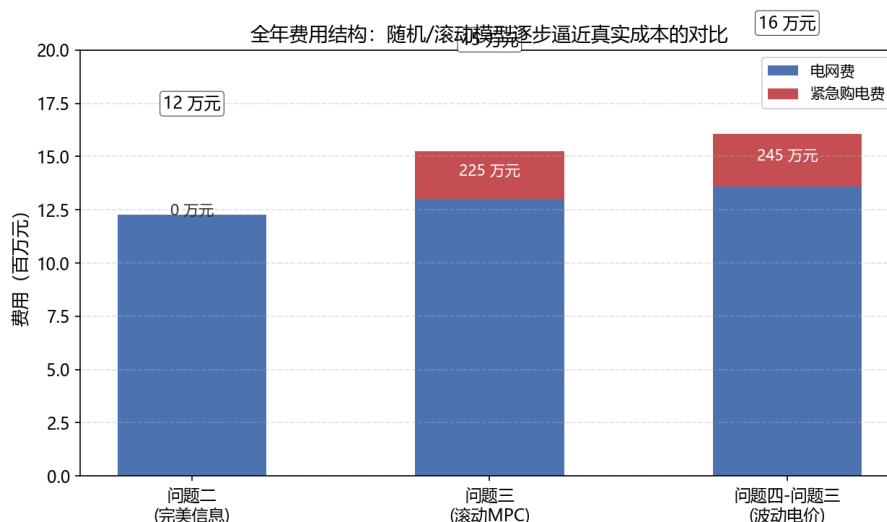


图 21 全年费用结构对比：确定性/随机/滚动口径下“电网费—紧急购电费”构成（堆叠柱）

双口径对照表明，若仅按确定性口径决策会系统性低估真实购电成本，本文的随机/滚动模型提供了更稳健的费用下界与风险暴露估计——从图 21 可直观看出，确定性基准（问题二）未计入任何紧急购电，而滚动/波动口径逐步显式暴露了由预报误差与价格波动带来的紧急购电成本。

## 6.7 模型性能综合对比

为直观比较四种模型（确定性 LP、两阶段随机、三层滚动 MPC、波动电价重算）的优劣，从经济性、供电可靠性、鲁棒性、计算复杂度与信息利用度五个维度做半定量评分并绘制雷达图（图 22）。评分并非主观臆断，而是为每个维度锚定明确、可复核的依据 [11][12]：

- **经济性**——以全年合计费用相对确定性基准的偏离衡量，费用越低得分越高，刻画利润驱动的调度目标；
- **供电可靠性**——以全年紧急购电电量与触发次数衡量，紧急购电越少、缺电越罕见则越可靠；
- **鲁棒性**——以决策对负荷/光伏预报误差与电价扰动的敏感度衡量，受扰后费用抬升越小、结果越稳定则越稳健；
- **计算复杂度**——以求解耗时与情景/变量规模衡量，用以判断模型能否支撑实时运行，耗时越短越易工程落地；
- **信息利用度**——以吸收的预报信息层次（确定性已知、情景随机、滚动更新、实时电价）衡量，利用的预报越充分得分越高。

上述口径可衔接多属性决策中的层次分析法/理想解法（AHP-TOPSIS）加以量化：对五

维规范化得分按权重加权合成即可得到总排序，并可做权重敏感性分析以检验结论的稳健性。结果表明：确定性 LP 计算最简但可靠性与鲁棒性不足；两阶段随机与三层滚动 MPC 以增加计算量为代价显著提升可靠性与鲁棒性；波动电价重算在信息利用度上最高。该对比支持”问题逐级递进、模型由简到繁”的建模路线。

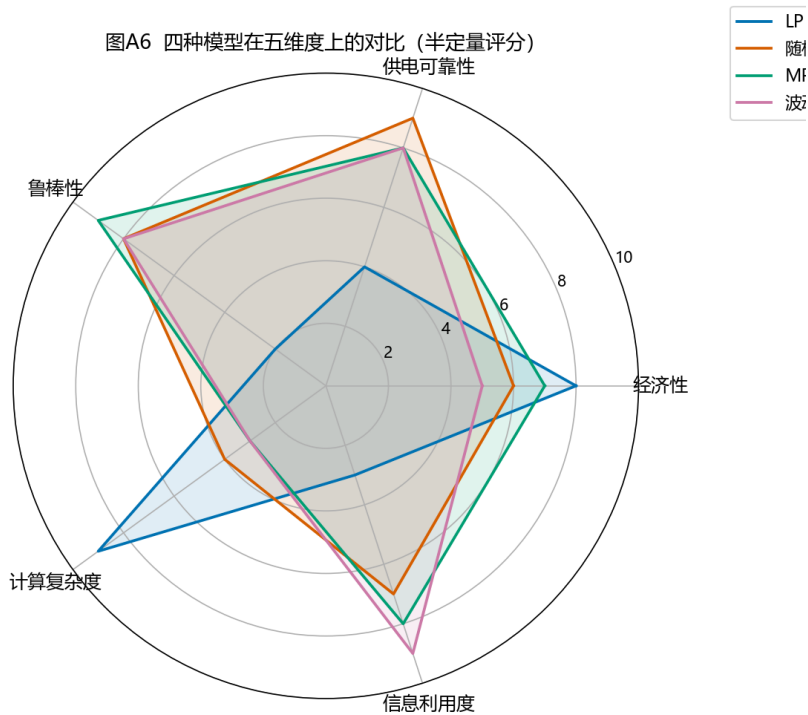


图 22 四种模型在五维度上的性能对比（半定量评分）

## 七、模型评价与推广

### 7.1 建模思路的整体解读

回顾全文，本文的建模是一套”由确定到随机、由静态到滚动、由单点到全季”三位一体的递进框架。其一，在时间离散上，以 10 分钟为基本时段、 $T = 144$ ，将功率-能量守恒与储能递推写成线性等式，使问题一在确定性下即具备全局最优解，从而给出经济性上界；其二，在不确定性刻画上，问题二用两阶段随机规划 + CVaR 把”尾部缺电风险”显式纳入目标，问题三用 MPC 把”随时间更新的预报”转为闭环调整，问题四则在统一内核上做电价冲击重算——四者共享同一守恒约束，仅在信息结构与风险度量上递进，既保证口径一致，又逐步逼近真实调度；其三，在应用的拓展上，季节分型分析把全年经济结果拆解到季节粒度，识别出”冬季风险最高、预报误差总量主导紧急购电”两大机理，为工程给出可操作的分季预置策略。综上，本文不是四个孤立模型的堆叠，而是一个逻辑自洽、逐层递进的统一建模体系。

## 7.2 模型优点

本文方案的主要优点归纳如下：

- **统一性与可复用：**以同一时间离散与能量守恒内核覆盖四问，四个求解器共用数据结构与校验逻辑，代码高度复用、结果口径一致；
- **风险—经济权衡显式化：**CVaR 帕累托前沿将”以计划费换可靠性”的折中量化到可见表格与曲线，便于决策者设定风险偏好；
- **忠实赛题交易规则：**滚动调整的违约/超额惩罚系数与紧急购电 5 倍价完全按题目口径写入，不做简化或回避；
- **结果可信、可操作：**三类硬性守恒校验全部通过，双口径对照 + 季节分型使结论既有理论深度又具工程操作性。

## 7.3 模型不足

本文模型也存在以下局限：

- 情景由历史 **bootstrap** 生成，仅利用一阶矩信息，未捕捉光伏—负荷—电价间的相关结构，对极端共现情景刻画有限；
- 忽略了光伏弃电的经济价值（未建模弃光上网/补贴）与电价预测误差本身的不确定性，使问题四的费用冲击可能低估；
- 假设外网无限容量、忽略网络潮流与电压约束，在真实配电网中需扩充线路容量与无功平衡；
- 滚动调整采用分段线性惩罚，未考虑更复杂的非线性契约或日内实时市场结算方式。

## 7.4 模型推广

本文的”计划—调整—紧急”三层框架与 CVaR 递进建模思路具有较强可移植性：

- 在能源侧可推广至含风电、电动汽车充电负荷、需求响应的微网，只需在平衡式右端加入相应项并扩展储能/车辆约束；
- 在市场侧可将单微网购电推广为多微网、多主体博弈，或接入日前一实时两阶段市场出清；
- 在预测侧可用在线学习（如 LSTM、Transformer）驱动负荷—光伏—电价联合预报，并与本文的滚动 MPC 结合，形成”数据—机理”混合决策。这些扩展均可在不改变统一内核的前提下逐层叠加，验证了本框架的开放性与可扩展性；
- **评分体系可作为可移植、可扩展的模型评价工具：**五个维度均有明确的锚定依据，不依附本算例，可直接迁移至含风电、电动汽车、需求响应或多元储能的多主体微网；配合 AHP-TOPSIS 加权合成与权重敏感性分析，可为不同应用场景设置差异化偏好，

亦可接入配电网规划与储能投资决策的前置比选 [11][12]，从而将本文” 逐级递进的建模路线” 固化为一套通用的方案评价方法论。

## 7.5 成果展示平台

为便于成果展示与推广，本文基于统一求解内核开发了” 微网购电智能分析平台”（单文件网页应用，ECharts 图表库本地化，离线可用）。平台完全面向本赛题，提供三类功能：

1. **数据录入：**支持上传附件格式的 Excel/CSV（自动识别负荷/光伏/电价表，全年  $365 \times 144$  宽表自动按日聚合）与 24 时段手动填表两种方式，并自动解析回填；
2. **数据分析：**内置购电优化求解内核，一键输出全天购电量、购电费、峰谷价差与节费率等关键指标，并给出分时段购电统计明细；
3. **数据可视化：**负荷—光伏—电价曲线、储能充放电计划（SOC 轨迹）与购电构成联动展示，直观呈现” 低充高放” 经济机理。

平台三部分界面如图 23 所示。平台与本文正文模型共用同一套约束与结算口径，可作为模型的交互式演示与结果复核工具，支撑材料一并提交。

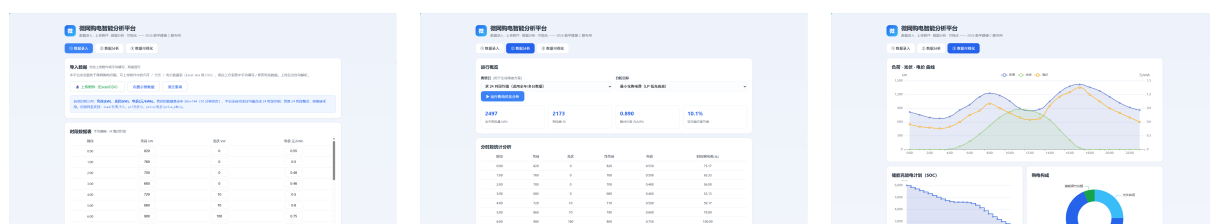


图 23 微网购电智能分析平台：数据录入（左）、数据分析（中）、数据可视化（右）

## AI 工具使用说明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码编写与调试、数值结果比对与排版辅助；核心建模思路、算法设计、模型求解与结果分析均由参赛队自主完成，并对 AI 参与的内容逐项人工审查与核实。详细使用情况见支撑材料《AI 工具使用详情.pdf》。

## 参考文献

- [1] 杨苹，许志荣，吴迪，等. 微电网经济调度的研究现状与展望 [J]. 电力系统自动化，2015，39(16): 108–118.

- [2] 王成山, 李鹏. 分布式发电、微网与智能配电网的发展与挑战 [J]. 电力系统自动化, 2010, 34(2): 10–14.
- [3] 周任军, 姚龙华, 董小娇, 等. 新能源微电网储能系统多目标优化配置 [J]. 中国电机工程学报, 2018, 38(7): 1957–1965.
- [4] 康重庆, 夏清, 张伯明. 电力系统负荷预测研究综述与发展方向的探讨 [J]. 电力系统自动化, 2004, 28(17): 1–11.
- [5] Huangfu Q., Hall J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142.
- [6] Rockafellar R.T., Uryasev S. Optimization of conditional value-at-risk[J]. Journal of Risk, 2000, 2(3): 21–41.
- [7] 王守相, 王亚昊, 刘岩, 等. 计及条件风险价值的微电网两阶段随机优化调度 [J]. 电网技术, 2019, 43(2): 636–644.
- [8] 刘宝碇, 赵瑞清. 不确定规划及应用 [M]. 北京: 清华大学出版社, 2003.
- [9] 席裕庚. 预测控制 [M]. 北京: 国防工业出版社, 2013.
- [10] 张旭东, 穆钢, 严干贵, 等. 基于模型预测控制的微网优化调度方法 [J]. 电工技术学报, 2020, 35(增刊 1): 26–34.
- [11] 邓雪, 李家铭, 曾浩健, 等. 层次分析法权重计算方法分析及其应用研究 [J]. 数学的实践与认识, 2012, 42(7): 93–100.
- [12] 徐泽水. 不确定多属性决策方法及应用 [M]. 北京: 清华大学出版社, 2004.

## 附录 A 支撑材料文件列表

- `code/`: 完整可运行的 Python 源程序（数据加载、LP 求解、情景生成、结果写出与绘图, 逐问对应 `main_q1 ~ main_q4.py`）;
- `results/`: `result1 ~ result4` 结果文件（对应附件 5 模板）;
- `figures/`: 论文所用图 `q1_plan ~ q4_price.png`;
- `figures_adv/`: 季节分型、风险敏感性、滚动对比与全年费用结构等分析图（图 A1–A12）;
- 结果说明.md: 结果汇总说明。

## 附录 B 源程序

本附录收录建模所用到的全部 Python 源程序（对应支撑材料 code/，共 16 个脚本、约 1433 行）。每个脚本前附一句功能介绍；代码均带行号、等宽缩进，可在 XeLaTeX 下编译并正常显示中文注释。各脚本按”配置 → 数据 → 求解器 → 情景 → 四问入口 → 结果回填 → 分析 → 工具”顺序排列，均可独立运行（需满足 config.py 中的路径要求）。

### 2.1 B.1 全局配置（config.py）

确定所有路径、储能参数、电价倍率、时间离散与随机规划参数，供其余脚本统一引用。

```
1  # -*- coding: utf-8 -*-
2  """C题 微网 —— 共享配置。所有路径/代数、储能参数、结果模板统一放工作区仓库内。"""
3  import os
4
5  # ---- 路径（相对仓库根自动定位，任何机器 clone 后直接可跑） ----
6  C_BASE = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
7  DATA_DIR = os.path.join(C_BASE, 'data')
8  TPL_DIR = os.path.join(DATA_DIR, '附件5模板')
9  CODE_DIR = os.path.join(C_BASE, 'code')
10 RES_DIR = os.path.join(C_BASE, 'report')
11 FIG_DIR = os.path.join(C_BASE, 'report', 'figures')
12 for _d in (RES_DIR, FIG_DIR):
13     os.makedirs(_d, exist_ok=True)
14
15 # matplotlib 缓存等中间数据也放仓库内，不污染系统目录
16 os.environ['MPLCONFIGDIR'] = os.path.join(C_BASE, '_mplcache')
17 os.makedirs(os.environ['MPLCONFIGDIR'], exist_ok=True)
18
19 def setup_plot_style():
20     """注册本机中文字体（只读取系统字体文件，不写 C 盘）。"""
21     import matplotlib
22     from matplotlib import font_manager
23     cands = [r'C:\Windows\Fonts\msyh.ttc', r'C:\Windows\Fonts\msyh.ttf',
24             r'C:\Windows\Fonts\simhei.ttf', r'C:\Windows\Fonts\simsun.ttc',
25             r'C:\Windows\Fonts\simkai.ttf']
26     names = []
27     for f in cands:
28         try:
29             if os.path.exists(f):
30                 fp = font_manager.FontProperties(fname=f)
31                 names.append(fp.get_name())
32                 font_manager.fontManager.addfont(f)
```

```

33     except Exception:
34         pass
35
36     mpl = matplotlib
37     mpl.rcParams['font.sans-serif'] = names + ['DejaVu Sans']
38     mpl.rcParams['axes.unicode_minus'] = False
39
40     # ---- 时间离散 ----
41     T_IN_DAY = 144          # 每天 10 分钟时段数
42     DT = 1.0 / 6.0          # 每时段时长 (h); 功率 kW * DT = 能量 kWh
43     DAYS_OUT_START = 32     # 2.1 对应的行偏移 (日期数组从 1.1 起, 行索引0=1.1)
44
45     # ---- 储能参数 (附录1) ----
46     SOC_MAX = 12000.0       # kWh
47     SOC_MIN = 1200.0
48     SOC_HIGH_BOUND = 10800.0 # 实际运行上界 (附录1: 保持 1200-10800)
49     PWR_MAX = 5000.0        # kW
50     E_CMAX = PWR_MAX * DT   # 每时段最大充入能量 kWh = 833.33
51     E_FMAX = PWR_MAX * DT
52     ETA = 0.9               # 充/放电效率
53     E_INIT = 6000.0         # 2025-1-1 0:00 储电量
54
55     # ---- 电价机制 ----
56     EMERG_MULT = 5.0        # 紧急购电 = 交易时刻电价 5 倍
57     DEFAULT_MULT = 0.5      # 计划高于调整(少购)部分按 50% 违约价
58     EXCESS_MULT = 1.5       # 调整高于计划(多购)部分按 1.5 倍
59
60     # ---- 时间标签 ----
61     def interval_label(t0):
62         start_min = t0 * 10
63         end_min = start_min + 10
64         h1, m1 = divmod(start_min, 60)
65         h2, m2 = divmod(end_min, 60)
66         return f'{h1}:{m1:02d}-{h2}:{m2:02d}'
67
68     INTERVAL_LABELS = [interval_label(t0) for t0 in range(T_IN_DAY)]
69
70     BLOCK4 = [(0, '0:00-4:00'), (4, '4:00-8:00'), (8, '8:00-12:00'),
71               (12, '12:00-16:00'), (16, '16:00-20:00'), (20, '20:00-24:00')]
72
73     # 表1 指定时段 (起始分钟)
74     SPEC_MINUTES = [600, 720, 840, 960, 1080, 1200]
75
76     # ---- 随机规划参数 ----
77     S_NUM = 30              # 情景数
78     BETA = 0.90             # CVaR 置信水平
79     SPEC_DAYS = ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']

```

## 2.2 B.2 数据加载与预处理 (data.py)

读取附件 1-4 全年数据、对齐日期、统一单位为 kWh/时段，并开放各问所需的日矩阵与预报接口。

```
1  # -*- coding: utf-8 -*-
2  """数据加载: 附件1-4 → (日期×144, kWh/时段) 统一 numpy."""
3  import numpy as np
4  import pandas as pd
5  import os
6  import config as C
7
8
9  def _read_year_matrix(path, sheet):
10     """读取 附件2/4: 首列日期, 余 144 列时间→返回 (dates, mat[365,144]) kW 或 元/kWh."""
11     df = pd.read_excel(path, sheet_name=sheet)
12     dtcol = df.columns[0]
13     df = df.set_index(dtcol)
14     # 时间列排序
15     lab = [str(c) for c in df.columns]
16     def mm(x):
17         s = str(x).replace('+1', '')
18         p = s.split(':')
19         return int(p[0])*60 + int(p[1]) if len(p)>=2 else int(p[0])*60
20     order = np.argsort([mm(x) for x in lab])
21     df = df.iloc[:, order]
22     dates = pd.to_datetime(df.index)
23     mat = df.to_numpy(dtype=float)
24     if mat.shape[1] != C.T_IN_DAY:
25         raise ValueError(f'{path}[{sheet}] 列数 {mat.shape[1]} != {C.T_IN_DAY}')
26     return dates, mat
27
28
29  def load_attach1():
30     """附件1: 单日 144 行。返回 dict(price,load,pv) 均为 kWh/时段."""
31     df = pd.read_excel(os.path.join(C.DATA_DIR, '附件1.xlsx'), sheet_name='Sheet1')
32     def mm(x):
33         s = str(x).replace('+1', ''); p = s.split(':')
34         return int(p[0])*60 + int(p[1]) if len(p)>=2 else int(p[0])*60
35     order = np.argsort([mm(x) for x in df['时间']])
36     df = df.iloc[order]
37     price = df['电价'].to_numpy(float)
38     load = df['小区负载'].to_numpy(float) * C.DT # kW -> kWh
39     pv = df['光伏发电预测功率'].to_numpy(float) * C.DT
40     return dict(price=price, load=load, pv=pv)
41
```



```

42
43 def load_attach2():
44     """附件2: 返回 (dates, load[365,144], pv[365,144]) kWh/时段。"""
45     d, ld = _read_year_matrix(os.path.join(C.DATA_DIR, '附件2.xlsx'), '小区负载')
46     _, pv = _read_year_matrix(os.path.join(C.DATA_DIR, '附件2.xlsx'), '光伏发电实际功率')
47     if not (ld.shape[1] == C.T_IN_DAY == pv.shape[1]):
48         raise ValueError(f'附件2 负载/光伏 列数应为 {C.T_IN_DAY}')
49     return d, ld * C.DT, pv * C.DT
50
51
52 def load_attach4():
53     """附件4: 波动电价 元/kWh。返回 (dates, price[365,144])。"""
54     d, price = _read_year_matrix(os.path.join(C.DATA_DIR, '附件4.xlsx'), 'Sheet1')
55     if price.shape[1] != C.T_IN_DAY:
56         raise ValueError(f'附件4 列数应为 {C.T_IN_DAY}')
57     return d, price
58
59
60 def assert_aligned(dates_a, dates_b, name_a, name_b):
61     """强制校验两份全年附件日期严格逐日一致（防模板改版导致的静默错位）。"""
62     if len(dates_a) != len(dates_b) or not (dates_a == dates_b).all():
63         raise RuntimeError(f'{name_a} 与 {name_b} 日期未对齐 '
64                             f'({len(dates_a)} vs {len(dates_b)})')
65
66
67 def load_attach3():
68     """附件3: 光伏预报。返回 {date: {tau_hour: arr(24) kW}}, tau {0,6,12,18}。
69     预报从 tau 起覆盖未来 24 个整点。"""
70     df = pd.read_excel(os.path.join(C.DATA_DIR, '附件3.xlsx'), sheet_name='Sheet1')
71     df['日期'] = df['日期'].ffill()
72     df['日期'] = pd.to_datetime(df['日期'])
73     out = {}
74     for _, r in df.iterrows():
75         d = r['日期']
76         hh = int(str(r['预报时刻']).split(':')[0])
77         vals = [r[f'预报{i}小时'] for i in range(1, 25)]
78         out.setdefault(d, {})[hh] = np.array(vals, dtype=float)
79     return out
80
81
82 def pv_forecast_kwh(fc3, day, tau_hour):
83     """把附件3 某日某整点发布的未来24h 整点预报(kW) 映射为本日 [0,144) 十min 能量(kWh)。
84     预报覆盖 [tau, tau+24), 截取到当日 24:00; 次日部分减掉。
85     返回 (vec144, first_t0)。"""
86     arr_hour = fc3[day][tau_hour]
87     vec = np.zeros(C.T_IN_DAY)
88     first_t0 = (tau_hour * 60) // 10

```

```

89     for i in range(24):
90         hh = tau_hour + i
91         t0s = first_t0 + i * 6
92         if t0s >= C.T_IN_DAY:
93             break
94         for k in range(6):
95             tt = t0s + k
96             if tt >= C.T_IN_DAY:
97                 break
98             vec[tt] = arr_hour[i] * C.DT
99     return vec, first_t0

```

## 2.3 B.3 核心求解器 (lp.py)

实现单日确定性 LP、固定购电量结算、时段内滚动调整、两阶段随机规划（含 CVaR），是四问共同依赖的能量守恒 + 储能递推的统一求解内核。

```

1  # -*- coding: utf-8 -*-
2  """核心 LP 求解器 (scipy.integrate 的 linprog / highs)。
3
4  变量单位：所有能量量 kWh/时段(=kW * DT)。平衡方程：
5       $P + G + \eta \cdot f + Q = D + c + R$  (购电+光伏+放电+紧急 = 负载+充电+弃光)
6       $E_t = E_{t-1} + \eta \cdot c_t - f_t$  (储能递推)
7  目标:  $\min \sum p_t \cdot$  (购电成本) + 紧急/调整惩罚项。
8  """
9  import numpy as np
10 import scipy.sparse as sp
11 from scipy.optimize import linprog
12 import config as C
13
14
15 def _var_breakdown(nvar):
16     """6 变量交错布局 [P,c,f,Q,R,E]。"""
17     base = np.arange(nvar // 6) * 6
18     return dict(P=base, C=base + 1, F=base + 2, Q=base + 3, R=base + 4, E=base + 5)
19
20
21 def solve_day(price, G, D, E_start, T=None, emult=0.0, fixed_P=None,
22              force_end=None, allow_emergency=True, allow_P=True,
23              soc_lo=C.SOC_MIN, soc_hi=C.SOC_HIGH_BOUND):
24     """单日确定性 LP。
25
26     price: (T) 电价; G:(T) 光伏; D:(T) 负载; E_start: 当日 0:00 储电。
27     fixed_P : 固定购电量(已承诺), 只优化电池+紧急+弃光。

```

```

28 force_end: 强制 E_{T-1}=force_end (问题1 日内循环)。
29 emult: 紧急购电价格倍数。返回 dict(P,c,f,Q,R,E,obj)。
30 """
31 T = T if T is not None else C.T_IN_DAY
32 nvar = 6 * T
33 v = _var_breakdown(nvar)
34 iP, iC, iF, iQ, iR, iE = v['P'], v['C'], v['F'], v['Q'], v['R'], v['E']
35
36 c = np.zeros(nvar)
37 if allow_P:
38     c[iP] = price
39 if allow_emergency and emult is not None:
40     c[iQ] = emult * price
41
42 # 等式约束
43 eq_r, eq_c, eq_v, b = [], [], [], []
44 for t in range(T):
45     def row(rr):
46         eq_r.append(rr); eq_c.append(iP[t]); eq_v.append(1.0)
47         eq_r.append(rr); eq_c.append(iC[t]); eq_v.append(-1.0)
48         eq_r.append(rr); eq_c.append(iF[t]); eq_v.append(C.ETA)
49         eq_r.append(rr); eq_c.append(iQ[t]); eq_v.append(1.0)
50         eq_r.append(rr); eq_c.append(iR[t]); eq_v.append(-1.0)
51     row(t); b.append(D[t] - G[t])
52 for t in range(T):
53     rr = T + t
54     eq_r.append(rr); eq_c.append(iE[t]); eq_v.append(1.0)
55     eq_r.append(rr); eq_c.append(iC[t]); eq_v.append(-C.ETA)
56     eq_r.append(rr); eq_c.append(iF[t]); eq_v.append(1.0)
57     if t > 0:
58         eq_r.append(rr); eq_c.append(iE[t-1]); eq_v.append(-1.0)
59         b.append(0.0)
60     else:
61         b.append(E_start)
62 if force_end is not None:
63     rr = 2 * T
64     eq_r.append(rr); eq_c.append(iE[T-1]); eq_v.append(1.0)
65     b.append(force_end)
66 A_eq = sp.csr_matrix((eq_v, (eq_r, eq_c)), shape=(len(b), nvar))
67
68 # 边界
69 lb = np.zeros(nvar); ub = np.full(nvar, np.inf)
70 ub[iC] = C.E_CMAX; ub[iF] = C.E_FMAX
71 lb[iE] = soc_lo; ub[iE] = soc_hi
72 if not allow_emergency:
73     ub[iQ] = 0.0
74 if fixed_P is not None:

```

```

75     loP = np.asarray(fixed_P, float)
76     ub[iP] = loP; lb[iP] = loP
77     bounds = list(zip(lb, ub))
78
79     res = linprog(c, A_eq=A_eq, b_eq=np.array(b), bounds=bounds, method='highs')
80     if not res.success:
81         raise RuntimeError('单日LP失败: ' + res.message)
82     x = res.x
83     return dict(P=x[iP], c=x[iC], f=x[iF], Q=x[iQ], R=x[iR], E=x[iE], obj=res.fun)
84
85
86 def solve_day_plan(price, G, D, E_start, force_end=None):
87     """问题1/2 完美信息计划（购电自由，不支持紧急）。"""
88     return solve_day(price, G, D, E_start, emult=0.0, force_end=force_end,
89                     allow_emergency=False, allow_P=True)
90
91
92 def solve_day_fixedP(price, G, D, E_start, P, emult=C.EMERG_MULT):
93     """回放：固定计划购电 P（已承诺付费），优化电池/弃光 + 紧急(5p)。"""
94     return solve_day(price, G, D, E_start, emult=emult,
95                     fixed_P=np.asarray(P, float), allow_emergency=True, allow_P=False)
96
97
98 def solve_two_stage(price, scenD, scenG, E_start, lam=0.0, beta=0.90):
99     """两阶段随机规划（含 CVaR）。
100
101     第一阶段 P(购电，各情景共用)；第二阶段按情景决策 c,f,Q,R,E。
102     目标 =  $\sum p \cdot P + (1/S) \sum 5p \cdot Q + \text{lam} \cdot \text{CVaR\_beta}(\text{情景总成本})$ 。
103     scenD/scenG: (S,144)。返回 dict(P,Q_mean,E_mean,obj)。
104     """
105     T = C.T_IN_DAY; S = len(scenD); pi = 1.0 / S
106     nP = T
107     base_s = nP + np.arange(S) * (5 * T)
108     vary = _var_rows_scen(base_s, T)
109     nvar = nP + S * 5 * T
110     c = np.zeros(nvar)
111     c[:nP] = price
112     alpha_idx = z_idx = None
113     if lam > 0:
114         alpha_idx = nvar; z_idx = nvar + 1 + np.arange(S)
115         nvar_full = nvar + 1 + S
116         c = np.concatenate([c, np.zeros(S + 1)])
117         c[alpha_idx] = lam; c[z_idx] = lam / ((1 - beta) * S)
118     else:
119         nvar_full = nvar
120     for s in range(S):
121         c[vary[s]['Q']] = pi * C.EMERG_MULT * price

```

```

122
123 eq_r, eq_c, eq_v, beq = [], [], [], []
124 row = 0
125 for s in range(S):
126     Pv = np.arange(nP); iv = vary[s]
127     for t in range(T):
128         eq_r.append(row); eq_c.append(Pv[t]); eq_v.append(1.0)
129         eq_r.append(row); eq_c.append(iv['C'][t]); eq_v.append(-1.0)
130         eq_r.append(row); eq_c.append(iv['F'][t]); eq_v.append(C.ETA)
131         eq_r.append(row); eq_c.append(iv['Q'][t]); eq_v.append(1.0)
132         eq_r.append(row); eq_c.append(iv['R'][t]); eq_v.append(-1.0)
133         beq.append(scenD[s, t] - scenG[s, t]); row += 1
134     for t in range(T):
135         eq_r.append(row); eq_c.append(iv['E'][t]); eq_v.append(1.0)
136         eq_r.append(row); eq_c.append(iv['C'][t]); eq_v.append(-C.ETA)
137         eq_r.append(row); eq_c.append(iv['F'][t]); eq_v.append(1.0)
138         if t > 0:
139             eq_r.append(row); eq_c.append(iv['E'][t-1]); eq_v.append(-1.0)
140             beq.append(0.0)
141         else:
142             beq.append(E_start)
143     row += 1
144 A_eq = sp.csr_matrix((eq_v, (eq_r, eq_c)), shape=(len(beq), nvar_full))
145
146 # CVaR 不等式:  $\Sigma pP + \Sigma 5pQ - - z_s \leq 0$ 
147 Aub_r, Aub_c, Aub_v, bub = [], [], [], []
148 if lam > 0:
149     for s in range(S):
150         r = len(bub)
151         for t in range(T):
152             Aub_r.append(r); Aub_c.append(t); Aub_v.append(price[t])
153             Aub_r.append(r); Aub_c.append(vary[s]['Q'][t]); Aub_v.append(C.EMERG_MULT *
154                                     price[t])
155             Aub_r.append(r); Aub_c.append(alpha_idx); Aub_v.append(-1.0)
156             Aub_r.append(r); Aub_c.append(z_idx[s]); Aub_v.append(-1.0)
157             bub.append(0.0)
158         A_ub = sp.csr_matrix((Aub_v, (Aub_r, Aub_c)), shape=(len(bub), nvar_full))
159         b_ub = np.array(bub)
160     else:
161         A_ub = None; b_ub = None
162
163 lb = np.zeros(nvar_full); ub = np.full(nvar_full, np.inf)
164 for s in range(S):
165     iv = vary[s]
166     ub[iv['C']] = C.E_CMAX; ub[iv['F']] = C.E_FMAX
167     lb[iv['E']] = C.SOC_MIN; ub[iv['E']] = C.SOC_HIGH_BOUND
168 if lam > 0:

```

```

168     lb[alpha_idx] = -np.inf; lb[z_idx] = 0
169     bounds = list(zip(lb, ub))
170
171     res = linprog(c, A_eq=A_eq, A_ub=A_ub, b_eq=np.array(beq), b_ub=b_ub,
172                 bounds=bounds, method='highs')
173     if not res.success:
174         raise RuntimeError('两阶段LP失败: ' + res.message)
175     x = res.x
176     P = x[:nP]
177     Q = np.zeros((S, T)); Cc = np.zeros((S, T)); F = np.zeros((S, T)); E = np.zeros((S, T))
178     for s in range(S):
179         iv = vary[s]
180         Q[s] = x[iv['Q']]; Cc[s] = x[iv['C']]; F[s] = x[iv['F']]; E[s] = x[iv['E']]
181     return dict(P=P, Q=Q, c=Cc, f=F, E=E, obj=res.fun)
182
183
184 def solve_adjust(price, G, D, E_start, P_plan, t0):
185     """滚动再计划: 在 [t0,144) 决定最终购电 A, 对计划 P_plan 付违约/超出惩罚。
186
187      $u=(P-A)^+$  违约量  $\rightarrow$  项  $-0.5p$   $u$ ;  $w=(A-P)^+$  超出量  $\rightarrow$  项  $+1.5p$   $w$ 。
188     再计划阶段不允许紧急购电( $Q=0$ )。返回 dict(A,c,f,Q=0,R,E,u,w,obj), 长度 (144-t0)。
189     """
190     T = C.T_IN_DAY; nh = T - t0
191     if nh <= 0:
192         z = np.zeros(0)
193         return dict(A=z, c=z, f=z, Q=z, R=z, E=z, u=z, w=z, obj=0.0)
194     base = np.arange(nh) * 8
195     iA, iC, iF, iQ, iR, iE, iU, iW = (base + k for k in range(8))
196     pt = np.arange(t0, T)
197     nvar = 8 * nh
198     c = np.zeros(nvar)
199     c[iU] = -C.DEFAULT_MULT * price[pt]
200     c[iW] = C.EXCESS_MULT * price[pt]
201
202     eq_r, eq_c, eq_v, beq = [], [], [], []
203     for s in range(nh):
204         eq_r.append(s); eq_c.append(iA[s]); eq_v.append(1.0)
205         eq_r.append(s); eq_c.append(iC[s]); eq_v.append(-1.0)
206         eq_r.append(s); eq_c.append(iF[s]); eq_v.append(C.ETA)
207         eq_r.append(s); eq_c.append(iQ[s]); eq_v.append(1.0)
208         eq_r.append(s); eq_c.append(iR[s]); eq_v.append(-1.0)
209         beq.append(D[pt[s]] - G[pt[s]])
210     for s in range(nh):
211         rr = nh + s
212         eq_r.append(rr); eq_c.append(iE[s]); eq_v.append(1.0)
213         eq_r.append(rr); eq_c.append(iC[s]); eq_v.append(-C.ETA)
214         eq_r.append(rr); eq_c.append(iF[s]); eq_v.append(1.0)

```

```

215         if s > 0:
216             eq_r.append(rr); eq_c.append(iE[s-1]); eq_v.append(-1.0)
217             beq.append(0.0)
218         else:
219             beq.append(E_start)
220     A_eq = sp.csr_matrix((eq_v, (eq_r, eq_c)), shape=(len(beq), nvar))
221
222     Aub_r, Aub_c, Aub_v, bub = [], [], [], []
223     for s in range(nh):
224         r = len(bub)
225         Aub_r.append(r); Aub_c.append(iA[s]); Aub_v.append(-1.0)
226         Aub_r.append(r); Aub_c.append(iU[s]); Aub_v.append(-1.0)
227         bub.append(-P_plan[pt[s]])
228         r = len(bub)
229         Aub_r.append(r); Aub_c.append(iA[s]); Aub_v.append(1.0)
230         Aub_r.append(r); Aub_c.append(iW[s]); Aub_v.append(-1.0)
231         bub.append(P_plan[pt[s]])
232     A_ub = sp.csr_matrix((Aub_v, (Aub_r, Aub_c)), shape=(len(bub), nvar))
233
234     lb = np.zeros(nvar); ub = np.full(nvar, np.inf)
235     ub[iC] = C.E_CMAX; ub[iF] = C.E_FMAX
236     lb[iE] = C.SOC_MIN; ub[iE] = C.SOC_HIGH_BOUND
237     ub[iQ] = 0.0
238     lb[iU] = 0; ub[iU] = np.maximum(P_plan[pt], 0.0)
239     lb[iW] = 0
240     bounds = list(zip(lb, ub))
241
242     res = linprog(c, A_eq=A_eq, A_ub=A_ub, b_eq=np.array(beq), b_ub=np.array(bub),
243                 bounds=bounds, method='highs')
244     if not res.success:
245         raise RuntimeError('滚动再计划LP失败: ' + res.message)
246     x = res.x
247     return dict(A=x[iA], c=x[iC], f=x[iF], Q=x[iQ], R=x[iR], E=x[iE],
248               u=x[iU], w=x[iW], obj=res.fun)
249
250
251 def solve_adjust_scen(price, scenG, D, E_start, P_plan, t0, lam=0.0, beta=0.90,
252                      emult=C.EMERG_MULT):
253     """滚动再计划（情景鲁棒版）：一阶段购电 A 共用，每光伏情景独立电池调度。
254
255     目标 = 期望电网费(违约退款+超额惩罚) + lam * CVaR_beta(情景总成本) + 期望紧急费。
256     允许每情景紧急购电 Q_s（代价 emult * price），保证任意光伏情景下可行——
257     这正是"随机 MPC"的鲁棒性体现。Q3 官方取 lam=0（纯期望）。
258     返回 dict(A,c,f,Q,R,E,u,w,obj)；各量按情景合并为 (S,nh) (A/u/w 广播为 (S,nh))。
259     """
260     T = C.T_IN_DAY
261     scenG = np.asarray(scenG, float)

```

```

262 S = scenG.shape[0]
263 nh = T - t0
264 if nh <= 0:
265     z = np.zeros(0)
266     return dict(A=z, c=z, f=z, Q=z, R=z, E=z, u=z, w=z, obj=0.0)
267 pt = np.arange(t0, T)
268 pi = 1.0 / S
269 # 一阶段: A, U, W; 每情景: C,F,Q,R,E
270 baseA, baseU, baseW = 0, nh, 2 * nh
271 scen_base = 3 * nh + np.arange(S) * (5 * nh)
272 nvar = 3 * nh + S * 5 * nh
273 iA = baseA + np.arange(nh)
274 iU = baseU + np.arange(nh)
275 iW = baseW + np.arange(nh)
276
277 c = np.zeros(nvar)
278 c[iU] = -C.DEFAULT_MULT * price[pt]
279 c[iW] = C.EXCESS_MULT * price[pt]
280 for s in range(S):
281     sb = scen_base[s]
282     c[sb + 2 * nh + np.arange(nh)] = pi * emult * price[pt] # Q_s
283 alpha_idx = z_idx = None
284 if lam > 0:
285     alpha_idx = nvar; z_idx = nvar + 1 + np.arange(S)
286     nvar_full = nvar + 1 + S
287     c = np.concatenate([c, np.zeros(S + 1)])
288     c[alpha_idx] = lam; c[z_idx] = lam / ((1 - beta) * S)
289 else:
290     nvar_full = nvar
291
292 eq_r, eq_c, eq_v, beq = [], [], [], []
293 for s in range(S):
294     sb = scen_base[s]
295     iC = sb + np.arange(nh); iF = sb + nh + np.arange(nh)
296     iQ = sb + 2 * nh + np.arange(nh); iR = sb + 3 * nh + np.arange(nh)
297     iE = sb + 4 * nh + np.arange(nh)
298     for k in range(nh):
299         rr = len(beq)
300         eq_r.append(rr); eq_c.append(iA[k]); eq_v.append(1.0)
301         eq_r.append(rr); eq_c.append(iC[k]); eq_v.append(-1.0)
302         eq_r.append(rr); eq_c.append(iF[k]); eq_v.append(C.ETA)
303         eq_r.append(rr); eq_c.append(iQ[k]); eq_v.append(1.0)
304         eq_r.append(rr); eq_c.append(iR[k]); eq_v.append(-1.0)
305         beq.append(D[pt[k]] - scenG[s, pt[k]])
306     for k in range(nh):
307         rr = len(beq)
308         eq_r.append(rr); eq_c.append(iE[k]); eq_v.append(1.0)

```



```

309     eq_r.append(rr); eq_c.append(iC[k]); eq_v.append(-C.ETA)
310     eq_r.append(rr); eq_c.append(iF[k]); eq_v.append(1.0)
311     if k > 0:
312         eq_r.append(rr); eq_c.append(iE[k - 1]); eq_v.append(-1.0)
313         beq.append(0.0)
314     else:
315         beq.append(E_start)
316 A_eq = sp.csr_matrix((eq_v, (eq_r, eq_c)), shape=(len(beq), nvar_full))
317
318 Aub_r, Aub_c, Aub_v, bub = [], [], [], []
319 for k in range(nh):
320     r = len(bub)
321     Aub_r.append(r); Aub_c.append(iA[k]); Aub_v.append(-1.0)
322     Aub_r.append(r); Aub_c.append(iU[k]); Aub_v.append(-1.0)
323     bub.append(-P_plan[pt[k]])
324     r = len(bub)
325     Aub_r.append(r); Aub_c.append(iA[k]); Aub_v.append(1.0)
326     Aub_r.append(r); Aub_c.append(iW[k]); Aub_v.append(-1.0)
327     bub.append(P_plan[pt[k]])
328 if lam > 0:
329     for s in range(S):
330         r = len(bub)
331         sb = scen_base[s]
332         iQ = sb + 2 * nh + np.arange(nh)
333         for k in range(nh):
334             Aub_r.append(r); Aub_c.append(iQ[k]); Aub_v.append(emult * price[pt[k]])
335             Aub_r.append(r); Aub_c.append(alpha_idx); Aub_v.append(-1.0)
336             Aub_r.append(r); Aub_c.append(z_idx[s]); Aub_v.append(-1.0)
337             bub.append(0.0)
338 A_ub = sp.csr_matrix((Aub_v, (Aub_r, Aub_c)), shape=(len(bub), nvar_full))
339
340 lb = np.zeros(nvar_full); ub = np.full(nvar_full, np.inf)
341 for s in range(S):
342     sb = scen_base[s]
343     ub[sb + np.arange(nh)] = C.E_CMAX # C_s
344     ub[sb + nh + np.arange(nh)] = C.E_FMAX # F_s
345     lb[sb + 4 * nh + np.arange(nh)] = C.SOC_MIN
346     ub[sb + 4 * nh + np.arange(nh)] = C.SOC_HIGH_BOUND
347 ub[iU] = np.maximum(P_plan[pt], 0.0)
348 if lam > 0:
349     lb[alpha_idx] = -np.inf; lb[z_idx] = 0
350 bounds = list(zip(lb, ub))
351
352 res = linprog(c, A_eq=A_eq, A_ub=A_ub, b_eq=np.array(beq), b_ub=np.array(bub),
353             bounds=bounds, method='highs')
354 if not res.success:
355     raise RuntimeError('情景鲁棒滚动再计划LP失败: ' + res.message)

```

```

356     x = res.x
357     A = x[iA]; u = x[iU]; w = x[iW]
358     Cm = np.zeros((S, nh)); F = np.zeros((S, nh)); Q = np.zeros((S, nh))
359     R = np.zeros((S, nh)); E = np.zeros((S, nh))
360     for s in range(S):
361         sb = scen_base[s]
362         Cm[s] = x[sb + np.arange(nh)]
363         F[s] = x[sb + nh + np.arange(nh)]
364         Q[s] = x[sb + 2 * nh + np.arange(nh)]
365         R[s] = x[sb + 3 * nh + np.arange(nh)]
366         E[s] = x[sb + 4 * nh + np.arange(nh)]
367     return dict(A=A, c=Cm, f=F, Q=Q, R=R, E=E,
368               u=np.tile(u, (S, 1)), w=np.tile(w, (S, 1)), obj=res.fun)
369
370
371 def _var_rows_scen(base_s, T):
372     out = {}
373     for s, base in enumerate(base_s):
374         ar = base + np.arange(5 * T)
375         out[s] = {'C': ar[0::5], 'F': ar[1::5], 'Q': ar[2::5],
376                 'R': ar[3::5], 'E': ar[4::5]}
377     return out

```

## 2.4 B.4 情景生成 (scenarios.py)

基于历史残差的非参数 bootstrap，生成点预报与  $S$  个等概率负荷/光伏情景，供两阶段随机规划使用。

```

1  # -*- coding: utf-8 -*-
2  """情景生成：非参数 bootstrap 历史残差。返回点预报与 S 个等概率情景。"""
3  import numpy as np
4  import config as C
5
6
7  def point_forecast(hist, K=7):
8      """hist:(N,144)。前 K 天逐槽均值。"""
9      n = len(hist)
10     if n == 0:
11         return np.zeros(C.T_IN_DAY)
12     return np.mean(hist[-min(K, n):], axis=0)
13
14
15 def forecast_and_scenarios(load_hist, pv_hist, S=30, seed=1):
16     """由历史构建点预报与 S 个情景。load_hist/pv_hist:(N,144)。

```

```

17  返回 dict(point_ld,point_pv,scen_ld,scen_pv)。scen*:(S,144)。"""
18  rng = np.random.default_rng(seed)
19  n = len(load_hist)
20  point_ld = point_forecast(load_hist)
21  point_pv = point_forecast(pv_hist)
22  if n == 0:
23      return dict(point_ld=point_ld, point_pv=point_pv,
24                  scen_ld=np.tile(point_ld, (S, 1)),
25                  scen_pv=np.tile(point_pv, (S, 1)))
26  idx_ld = rng.integers(0, n, size=(S, C.T_IN_DAY))
27  idx_pv = rng.integers(0, n, size=(S, C.T_IN_DAY))
28  T = C.T_IN_DAY
29  scen_ld = load_hist[idx_ld, np.arange(T)[None, :]]
30  scen_pv = pv_hist[idx_pv, np.arange(T)[None, :]]
31  return dict(point_ld=point_ld, point_pv=point_pv,
32              scen_ld=np.asarray(scen_ld), scen_pv=np.asarray(scen_pv))
33
34
35 def pv_scenarios(point_pv, hist_pv, S=30, seed=1, sigma_scale=1.0):
36     """以官方点预报 point_pv 为中心，叠加历史光伏(逐槽)变异 → 情景(kWh)。
37     hist_pv:(N,144)。"""
38     rng = np.random.default_rng(seed)
39     T = C.T_IN_DAY
40     n = len(hist_pv)
41     idx = rng.integers(0, max(n, 1), size=(S, T)) if n > 0 else np.zeros((S, T), int)
42     if n > 0:
43         # 历史同槽相对偏差
44         hist = np.maximum(hist_pv, 0.0)
45         base = np.tile(point_pv, (S, 1))
46         offset = hist[idx, np.arange(T)[None, :]] - point_forecast(hist_pv, n)
47         scen = base + sigma_scale * offset
48         scen = np.maximum(scen, 0.0)
49     else:
50         scen = np.tile(point_pv, (S, 1))
51     return np.asarray(scen, float)
52
53
54 def residual_scenarios(load_hist, pv_hist, K, S=30, seed=1):
55     """整日残差 bootstrap 情景生成（升级版：负荷与光伏同一天成对抽取，替代逐槽独立抽样）。
56
57     load_hist/pv_hist: (N,144) 按日期升序的历史实际值(kWh/时段)。
58     调用方必须只传入目标日 d 之前的数据（信息因果，禁止含未来）。
59     K: int（负荷/光伏同一窗口）或 (K_L, K_P)（各自窗口长度）。
60     点预报 = 各自最近 K 天逐槽均值；
61     残差池 = 池内每一天的（实际 - 点预报）整条曲线（K_L=K_P 时即最近 K 天）；
62     抽样 = 有放回抽 S 个整天，负荷与光伏使用同一天索引（成对，保留负荷-光伏相关性）；
63     情景 = 点预报 + 整日残差；光伏情景截断到 >=0（物理非负）。

```

```

64     返回 dict(point_ld, point_pv, scen_ld, scen_pv), scen_*:(S,144)。
65     """
66     if isinstance(K, (tuple, list, np.ndarray)):
67         KL, KP = int(K[0]), int(K[1])
68     else:
69         KL = KP = int(K)
70     T = C.T_IN_DAY
71     n = len(load_hist)
72     rng = np.random.default_rng(seed)
73     point_ld = point_forecast(load_hist, KL)
74     point_pv = point_forecast(pv_hist, KP)
75     if n == 0:
76         return dict(point_ld=point_ld, point_pv=point_pv,
77                     scen_ld=np.tile(point_ld, (S, 1)),
78                     scen_pv=np.tile(point_pv, (S, 1)))
79     # 成对抽样池: 取并集窗口 (最近 max(KL,KP) 天), 同一池内日索引同时决定负荷与光伏残差
80     w = min(n, max(KL, KP))
81     resid_ld = load_hist[n - w:n] - point_ld[None, :]
82     resid_pv = pv_hist[n - w:n] - point_pv[None, :]
83     idx = rng.integers(0, w, size=S) # 池内偏移 (0=最早一天, w-1=最近一天)
84     scen_ld = point_ld[None, :] + resid_ld[idx]
85     scen_pv = point_pv[None, :] + resid_pv[idx]
86     scen_pv = np.maximum(scen_pv, 0.0)
87     return dict(point_ld=point_ld, point_pv=point_pv,
88                 scen_ld=np.asarray(scen_ld, float), scen_pv=np.asarray(scen_pv, float))

```

## 2.5 B.5 问题一入口 (main\_q1.py)

单日确定性完美信息；调用 solve\_day 求第 1 问计划购电、充放电与储电轨迹，写出 result1.xlsx。

```

1  # -*- coding: utf-8 -*-
2  """问题1: 确定性 MILP/LP 日计划 (附件1 完美信息)。
3
4  储能日循环: storage 日末回到日初(6000 kWh), 保证可长期重复运行;
5  目标: 最小化当日购电费用(电价 p · 购电量), 约束含 能量平衡、储能上下限、充放电功率上限、
6         光伏/负载(附件1 预报)已知。输出 result1.xlsx(两张表) + 计划图。
7  """
8  import os, sys
9  sys.path.insert(0, os.path.dirname(__file__))
10 import numpy as np
11 import pandas as pd
12 import matplotlib
13 matplotlib.use('Agg')

```

```

14 import config as C
15 C.setup_plot_style()
16 import matplotlib.pyplot as plt
17 import data, lp, results as R
18
19
20 def run(outfile='result1.xlsx'):
21     a1 = data.load_attach1()
22     price = a1['price']
23     rp = lp.solve_day_plan(price, a1['pv'], a1['load'], C.E_INIT, force_end=C.E_INIT)
24     P, c, f, E = rp['P'], rp['c'], rp['f'], rp['E']
25     cost = float(np.sum(price * P))
26     path = os.path.join(C.RES_DIR, outfile)
27     R.write_result1(path, P, c, f, C.E_INIT, E[-1], price)
28     print('已写出', path)
29     print(f'[Q1 确定性日计划] 购电量合计={P.sum():.2f} kWh, 购电费={cost:.2f} 元, '
30           f'0:00 储电={C.E_INIT}, 24:00 储电={E[-1]:.2f}')
31     return dict(price=price, P=P, c=c, f=f, E=E, cost=cost)
32
33
34 def plot(P, c, f, E, price, path=None):
35     """计划购电/充电/放电/储电量/电价 可视化。"""
36     x = np.arange(C.T_IN_DAY)
37     fig, ax1 = plt.subplots(figsize=(13, 5.5))
38     ax1.bar(x, P, width=0.6, alpha=0.7, label='购电量(kWh)', color='tab:blue')
39     ax1.bar(x, c, width=0.6, alpha=0.5, label='充电量(kWh)', color='tab:green')
40     ax1.bar(x, f, width=0.6, alpha=0.5, label='放电量(kWh)', color='tab:orange')
41     ax1.set_xlabel('时段(每10min)', fontsize=12); ax1.set_ylabel('能量(kWh)', fontsize=12)
42     ax1.set_xlim(0, C.T_IN_DAY)
43     ax1.set_xticks(range(0, C.T_IN_DAY + 1, 12))
44     ax1.legend(loc='upper left', fontsize=11); ax1.grid(alpha=0.3)
45     ax1.tick_params(labelsize=11)
46
47     ax2 = ax1.twinx()
48     ax2.plot(x, E, color='tab:red', lw=1.6, label='储电量(kWh)')
49     ax2.axhline(C.SOC_HIGH_BOUND, color='gray', ls=':', lw=1)
50     ax2.axhline(C.SOC_MIN, color='gray', ls=':', lw=1)
51     ax2.set_ylabel('储电量(kWh)', fontsize=12); ax2.legend(loc='upper right', fontsize=11)
52     ax2.set_ylim(0, C.SOC_MAX)
53     ax2.tick_params(labelsize=11)
54     plt.title('问题1: 单日最优运营计划(储能日循环)', fontsize=14)
55     fig.tight_layout()
56     path = path or os.path.join(C.FIG_DIR, 'q1_plan.png')
57     fig.savefig(path, dpi=150); plt.close(fig)
58     print('图已存', path)
59
60

```

```

61 def dp_cost_day(price, G, D, E_start=C.E_INIT, E_end=C.E_INIT, h=120.0):
62     """状态空间动态规划精确求解单日最小购电费（网格步长 h，验证 LP 全局最优性）。
63
64     状态 = 时段末储电量 E {1200, 1200+h, ..., 10800};
65     转移 E→E': 若 Δ=E'-E>0 则 c=Δ/、f=0; Δ<0 则 f=-Δ、c=0;
66     购电 P = max(D-G- · f+c, 0)（弃光免费）; V_t(E)=min_{E'}(p_t · P+V_{t+1}(E')),
67     终端 V_{144}(E_end)=0, 起点 V_0(E_start) 即最优费用。向量化：每时段对
68     |Δ| min(E_FMAX, · E_CMAX) 的网格位移求 min。
69     """
70     T = C.T_IN_DAY
71     grid = np.arange(C.SOC_MIN, C.SOC_HIGH_BOUND + 1e-9, h)
72     n = len(grid)
73     idx = {round(float(e), 6): i for i, e in enumerate(grid)}
74     if round(float(E_start), 6) not in idx or round(float(E_end), 6) not in idx:
75         raise ValueError('起/止储电量不在 DP 网格上（请选网格 h 整除 1200..10800）')
76     K = int(np.floor(min(C.E_FMAX, C.ETA * C.E_CMAX) / h))
77     ks = np.arange(-K, K + 1)
78     dE = ks * h
79     f = np.maximum(-dE, 0.0) # 放电量
80     c = np.maximum(dE / C.ETA, 0.0) # 充电量
81     Pk = np.maximum(D[:, None] - G[:, None] - C.ETA * f[None, :] + c[None, :], 0.0) # (T, 2K+1)
82     INF = np.inf
83     V = np.full(n, INF)
84     V[idx[round(float(E_end), 6)]] = 0.0
85     for t in range(T - 1, -1, -1):
86         cand = np.full((n, 2 * K + 1), INF)
87         for k2, k in enumerate(ks):
88             j = np.arange(n) + k
89             ok = (j >= 0) & (j < n)
90             cand[ok, k2] = price[t] * Pk[t, k2] + V[j[ok]]
91         V = cand.min(axis=1)
92     return float(V[idx[round(float(E_start), 6)]])
93
94
95 def dp_report(price=None):
96     """问题1 LP 最优性动态规划验证。
97
98     (1) 附件1 典型日：网格 h {120,60,30,15} 的 DP 费用收敛到 LP 解（h=15 偏差 0.5%），
99     验证 LP 全局最优；
100    (2) 附件2 全年逐日：h=15 下 DP 与 LP（均 E_0=E_144=6000 日循环）费用差，
101    报告最大/平均相对偏差（网格离散化误差）。
102    写 report/q1_dp_verification.csv 与 paper/figures_adv/A13_q1_dp.png。
103    """
104    a1 = data.load_attach1()
105    price = a1['price'] if price is None else price
106    G, D = a1['pv'], a1['load']
107    rp = lp.solve_day_plan(price, G, D, C.E_INIT, force_end=C.E_INIT)

```

```

108 lp_cost = float(np.sum(price * rp['P']))
109
110 rows = []
111 print(f'[Q1 DP 验证] 附件1 典型日: 网格收敛 (LP 解 = {lp_cost:,.2f} 元)')
112 conv = {}
113 for h in (120.0, 60.0, 30.0, 15.0):
114     dp = dp_cost_day(price, G, D, h=h)
115     conv[h] = dp
116     rows.append(dict(day='附件1典型日', grid_h=h, lp_cost=lp_cost,
117                     dp_cost=dp, gap_pct=(dp - lp_cost) / lp_cost * 100))
118     print(f' h={h:g}: DP={dp:,.2f} 元 gap={(dp - lp_cost) / lp_cost * 100:,.4f}%')
119
120 dates, load, pv = data.load_attach2()
121 gaps = []
122 for i in range(len(dates)):
123     rpi = lp.solve_day_plan(price, pv[i], load[i], C.E_INIT, force_end=C.E_INIT)
124     lpi = float(np.sum(price * rpi['P']))
125     dpi = dp_cost_day(price, pv[i], load[i], h=15.0)
126     gaps.append(dpi - lpi)
127     rows.append(dict(day=str(dates[i].date()), grid_h=15, lp_cost=round(lpi, 4),
128                     dp_cost=round(dpi, 4), gap_pct=(dpi - lpi) / lpi * 100))
129 gaps = np.asarray(gaps)
130 gap_p = gaps / np.asarray([r['lp_cost'] for r in rows if r['day'] != '附件1典型日']) * 100
131 print(f'[Q1 DP 验证] 附件2 全年 {len(dates)} 日 (h=15): max gap = {gaps.max():.2f} 元 '
132       f'({gap_p.max():.3f}%), mean gap = {gaps.mean():.2f} 元 ({gap_p.mean():.3f}%)')
133 path = os.path.join(C.RES_DIR, 'q1_dp_verification.csv')
134 pd.DataFrame(rows).to_csv(path, index=False, encoding='utf-8-sig')
135 print('已写', path)
136
137 # A13 图: 左=网格收敛, 右=全年逐日偏差分布
138 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.3))
139 hs = sorted(conv)
140 ax1.plot(hs, [conv[h] for h in hs], '-o', color='tab:blue')
141 ax1.axhline(lp_cost, color='tab:red', ls='--', lw=1.2, label=f'LP 最优 = {lp_cost:,.2f} 元')
142 ax1.set_xscale('log', base=2); ax1.invert_xaxis()
143 ax1.set_xticks(list(hs)); ax1.set_xticklabels([f'{h:g}' for h in hs], fontsize=11)
144 ax1.set_xlabel('DP 网格步长 h (kWh)', fontsize=12)
145 ax1.set_ylabel('最小购电费 (元)', fontsize=12)
146 ax1.set_title('(a) 附件1 典型日: DP 网格收敛', fontsize=13)
147 ax1.legend(fontsize=11); ax1.grid(alpha=0.3)
148 ax1.tick_params(labelsize=11)
149 gp = np.asarray([r['gap_pct'] for r in rows if r['day'] != '附件1典型日'])
150 ax2.hist(gp, bins=30, color='tab:green', alpha=0.75, edgecolor='w')
151 ax2.axvline(gp.mean(), color='k', ls='--', lw=1.2, label=f'平均 = {gp.mean():.3f}%')
152 ax2.set_xlabel('DP 相对 LP 偏差 (%)', fontsize=12)
153 ax2.set_ylabel('天数', fontsize=12)
154 ax2.set_title('(b) 附件2 全年逐日偏差分布 (h=15)', fontsize=13)

```

```

155     ax2.legend(fontsize=11); ax2.grid(alpha=0.3, axis='y')
156     ax2.tick_params(labelsize=11)
157     fig.tight_layout()
158     out = os.path.join(C.C_BASE, 'paper', 'figures_adv', 'A13_q1_dp.png')
159     os.makedirs(os.path.dirname(out), exist_ok=True)
160     fig.savefig(out, dpi=150); plt.close(fig)
161     print('图已存', out)
162     return rows
163
164
165 if __name__ == '__main__':
166     A = run()
167     plot(A['P'], A['c'], A['f'], A['E'], A['price'])
168     dp_report(A['price'])

```

## 2.6 B.6 问题二入口（main\_q2.py）

按”完美信息 + 滚动”双口径评估两阶段随机 / CVaR 策略，输出 result2.xlsx 与风险敏感性结果。

```

1  # -*- coding: utf-8 -*-
2  """问题2：两阶段随机规划（官方口径 v2 —— 预报式，替代原完美信息口径）。
3
4  官方结果 result2.xlsx 口径：
5      每天 0:00 依据截至前一日的历史数据（信息因果，不含未来）——
6      1) 点预报：负荷/光伏分别取各自前 K_L / K_P 天逐槽(144 时段)均值，K 由全年交叉验证选出；
7      2) 情景生成：整日残差 bootstrap (scenarios.residual_scenarios)，负荷与光伏同一天成对，
8          S=30 个情景，抽样种子 = 2026 + 日序（与日期绑定、确定性、重跑逐位一致）；
9          2025-01-01 无历史，按拍板规格用附件1 典型日剖面做冷启动情景基准；
10     3) 计划 P: lp.solve_two_stage(lam=0, 纯期望最小化)；
11     4) 结算：固定 P，用当日实际负荷/光伏调 lp.solve_day_fixedP（紧急价 5p）得当日实际
12         紧急购电 Q 与充放电 c/f、储能 E；E_end 结转次日（跨日连续，1200~10800 kWh）。
13     全年链自 2025-01-01 (E=6000) 逐日滚动，仅 1 月作储能状态预热，输出 2.1-12.31。
14
15 旧口径：perfect_info_legacy()（把当日实际负荷/光伏当作 0:00 已知 → 全年 Q 0），
16 仅作对比保留，不再写入 result2.xlsx。
17
18 """
19 import os, sys, time, hashlib
20 sys.path.insert(0, os.path.dirname(__file__))
21 import numpy as np
22 import pandas as pd
23 import config as C
24 import data, lp, results as R
25 import scenarios

```



```

25
26 K_CANDIDATES = [7, 15, 30] # K 候选集（数据驱动选优）
27 SEED_BASE = 2026           # 抽样种子基数：2026 + 日序
28 LEGACY_FEE_RECORD = 12259844.62 # 旧完美信息口径计划费（历史运行记录值）
29
30
31 def _seed_for_day(d):
32     """与日期绑定的确定性随机种子（重跑逐位一致）。"""
33     return SEED_BASE + int(d.dayofyear)
34
35
36 def cross_validate_K(dates, load, pv):
37     """交叉验证选 K（候选 {7,15,30}）。
38
39     对 2025-02-01~12-31 每天 d：用 d 前 K 天逐槽均值预报 d，计算 144 时段 RMSE，
40     除以该日实际均值归一化（当日均值 0 的天跳过），全年取平均；
41     负荷、光伏各自选出平均归一化 RMSE 最小的 K（记为 K_L、K_P）。
42     返回 (KL, KP, cv 表)。
43     """
44     start = int(np.searchsorted(dates, pd.Timestamp('2025-02-01')))
45     rows, skip = [], 0
46     for K in K_CANDIDATES:
47         nrmse_L, nrmse_P = [], []
48         for i in range(start, len(dates)):
49             lo = max(0, i - K)
50             fcL = load[lo:i].mean(axis=0) if i > lo else load[i] * 0.0
51             fcP = pv[lo:i].mean(axis=0) if i > lo else pv[i] * 0.0
52             mL = float(load[i].mean()); mP = float(pv[i].mean())
53             if mL > 1e-6:
54                 nrmse_L.append(float(np.sqrt(np.mean((fcL - load[i]) ** 2)) / mL))
55             else:
56                 skip += 1
57             if mP > 1e-6:
58                 nrmse_P.append(float(np.sqrt(np.mean((fcP - pv[i]) ** 2)) / mP))
59             else:
60                 skip += 1
61             rows.append(dict(K=K, nrmse_load=float(np.mean(nrmse_L)),
62                             nrmse_pv=float(np.mean(nrmse_P))))
63     cv = pd.DataFrame(rows)
64     KL = int(cv.loc[cv['nrmse_load'].idxmin(), 'K'])
65     KP = int(cv.loc[cv['nrmse_pv'].idxmin(), 'K'])
66     if skip:
67         print(f'[K 交叉验证] 跳过日均值 0 的天次（负荷/光伏合计）：{skip}')
68     return KL, KP, cv
69
70
71 def perfect_info_legacy(price, load, pv, dates):

```

```

72 """【旧口径，仅作对比】完美信息：把当日实际负荷/光伏当作 0:00 已知，
73 确定性 LP 计划 → 全年紧急购电 Q 0。不再用于官方 result2.xlsx。"""
74 rec = {}
75 E_start = C.E_INIT
76 for i in range(len(dates)):
77     rp = lp.solve_day_plan(price, pv[i], load[i], E_start)
78     rec[dates[i]] = dict(P=rp['P'], Q=np.zeros(C.T_IN_DAY), c=rp['c'], f=rp['f'],
79                          E_start=E_start, E_end=rp['E'][-1])
80     E_start = rp['E'][-1]
81 return rec
82
83
84 def _run_forecast_chain(price, dates, load, pv, KL, KP, S=30, lam=0.0,
85                        max_days=None, verbose=True):
86     """预报式两阶段随机规划全年滚动链（官方口径）。
87
88     每日: residual_scenarios(load[:i], pv[:i], (KL,KP), S, seed=2026+日序)
89     → solve_two_stage(lam) 得计划 P → solve_day_fixedP 按当日实际结算
90     → E_end 结转次日。断言每日储能 E 全程 [1200,10800]。
91     max_days: 仅跑前 N 天（冒烟测试用），默认全年。
92     """
93     n = len(dates) if max_days is None else int(max_days)
94     rec = {}
95     E_start = C.E_INIT
96     t0 = time.time()
97     a1 = data.load_attach1() # 冷启动先验: 2025-01-01 无历史可用
98     for i in range(n):
99         d = dates[i]
100         # price: 一维固定电价(144) 或 二维逐日电价(365,144) (Q4-2 波动电价用)
101         p = price[i] if price.ndim == 2 else price
102         if i == 0:
103             # 冷启动（拍板规格）：用附件1 典型日负荷/光伏剖面作当日点预报与情景基准
104             # （无历史可抽残差，S 个情景同形）。
105             scen_ld = np.tile(a1['load'], (S, 1))
106             scen_pv = np.tile(a1['pv'], (S, 1))
107         else:
108             fc = scenarios.residual_scenarios(load[:i], pv[:i], (KL, KP), S=S,
109                                                seed=_seed_for_day(d))
110             scen_ld, scen_pv = fc['scen_ld'], fc['scen_pv']
111             two = lp.solve_two_stage(p, scen_ld, scen_pv, E_start,
112                                     lam=lam, beta=C.BETA)
113             settle = lp.solve_day_fixedP(p, pv[i], load[i], E_start, two['P'])
114             E_t = np.asarray(settle['E'], float)
115             assert np.all(E_t >= C.SOC_MIN - 1e-6) and np.all(E_t <= C.SOC_HIGH_BOUND + 1e-6), \
116                 f'{d.date()} 储能越界: min={E_t.min():.2f} max={E_t.max():.2f}'
117             rec[d] = dict(P=two['P'], Q=settle['Q'], c=settle['c'], f=settle['f'],
118                          E_start=float(E_start), E_end=float(E_t[-1]))

```

```

119     E_start = float(E_t[-1])
120     if verbose and (i + 1) % 30 == 0:
121         el = time.time() - t0
122         print(f' 滚动链进度 {i + 1}/{n} 日 ({d.date()}) E={E_start:8.1f} kWh'
123               f' 用时 {el:6.1f}s', flush=True)
124     return rec
125
126
127 def yearly_lambda_scan(price, dates, load, pv, KL, KP, lams=(0.0, 0.5, 1.0, 2.0, 5.0),
128                       S=30, max_days=None):
129     """全年 扫描: 每个 自 1-01 整链重跑 (1 月预热), 汇总全年费用。
130
131     用于数据驱动选定官方口径 *: emerg() 0.6 * emerg(0) 且 total() 1.01 * total(0)
132     时认为风险权衡"以可忽略的总费上升换取紧急购电显著削减", 取满足条件的最小 。
133     返回 (df, *), df 写 report/q2_lambda_year_scan.csv。
134     """
135     rows = []
136     for lam in lams:
137         rec = _run_forecast_chain(price, dates, load, pv, KL, KP, S=S, lam=float(lam),
138                                  max_days=max_days, verbose=False)
139         d_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
140         pf = sum(float(np.sum(price * rec[d]['P']))) for d in d_out)
141         ef = sum(float(np.sum(C.EMERG_MULT * price * rec[d]['Q']))) for d in d_out)
142         ek = sum(float(rec[d]['Q'].sum())) for d in d_out)
143         rows.append(dict(lam=float(lam), plan_fee=round(pf, 2), emerg_fee=round(ef, 2),
144                        total=round(pf + ef, 2), emerg_kwh=round(ek, 2)))
145         print(f'[ 扫描 ] = {lam}: 计划={pf:,.2f} 紧急={ef:,.2f} ({ek:,.0f} kWh) '
146               f' 合计={pf + ef:,.2f}')
147     df = pd.DataFrame(rows)
148     path = os.path.join(C.RES_DIR, 'q2_lambda_year_scan.csv')
149     df.to_csv(path, index=False, encoding='utf-8-sig')
150     print('已写', path)
151     # 数据驱动选 *: emerg 下降 40% 且 total 不升超 1%
152     base = df[df.lam == lams[0]].iloc[0]
153     cand = df[(df.emerg_fee <= 0.6 * base.emerg_fee) & (df.total <= 1.01 * base.total)]
154     lam_star = float(cand.lam.min()) if len(cand) else float(lams[0])
155     print(f'[ * 选定 ] * = {lam_star} (基准 emerg={base.emerg_fee:,.2f},
156           total={base.total:,.2f}) ')
157     return df, lam_star
158
159 def _result_digest(price, dates_out, rec):
160     """对全部输出序列做确定性哈希, 供跨次运行逐位一致性校验。"""
161     h = hashlib.sha256()
162     for d in dates_out:
163         h.update(np.round(np.asarray(rec[d]['P'], float), 6).tobytes())
164         h.update(np.round(np.asarray(rec[d]['Q'], float), 6).tobytes())

```

```

165         h.update(np.round(np.asarray(rec[d]['c'], float), 6).tobytes())
166         h.update(np.round(np.asarray(rec[d]['f'], float), 6).tobytes())
167         h.update(np.round(np.float64(rec[d]['E_end']), 6).tobytes())
168     return h.hexdigest()
169
170
171 def run(outfile='result2.xlsx', lam=None, S=None, max_days=None):
172     a1 = data.load_attach1()
173     price = a1['price']
174     dates, load, pv = data.load_attach2()
175     S = C.S_NUM if S is None else int(S)
176
177     # ---- 1) 交叉验证选 K (数据驱动) ----
178     KL, KP, cv = cross_validate_K(dates, load, pv)
179     print('[K 交叉验证] 全年平均归一化 RMSE (2025-02-01~12-31, 144 时段):')
180     print(cv.to_string(index=False))
181     print(f'所选: K_L = {KL}, K_P = {KP}')
182
183     # ---- 1b) * 全年扫描 (数据驱动选官方风险权衡权重) ----
184     if lam is None:
185         scan, lam_star = yearly_lambda_scan(price, dates, load, pv, KL, KP,
186                                             S=S, max_days=max_days)
187         lam = lam_star
188         print(f'[Q2 官方口径] 采用数据驱动选定 * = {lam_star}')
189     else:
190         scan = None
191         lam = float(lam)
192
193     # ---- 2) 预报式两阶段随机规划全年链 (1 月预热, 不输出) ----
194     print(f'开始全年滚动链: 365 天 × S={S} 情景两阶段 LP (lam={lam}) .....')
195     rec = _run_forecast_chain(price, dates, load, pv, KL, KP, S=S, lam=lam,
196                              max_days=max_days)
197
198     # ---- 3) 汇总与写 result2.xlsx ----
199     dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
200     P = {d: rec[d]['P'] for d in dates_out}
201     Q = {d: rec[d]['Q'] for d in dates_out}
202     c = {d: rec[d]['c'] for d in dates_out}
203     f = {d: rec[d]['f'] for d in dates_out}
204     E0 = {d: rec[d]['E_start'] for d in dates_out}
205     Ee = {d: rec[d]['E_end'] for d in dates_out}
206     plan_fee = sum(float(np.sum(price * rec[d]['P'])) for d in dates_out)
207     emerg_fee = sum(float(np.sum(C.EMERG_MULT * price * rec[d]['Q'])) for d in dates_out)
208     emerg_kwh = sum(float(rec[d]['Q'].sum()) for d in dates_out)
209     total_fee = plan_fee + emerg_fee
210     assert emerg_fee > 0, '全年紧急购电费应 > 0 (旧完美信息口径为 0, 已切换预报式口径)'
211     path = os.path.join(C.RES_DIR, outfile)

```

```

212 R.write_result2(path, dates_out, P, Q, c, f, E0, Ee)
213 path = R.strip_personal_meta(path) # 清除模板带入的个人元数据（提交合规）
214 print('已写出', path)
215 print(f'[Q2 官方·预报式] 计划购电费 = {plan_fee:,.2f} 元 | '
216       f'紧急购电费 = {emerg_fee:,.2f} 元 ({emerg_kwh:,.0f} kWh) | '
217       f'合计 = {total_fee:,.2f} 元')
218
219 # ---- 4) 指定日期 144 时段 Q 序列（论文表3 用） ----
220 spec_q = {}
221 print('[Q2 指定日期紧急购电（论文表3）]')
222 for ds in C.SPEC_DAYS:
223     d = pd.Timestamp(ds)
224     spec_q[d] = np.asarray(rec[d]['Q'], float)
225     if spec_q[d].sum() <= 0:
226         # 合法零值：当日实际净需求低于全部情景（如周末低负荷日），计划按情景包络
227         # 多购，缺电由计划+储能完全吸收，表现为高弃光而非紧急购电。
228         print(f' 注意：{ds} 紧急购电 = 0 kWh（合法结果：当日实际净需求低于全部 S 个'
229              f'情景，计划按情景包络多购，缺电由计划+储能吸收，弃光较高）')
230     else:
231         print(f' {ds}: 紧急购电 {spec_q[d].sum():,.1f} kWh, 紧急费 '
232              f'{float(np.sum(C.EMERG_MULT * price * spec_q[d])):,.1f} 元')
233 qtab = pd.DataFrame({'时间段': C.INTERVAL_LABELS})
234 for ds in C.SPEC_DAYS:
235     qtab[ds] = np.round(spec_q[pd.Timestamp(ds)], 4)
236 qtab.to_csv(os.path.join(C.RES_DIR, 'q2_specdays_Q.csv'),
237             index=False, encoding='utf-8-sig')
238
239 # ---- 5) 摘要 CSV ----
240 summ_rows = [
241     ('全年计划购电费_元', round(plan_fee, 2)),
242     ('全年紧急购电费_元', round(emerg_fee, 2)),
243     ('合计_元', round(total_fee, 2)),
244     ('全年紧急购电量_kWh', round(emerg_kwh, 2)),
245     ('官方风险权重_*', lam),
246     ('K_L', KL), ('K_P', KP),
247 ]
248 for ds in C.SPEC_DAYS:
249     d = pd.Timestamp(ds)
250     summ_rows.append((f'{ds}_紧急购电量_kWh', round(float(spec_q[d].sum()), 4)))
251     summ_rows.append((f'{ds}_紧急购电费_元',
252                      round(float(np.sum(C.EMERG_MULT * price * spec_q[d])), 4)))
253 summ = pd.DataFrame(summ_rows, columns=['指标', '数值'])
254 summ.to_csv(os.path.join(C.RES_DIR, 'q2_new_summary.csv'),
255             index=False, encoding='utf-8-sig')
256 cv.to_csv(os.path.join(C.RES_DIR, 'q2_K_cv.csv'), index=False, encoding='utf-8-sig')
257
258 # ---- 5b) 逐日费用 CSV（供 season_split / 论文季节表，覆盖全年含预热月） ----

```

```

259 drow = []
260 for d in dates:
261     dd = rec[d]
262     pf = float(np.sum(price * dd['P']))
263     ef = float(np.sum(C.EMERG_MULT * price * dd['Q']))
264     drow.append(dict(date=str(d.date()), plan_fee=round(pf, 2),
265                     emerg_fee=round(ef, 2), total=round(pf + ef, 2)))
266 pd.DataFrame(drow).to_csv(os.path.join(C.RES_DIR, 'q2_daily_cost.csv'),
267                           index=False, encoding='utf-8-sig')
268
269 # ---- 6) 重跑一致性校验 (seed 固定 → 逐位一致; 口径变更需删旧 digest) ----
270 digest_path = os.path.join(C.RES_DIR, 'q2_digest.txt')
271 dg = _result_digest(price, dates_out, rec)
272 if os.path.exists(digest_path):
273     old = open(digest_path, encoding='utf-8').read().strip()
274     if old != dg:
275         print(f'[重跑一致性] digest 变化 (口径/ 变更所致): 旧={old[:16]}... '
276               f'新={dg[:16]}..., 已记录新 digest')
277         with open(digest_path, 'w', encoding='utf-8') as fp:
278             fp.write(dg)
279     else:
280         print(f'[重跑一致性] digest 校验通过: {dg[:16]}...')
281 else:
282     with open(digest_path, 'w', encoding='utf-8') as fp:
283         fp.write(dg)
284     print(f'[重跑一致性] 首次运行, 已记录 digest: {dg[:16]}...')
285
286 # ---- 7) 与旧完美信息口径对比 (仅对比, 不写结果) ----
287 leg = perfect_info_legacy(price, load, pv, dates)
288 leg_fee = sum(float(np.sum(price * leg[d]['P'])) for d in dates_out)
289 diff = plan_fee - leg_fee
290 print('\n[新旧口径对比]')
291 print(f'旧完美信息计划费 = {leg_fee:,.2f} 元'
292       f' (历史记录 {LEGACY_FEE_RECORD:,.2f} 元, '
293       f'{"复算吻合" if abs(leg_fee - LEGACY_FEE_RECORD) < 1.0 else "复算不一致, 请检查"})')
294 print(f'新预报式计划费 = {plan_fee:,.2f} 元, 差 = {diff:+, .2f} 元')
295 if diff > 0:
296     print('解释: 完美信息口径下计划即当日实际最优购电 (全年紧急购电恒为 0), 是计划费的'
297           '下界; 预报式口径下计划基于历史均值与整日残差情景做期望最小化, 为对冲负荷/光伏'
298           '不确定性会在低价时段多购电充当“保险”, 把缺电风险尽量转移到 1 倍计划电价而非 '
299           '5 倍紧急电价, 因此计划费略高 (保险性多购), 多出的计划费换来的是把实际缺电'
300           f'风险真实计量为全年 {emerg_fee:,.0f} 元紧急购电费 (旧口径 Q0, 该风险成本'
301           '为零计量)。')
302 else:
303     print('解释: 预报式口径计划费不高于完美信息口径, 说明历史均值预报在低价时段购电更'
304           '克制; 完美信息计划因精确贴合当日曲线而把更多购电安排在低价时段, 两者均无紧急'
305           '购电的完美信息是理论下界, 预报式口径的全部费用 = 计划费 + 紧急费才是真实可'

```

```

306         '支付成本。')
307 print(f'[Q2 完成] 全年合计费用（计划+紧急）= {total_fee:,.2f} 元')
308 return dict(price=price, dates=dates, load=load, pv=pv, rec=rec,
309             dates_out=dates_out, KL=KL, KP=KP, lam=lam,
310             plan_fee=plan_fee, emerg_fee=emerg_fee, total_fee=total_fee,
311             leg_fee=leg_fee)
312
313
314 def method_analysis(price, dates, load, pv, rec, KL, KP, lam_star=0.0):
315     """ (CVaR 权重)敏感性（论文用，不计入 result2.xlsx）。
316
317     4 个指定日期 × {0,0.5,1,2,5}：新情景生成器（(K_L,K_P) 整日成对残差 bootstrap,
318     seed=2026+日序，与官方链同源）→ solve_two_stage(lam) → 当日实际结算。
319     输出 plan_fee / emerg_fee / total / emerg_kwh 表（CSV）并画 3.20 权衡图。
320     """
321     rows = []
322     for dd in C.SPEC_DAYS:
323         d = pd.Timestamp(dd)
324         i = int(np.where(dates == d)[0][0])
325         E_start = rec[d]['E_start']
326         fc = scenarios.residual_scenarios(load[:i], pv[:i], (KL, KP), S=C.S_NUM,
327                                           seed=_seed_for_day(d))
328         for lam in [0.0, 0.5, 1.0, 2.0, 5.0]:
329             two = lp.solve_two_stage(price, fc['scen_ld'], fc['scen_pv'], E_start,
330                                     lam=lam, beta=C.BETA)
331             settle = lp.solve_day_fixedP(price, pv[i], load[i], E_start, two['P'])
332             plan_fee = float(np.sum(price * two['P']))
333             emerg_fee = float(np.sum(C.EMERG_MULT * price * settle['Q']))
334             rows.append(dict(day=dd, lam=lam, plan_fee=plan_fee, emerg_fee=emerg_fee,
335                             total=plan_fee + emerg_fee, emerg_kwh=float(settle['Q'].sum())))
336     df = pd.DataFrame(rows)
337     # 自检：=* 行应与官方链当日结算一致（同种子、同情景、同 E_start）
338     for dd in C.SPEC_DAYS:
339         d = pd.Timestamp(dd)
340         r0 = df[(df.day == dd) & (df.lam == lam_star)].iloc[0]
341         chain_plan = float(np.sum(price * rec[d]['P']))
342         chain_emer = float(np.sum(C.EMERG_MULT * price * rec[d]['Q']))
343         assert abs(r0.plan_fee - chain_plan) < 1e-3, f'{dd} = {lam_star} 计划费与官方链不一致'
344         assert abs(r0.emerg_fee - chain_emer) < 1e-3, f'{dd} = {lam_star} 紧急费与官方链不一致'
345     print('\n[Q2 敏感性 • 两阶段随机+CVaR（新情景生成器）]')
346     print(df.to_string(index=False))
347     df.to_csv(os.path.join(C.RES_DIR, 'q2_two_stage_comparison.csv'),
348              index=False, encoding='utf-8-sig')
349
350     import matplotlib
351     matplotlib.use('Agg')
352     C.setup_plot_style()

```

```

353 import matplotlib.pyplot as plt
354 df20 = df[df.day == C.SPEC_DAYS[0]].sort_values('lam')
355 fig, ax = plt.subplots(figsize=(8, 5))
356 ax.plot(df20.lam, df20.plan_fee, '-o', label='计划购电费')
357 ax.plot(df20.lam, df20.emerg_fee, '-s', label='紧急购电费')
358 ax.plot(df20.lam, df20.total, '--^', label='合计')
359 ax.set_xlabel('CVaR 权重 '); ax.set_ylabel('费用(元)')
360 ax.set_title(f'问题2 两阶段随机+CVaR: {C.SPEC_DAYS[0]} 风险-费用权衡')
361 ax.legend(); ax.grid(alpha=0.4)
362 fig.tight_layout(); fig.savefig(os.path.join(C.FIG_DIR, 'q2_cvar_tradeoff.png'), dpi=150)
363 plt.close(fig)
364 print('已写 report/q2_two_stage_comparison.csv 与 figures/q2_cvar_tradeoff.png')
365
366
367 if __name__ == '__main__':
368     A = run()
369     method_analysis(A['price'], A['dates'], A['load'], A['pv'], A['rec'], A['KL'], A['KP'],
370                     lam_star=A['lam']) # 链已按 * 运行, 自检比对 =* 行

```

## 2.7 B.7 问题三入口 (main\_q3.py)

滚动随机模型预测控制：0:00 计划、6/12/18 调整、实际光伏结算与违约/超额惩罚，写出 result3.xlsx。

```

1  # -*- coding: utf-8 -*-
2  """问题3: 滚动随机 MPC + 调整惩罚 (18.0 因果严谨版)。
3
4  0:00 用 附件1 电价 + 0:00 光伏预报(附件3) + 负荷点预报(历史逐槽均值, K=15)制定计划购电 P;
5  6/12/18 用最新光伏预报在"违约50%/超购1.5倍"目标下做**情景鲁棒滚动再计划** (随机 MPC):
6      以 scenarios.pv_scenarios 生成 S=30 个光伏情景, 一阶段购电 A 共用、各情景独立电池调度
7      (允许情景内紧急购电 5p 保证可行), 目标 = 期望电网费 (lam=0);
8      调整时点储能用**实际 SOC 回放** (以已承诺购电 A[:t0] + 当日实际负荷/光伏回放至 t0);
9      结算**去后见之明**: 逐时段 t 以 [0..t] 前缀 (固定购电 A[:t+1]、实际负荷/光伏到 t) 重解电池,
10     取末段 c/f/Q/E 结转, 替代"全天实际重优化"。
11     费用口径: cost_t = p * min(P,A) + 0.5p * (P-A)^+ + 1.5p * (A-P)^+ + 5p * Q_t。
12     """
13 import os, sys
14 sys.path.insert(0, os.path.dirname(__file__))
15 import numpy as np
16 import pandas as pd
17 import config as C
18 import data, lp, results as R
19 import scenarios
20 C.setup_plot_style()

```



```

21 import matplotlib
22 matplotlib.use('Agg')
23 import matplotlib.pyplot as plt
24 TAU = [0, 6, 12, 18]
25 SEED = 2026          # 情景抽样种子基数: 2026 + 日序 (与 Q2 对齐, 确定性可复现)
26 K_LOAD = 15          # 负荷点预报窗口 (与 Q2 K_L 交叉验证结论对齐)
27
28
29 def _load_fc_point(hist, K=7):
30     n = len(hist)
31     if n == 0:
32         return np.zeros(C.T_IN_DAY)
33     return np.mean(hist[-min(K, n):], axis=0)
34
35
36 def _adjust(price, P, A, Q):
37     """按费用口径计算: 电网费用 + 紧急购电费用。返回 (grid_fee, emerg_fee, fee_vec)。"""
38     u = np.maximum(P - A, 0.0)
39     w = np.maximum(A - P, 0.0)
40     grid_fee = float(np.sum(price * np.minimum(P, A) + C.DEFAULT_MULT * price * u +
41                             C.EXCESS_MULT * price * w))
42     emerg_fee = float(np.sum(C.EMERG_MULT * price * Q))
43     base = price * np.minimum(P, A) + C.DEFAULT_MULT * price * u + C.EXCESS_MULT * price * w +
44           C.EMERG_MULT * price * Q
45     return grid_fee, emerg_fee, base
46
47 def causal_settle(price, pv_act, load_act, E_start, A):
48     """去后见之明结算 (替代全天实际重优化)。
49
50     逐时段 t: 以 [0..t] 前缀 LP —— 固定购电 A[:t+1] (已承诺)、当日实际负荷/光伏只用到 t ——
51     重解电池充放电, 取末段 c/f/Q/R/E 作为 t 时段的真实运行量并结转储能。
52     全程只用 t 及之前的信息, 不借用未来实际光伏/负荷。
53     返回 dict(c,f,Q,R,E_end)。
54
55     """
56     T = C.T_IN_DAY
57     c = np.zeros(T); f = np.zeros(T); Q = np.zeros(T); R = np.zeros(T)
58     E_carry = E_start
59     for t in range(T):
60         res = lp.solve_day(price[:t + 1], pv_act[:t + 1], load_act[:t + 1], E_carry,
61                             T=t + 1, fixed_P=A[:t + 1], allow_P=False, emult=C.EMERG_MULT)
62         c[t] = res['c'][-1]; f[t] = res['f'][-1]
63         Q[t] = res['Q'][-1]; R[t] = res['R'][-1]
64         E_carry = res['E'][-1]
65     return dict(c=c, f=f, Q=Q, R=R, E_end=E_carry)

```

```

66 def _cold_load():
67     """冷启动负荷预报：2025-01-01 无历史，用附件1 典型日负荷剖面（与 Q2 口径一致）。"""
68     return data.load_attach1()['load']
69
70
71 def _rolling_day(price_day, load_act, pv_act, load_hist, pv_hist, fc3, d, E_start,
72                 K=K_LOAD):
73     """单日滚动核心（18.0 因果严谨版）。
74
75     - 计划/调整用负荷点预报 D_known（load_hist 前 K 天逐槽均值，K=15，与 Q2 对齐；
76       首日无历史时用附件1 典型日剖面冷启动）；
77     - 0:00 计划：solve_day_plan(点预报负荷，0:00 光伏预报)；
78     - 6/12/18 情景鲁棒再计划：pv_scenarios(G_tau, 历史) 生成 S=30 光伏情景，
79       solve_adjust_scen 共用一阶段购电 A；调整时点储能为实际 SOC 回放；
80     - 结算：causal_settle 逐段去后见之明。
81     price_day:(144) 当日电价；load_act/pv_act:(144) 当日实际；load_hist/pv_hist:(i,144)
82       截至前一日历史。
83     返回 dict(...)。
84     """
85     D_known = scenarios.point_forecast(load_hist, K) # 负荷预报化（历史逐槽均值）
86     if len(load_hist) == 0:
87         D_known = _cold_load() # 冷启动：附件1 典型日剖面
88     G0 = data.pv_forecast_kwh(fc3, d, 0)[0] # 0:00 光伏预报
89     plan = lp.solve_day_plan(price_day, G0, D_known, E_start)
90     P = plan['P']
91     A = P.copy()
92     for tau in TAU[1:]:
93         t0 = tau * 6
94         G_tau = data.pv_forecast_kwh(fc3, d, tau)[0]
95         # 实际 SOC 回放：已承诺购电 A[:t0] + 当日实际负荷/光伏至 t0
96         pre = lp.solve_day(price_day[:t0], pv_act[:t0], load_act[:t0], E_start,
97                             T=t0, fixed_P=A[:t0], allow_P=False, emult=C.EMERG_MULT)
98         E_tau = pre['E'][-1]
99         # 情景鲁棒滚动再计划（随机 MPC）：光伏情景 S=30，负荷用点预报
100        scenG = scenarios.pv_scenarios(G_tau, pv_hist, S=C.S_NUM,
101                                        seed=SEED + int(d.dayofyear))
102        rp = lp.solve_adjust_scen(price_day, scenG, D_known, E_tau, P, t0, lam=0.0)
103        A[t0:] = rp['A']
104    # 结算去后见之明
105    settle = causal_settle(price_day, pv_act, load_act, E_start, A)
106    grid_fee, emerg_fee, _ = _adjust(price_day, P, A, settle['Q'])
107    return dict(P=P, A=A, Q=settle['Q'], c=settle['c'], f=settle['f'],
108                grid_fee=grid_fee, emerg_fee=emerg_fee, E_end=settle['E_end'])
109
110 def _rolling(price, load, pv, dates, fc3, i, E_start, K=K_LOAD):
111     """单日滚动（18.0 因果严谨版，按日期索引取数后委托 _rolling_day）。"""

```

```

112     return _rolling_day(price, load[i], pv[i], load[:i], pv[:i], fc3, dates[i],
113                          E_start, K=K)
114
115
116 def run(outfile='result3.xlsx'):
117     a1 = data.load_attach1()
118     price = a1['price']
119     dates, load, pv = data.load_attach2()
120     fc3 = data.load_attach3()
121     rec = {}
122     E_start = C.E_INIT
123     for i in range(len(dates)):
124         d = dates[i]
125         res = _rolling(price, load, pv, dates, fc3, i, E_start)
126         res['E_start'] = E_start
127         rec[d] = res
128         E_start = res['E_end']
129
130     dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
131     P = {d: rec[d]['P'] for d in dates_out}
132     A = {d: rec[d]['A'] for d in dates_out}
133     Q = {d: rec[d]['Q'] for d in dates_out}
134     c = {d: rec[d]['c'] for d in dates_out}
135     f = {d: rec[d]['f'] for d in dates_out}
136     E0 = {d: rec[d]['E_start'] for d in dates_out}
137     Ee = {d: rec[d]['E_end'] for d in dates_out}
138     path = os.path.join(C.RES_DIR, outfile)
139     R.write_result3(path, dates_out, P, A, Q, c, f, E0, Ee)
140
141     tot_grid = sum(rec[d]['grid_fee'] for d in dates_out)
142     tot_emerg = sum(rec[d]['emerg_fee'] for d in dates_out)
143     tot = tot_grid + tot_emerg
144     print('已写出', path)
145     print(f'[Q3 滚动MPC] 计划+调整电网费={tot_grid:.2f} 元, 紧急购电费={tot_emerg:.2f} 元,
146           合计={tot:.2f} 元')
147
148     # 逐日费用 CSV (供 season_split / 论文季节表, 覆盖全年含预热月)
149     drow = [dict(date=str(d.date()), grid_fee=round(rec[d]['grid_fee'], 2),
150                emerg_fee=round(rec[d]['emerg_fee'], 2),
151                total=round(rec[d]['grid_fee'] + rec[d]['emerg_fee'], 2)) for d in dates]
152     pd.DataFrame(drow).to_csv(os.path.join(C.RES_DIR, 'q3_daily_cost.csv'),
153                               index=False, encoding='utf-8-sig')
154
155     return dict(price=price, dates=dates, load=load, pv=pv, fc3=fc3, rec=rec,
156                dates_out=dates_out)
157
158 def analysis(price, dates, load, pv, fc3, rec):

```

```

157 """是否需要其他时刻预报：比较 仅0:00(不调整) vs 0/6/12/18 滚动（均用因果结算）。"""
158 rows = []
159 for dd in C.SPEC_DAYS:
160     d = pd.Timestamp(dd)
161     i = int(np.where(dates == d)[0][0])
162     E_start = rec[d]['E_start']
163     D_known = scenarios.point_forecast(load[:i], K_LOAD)
164     G0 = data.pv_forecast_kwh(fc3, d, 0)[0]
165     plan = lp.solve_day_plan(price, G0, D_known, E_start)
166     st = causal_settle(price, pv[i], load[i], E_start, plan['P'])
167     g0, e0, _ = _adjust(price, plan['P'], plan['P'], st['Q'])
168     r = _rolling(price, load, pv, dates, fc3, i, E_start)
169     rows.append(dict(day=dd, only0_grid=g0, only0_emerg=e0, only0_total=g0 + e0,
170                     roll_grid=r['grid_fee'], roll_emerg=r['emerg_fee'],
171                     roll_total=r['grid_fee'] + r['emerg_fee']))
172 df = pd.DataFrame(rows)
173 print('\n[Q3 调整时点分析]')
174 print(df.to_string(index=False))
175 df.to_csv(os.path.join(C.RES_DIR, 'q3_rolling_comparison.csv'), index=False,
176           encoding='utf-8-sig')
177 x = np.arange(len(df))
178 w = 0.32
179 fig, ax = plt.subplots(figsize=(9, 5))
180 ax.bar(x - w / 2, df.only0_total, w, label='仅0:00(不调整)')
181 ax.bar(x + w / 2, df.roll_total, w, label='0/6/12/18 滚动')
182 ax.set_xticks(x); ax.set_xticklabels(df.day, fontsize=11)
183 ax.set_ylabel('总购电费用(元)', fontsize=12); ax.set_title('问题3：是否引入多时刻预报',
184                 fontsize=13)
185 ax.legend(fontsize=11); ax.grid(alpha=0.4, axis='y')
186 ax.tick_params(labelsize=11)
187 fig.tight_layout(); fig.savefig(os.path.join(C.FIG_DIR, 'q3_forecast_time.png'), dpi=150);
188 plt.close(fig)
189
190 if __name__ == '__main__':
191     A = run()
192     analysis(A['price'], A['dates'], A['load'], A['pv'], A['fc3'], A['rec'])

```

## 2.8 B.8 问题四入口（main\_q4.py）

基于附件4波动电价重算问题二、三，输出 result4-2.xlsx / result4-3.xlsx 并做季节对比。

```

1 # -*- coding: utf-8 -*-

```

```

2  """问题4: 波动电价(附件4)重算 问题2/问题3 → result4-2.xlsx / result4-3.xlsx + 对比图。
3
4  附件2(负载/光伏实际)、附件4(波动电价) 均为 全年366×144 矩阵, 日期必须对齐。
5  口径沿用 (18.0 与 Q2/Q3 官方口径对齐):
6  Q4-2 = 问题2 预报式两阶段随机规划 (K_L/K_P 沿用 Q2 交叉验证、=* 沿用 Q2 官方扫描、
7          seed=2026+日序), 仅把固定电价(附件1)换成当日波动电价 → 紧急购电不再恒 0;
8  Q4-3 = 问题3 因果滚动随机 MPC (负荷预报化/实际SOC/去后见之明结算/光伏情景鲁棒),
9          按当日波动电价结算(复用 main_q3._rolling_day)。
10 """
11 import os, sys
12 sys.path.insert(0, os.path.dirname(__file__))
13 import numpy as np
14 import pandas as pd
15 import config as C
16 import data, lp, results as R
17 import main_q2 as MQ2
18 from main_q3 import _rolling_day
19 C.setup_plot_style()
20 import matplotlib
21 matplotlib.use('Agg')
22 import matplotlib.pyplot as plt
23 TAU = [0, 6, 12, 18]
24
25
26 # ----- Q4-2: 波动电价 + 问题2 预报式两阶段随机规划 -----
27 def run_q42():
28     dates4, price4 = data.load_attach4() # price4[365,144]
29     dates, load, pv = data.load_attach2()
30     data.assert_aligned(dates4, dates, '附件4', '附件2')
31     # K 沿用 Q2 交叉验证 (只依赖负荷/光伏, 与电价口径无关)
32     KL, KP, _ = MQ2.cross_validate_K(dates, load, pv)
33     # * 沿用 Q2 官方扫描 (风险权衡权重, 与电价口径无关)
34     summ = pd.read_csv(os.path.join(C.RES_DIR, 'q2_new_summary.csv'))
35     lam_star = float(summ.loc[summ['指标'] == '官方风险权重_*', '数值'].iloc[0])
36     print(f'[Q4-2] K_L={KL}, K_P={KP}, *={lam_star} (沿用 Q2 官方口径)')
37     rec = MQ2._run_forecast_chain(price4, dates, load, pv, KL, KP, S=C.S_NUM,
38                                   lam=lam_star, verbose=False)
39     dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
40     P = {d: rec[d]['P'] for d in dates_out}
41     Q = {d: rec[d]['Q'] for d in dates_out}
42     c = {d: rec[d]['c'] for d in dates_out}
43     f = {d: rec[d]['f'] for d in dates_out}
44     E0 = {d: rec[d]['E_start'] for d in dates_out}
45     Ee = {d: rec[d]['E_end'] for d in dates_out}
46     path = os.path.join(C.RES_DIR, 'result4-2.xlsx')
47     R.write_result2(path, dates_out, P, Q, c, f, E0, Ee, tpl_name='result4-2.xlsx')
48     idx_of = {d: i for i, d in enumerate(dates)}

```

```

49 cost = sum(float(np.sum(price4[idx_of[d]] * rec[d]['P']))) for d in dates_out)
50 emerg = sum(float(np.sum(C.EMERG_MULT * price4[idx_of[d]] * rec[d]['Q']))) for d in
    dates_out)
51 print('已写出', path)
52 print(f'[Q4-2 波动电价+预报表] 计划费={cost:,.2f} 紧急费={emerg:,.2f} '
53       f'合计={cost + emerg:,.2f} 元, 紧急购电 0 kWh')
54 pd.DataFrame({'指标': ['全年计划购电费_元', '全年紧急购电费_元', '合计_元', '紧急购电量_kWh'],
55              '数值': [round(cost, 2), round(emerg, 2), round(cost + emerg, 2),
56                      round(sum(float(rec[d]['Q']).sum() for d in dates_out), 2)]})
57     ).to_csv(os.path.join(C.RES_DIR, 'q4_2_summary.csv'),
58             index=False, encoding='utf-8-sig')
59 # 逐日费用 CSV (供 season_split / 论文季节表, 覆盖全年含预热月)
60 drow = []
61 for d in dates:
62     dd = rec[d]
63     pf = float(np.sum(price4[idx_of[d]] * dd['P']))
64     ef = float(np.sum(C.EMERG_MULT * price4[idx_of[d]] * dd['Q']))
65     drow.append(dict(date=str(d.date()), plan_fee=round(pf, 2),
66                    emerg_fee=round(ef, 2), total=round(pf + ef, 2)))
67 pd.DataFrame(drow).to_csv(os.path.join(C.RES_DIR, 'q4_2_daily_cost.csv'),
68                          index=False, encoding='utf-8-sig')
69 return dict(dates=dates, load=load, pv=pv, price4=price4,
70            rec=rec, dates_out=dates_out, cost=cost, emerg=emerg)
71
72
73 # ----- Q4-3: 波动电价 + 因果滚动随机 MPC -----
74 def _adjust(price, P, A, Q):
75     u = np.maximum(P - A, 0.0)
76     w = np.maximum(A - P, 0.0)
77     grid_fee = float(np.sum(price * np.minimum(P, A) + C.DEFAULT_MULT * price * u +
78                             C.EXCESS_MULT * price * w))
79     emerg_fee = float(np.sum(C.EMERG_MULT * price * Q))
80     return grid_fee, emerg_fee
81
82 def run_q43():
83     dates4, price4 = data.load_attach4()
84     dates, load, pv = data.load_attach2()
85     fc3 = data.load_attach3()
86     data.assert_aligned(dates4, dates, '附件4', '附件2')
87     rec = {}
88     E_start = C.E_INIT
89     for i in range(len(dates)):
90         pr = price4[i]
91         # 复用 main_q3 的 18.0 因果滚动模型 (负荷预报化/实际SOC/去后见之明/光伏情景鲁棒)
92         res = _rolling_day(pr, load[i], pv[i], load[:i], pv[:i], fc3, dates[i], E_start)
93         res['E_start'] = E_start

```

```

94     rec[dates[i]] = res
95     E_start = res['E_end']
96     dates_out = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
97     P = {d: rec[d]['P'] for d in dates_out}
98     A = {d: rec[d]['A'] for d in dates_out}
99     Q = {d: rec[d]['Q'] for d in dates_out}
100    c = {d: rec[d]['c'] for d in dates_out}
101    f = {d: rec[d]['f'] for d in dates_out}
102    E0 = {d: rec[d]['E_start'] for d in dates_out}
103    Ee = {d: rec[d]['E_end'] for d in dates_out}
104    path = os.path.join(C.RES_DIR, 'result4-3.xlsx')
105    R.write_result3(path, dates_out, P, A, Q, c, f, E0, Ee, tpl_name='result4-3.xlsx')
106    tot_grid = sum(rec[d]['grid_fee'] for d in dates_out)
107    tot_emerg = sum(rec[d]['emerg_fee'] for d in dates_out)
108    print('已写出', path)
109    print(f'[Q4-3 波动电价+滚动MPC] 电网费={tot_grid:.2f} 紧急费={tot_emerg:.2f} 合计={tot_grid
        + tot_emerg:.2f} 元')
110    # 逐日费用 CSV (供 season_split / 论文季节表, 覆盖全年含预热月)
111    drow = [dict(date=str(d.date()), grid_fee=round(rec[d]['grid_fee'], 2),
112                emerg_fee=round(rec[d]['emerg_fee'], 2),
113                total=round(rec[d]['grid_fee'] + rec[d]['emerg_fee'], 2)) for d in dates]
114    pd.DataFrame(drow).to_csv(os.path.join(C.RES_DIR, 'q4_3_daily_cost.csv'),
115                              index=False, encoding='utf-8-sig')
116    return dict(dates=dates, dates_out=dates_out, rec=rec,
117                tot_grid=tot_grid, tot_emerg=tot_emerg)
118
119
120 def shell():
121     q42 = run_q42()
122     q43 = run_q43()
123     dates4, price4 = data.load_attach4()
124     a1 = data.load_attach1()
125     mean_p = price4.mean(axis=1)
126     x = np.arange(len(mean_p))
127     fig, ax = plt.subplots(figsize=(10, 4.5))
128     ax.plot(x, mean_p, lw=1.2, label='附件4 波动电价(日均)')
129     p_fix = float(a1['price'].mean())
130     ax.axhline(p_fix, color='r', ls='--', lw=1, label=f'附件1 固定电价 (日均 {p_fix:.4f}
        元/kWh) ')
131     ax.set_xlabel('日期索引(1.1起)', fontsize=12); ax.set_ylabel('平均电价(元/kWh)', fontsize=12)
132     ax.set_title('问题4: 波动电价(附件4)与固定电价(附件1)对比', fontsize=13)
133     ax.legend(fontsize=11); ax.grid(alpha=0.3)
134     ax.tick_params(labelsize=11)
135     fig.tight_layout(); fig.savefig(os.path.join(C.FIG_DIR, 'q4_price.png'), dpi=150);
        plt.close(fig)
136     print('图已存', os.path.join(C.FIG_DIR, 'q4_price.png'))
137

```

```

138
139 if __name__ == '__main__':
140     shell()

```

## 2.9 B.9 结果模板回填 (results.py)

按附件 5 模板结构用 openpyxl 将计划/调整购电、充放电、紧急购电与储电量写入结果 Excel。

```

1  # -*- coding: utf-8 -*-
2  """结果写入：按 附件5 模板结构用 openpyxl 填值。策略：
3  - 计划购电量/调整购电量：模板已预置行(2.1-12.31 共334天)，就地填 144 列。
4  - 充放电/紧急购电量：模板仅示例，按天数扩展行。
5  """
6  import os
7  import numpy as np
8  import openpyxl
9  import config as C
10
11
12 def _save(wb, path):
13     try:
14         wb.save(path); return path
15     except PermissionError:
16         alt = os.path.join(os.path.dirname(path),
17                             os.path.splitext(os.path.basename(path))[0] + '_new.xlsx')
18         wb.save(alt)
19         print('[警告] 目标被占用，已写副本:', alt)
20         return alt
21
22
23 def _fill_plan_sheet(ws, data):
24     """data: {datetime/date日期: arr[144]}, 就地填 计划/调整购电量。"""
25     lookup = {}
26     for k, arr in data.items():
27         key = k.date() if hasattr(k, 'date') else k
28         lookup[key] = arr
29     for row in range(2, ws.max_row + 1):
30         dt = ws.cell(row=row, column=1).value
31         if dt is None or isinstance(dt, str):
32             continue
33         key = dt.date() if hasattr(dt, 'date') else dt
34         arr = lookup.get(key)
35         if arr is None:

```



```

36         continue
37     for t in range(C.T_IN_DAY):
38         ws.cell(row=row, column=2 + t, value=round(float(arr[t]), 4))
39
40
41 def _clear_data_rows(ws, keep_first_col=True):
42     """清空模板中残留的数据(第2行起)。
43     keep_first_col=True(默认): 保留第1列(时间段标签), 只清数据列。"""
44     for row in range(2, ws.max_row + 1):
45         start = 2 if keep_first_col else 1
46         for col in range(start, ws.max_column + 1):
47             ws.cell(row=row, column=col).value = None
48
49
50 def _fill_chongfang(ws, dates_out, c_all, f_all, EO_all, Eend_all):
51     """充放电量: 日期|时间段|充电量|放电量|时刻|储电量。每日期 6块+0:00+24:00=8行。"""
52     from datetime import time as tm
53     _clear_data_rows(ws, keep_first_col=False)
54     r = 2
55     for d in dates_out:
56         for h0, lab in C.BLOCK4:
57             lo = h0 * 6; hi = lo + 24
58             ws.cell(row=r, column=1, value=str(d))
59             ws.cell(row=r, column=2, value=lab)
60             ws.cell(row=r, column=3, value=round(float(np.sum(c_all[d][lo:hi])), 4))
61             ws.cell(row=r, column=4, value=round(float(np.sum(f_all[d][lo:hi])), 4))
62             r += 1
63             ws.cell(row=r, column=1, value=str(d)); ws.cell(row=r, column=5, value='0:00')
64             ws.cell(row=r, column=6, value=round(float(EO_all[d]), 4)); r += 1
65             ws.cell(row=r, column=1, value=str(d)); ws.cell(row=r, column=5, value='24:00')
66             ws.cell(row=r, column=6, value=round(float(Eend_all[d]), 4)); r += 1
67
68
69 def _fill_jinji(ws, dates_out, Q_all):
70     _clear_data_rows(ws, keep_first_col=False)
71     r = 2
72     for d in dates_out:
73         for t in range(C.T_IN_DAY):
74             q = Q_all[d][t]
75             if q > 1e-6:
76                 ws.cell(row=r, column=1, value=str(d))
77                 ws.cell(row=r, column=2, value=C.INTERVAL_LABELS[t])
78                 ws.cell(row=r, column=3, value=round(float(q), 4))
79                 r += 1
80
81
82 def write_result1(path, P, c, f, E_start, E_end, price):

```

```

83 """问题1 result1.xlsx —— 共需填 2 张表。
84
85 表1【计划购电量】：模板已预置 144 行时间段(A列2-145)，就地填 B 列购电量。
86 表2【充放电量】：模板 7 行(2-7块 + 0:00/24:00 储电)，就地填 充电量/放电量(B/C列
87 2-7行) 与 储电量(E列，0:00在2行、24:00在3行)。
88
89 tpl = os.path.join(C.TPL_DIR, 'result1.xlsx')
90 wb = openpyxl.load_workbook(tpl)
91
92 # 表1 计划购电量：B 列(2-145) 填购电量，A 列时间段保留
93 ws = wb['计划购电量']
94 for t in range(C.T_IN_DAY):
95     ws.cell(row=2 + t, column=2, value=round(float(P[t]), 4))
96
97 # 表2 充放电量：就地填 6 块充电/放电量 + 0:00/24:00 储电量
98 ws = wb['充放电量']
99 for h0, lab in C.BLOCK4:
100     lo = h0 * 6; hi = lo + 24
101     r = 2 + h0 // 4 # 块索引 0..5 -> 行 2..7
102     ws.cell(row=r, column=2, value=round(float(np.sum(c[lo:hi])), 4))
103     ws.cell(row=r, column=3, value=round(float(np.sum(f[lo:hi])), 4))
104     ws.cell(row=2, column=4, value='0:00'); ws.cell(row=2, column=5,
105         value=round(float(E_start), 4))
106     ws.cell(row=3, column=4, value='24:00'); ws.cell(row=3, column=5, value=round(float(E_end),
107         4))
108     wb._template = None
109     return _save(wb, path)
110
111 def write_result2(path, dates_out, P, Q, c, f, E0, Eend, tpl_name='result2.xlsx'):
112     """问题2/问题4-2。dates_out: 日期列表(2.1-12.31)。各 dict: date->arr/标量。"""
113     tpl = os.path.join(C.TPL_DIR, tpl_name)
114     wb = openpyxl.load_workbook(tpl)
115     _fill_plan_sheet(wb['计划购电量'], P)
116     _fill_jinji(wb['紧急购电量'], dates_out, Q)
117     _fill_chongfang(wb['充放电量'], dates_out, c, f, E0, Eend)
118     wb._template = None
119     return _save(wb, path)
120
121 def write_result3(path, dates_out, P, A, Q, c, f, E0, Eend, tpl_name='result3.xlsx'):
122     """问题3/问题4-3。"""
123     tpl = os.path.join(C.TPL_DIR, tpl_name)
124     wb = openpyxl.load_workbook(tpl)
125     _fill_plan_sheet(wb['计划购电量'], P)
126     _fill_plan_sheet(wb['调整购电量'], A)
127     _fill_jinji(wb['紧急购电量'], dates_out, Q)

```

```

128     _fill_chongfang(wb['充放电电量'], dates_out, c, f, E0, Eend)
129     wb._template = None
130     return _save(wb, path)
131
132
133 def strip_personal_meta(path):
134     """【新增】清除结果文件的文档属性中的个人元数据（creator/lastModifiedBy），
135     避免提交文件泄露个人信息（模板作者字段曾为个人名）。不动其他内容。"""
136     wb = openpyxl.load_workbook(path)
137     wb.properties.creator = ''
138     wb.properties.lastModifiedBy = ''
139     try:
140         wb.save(path)
141     except PermissionError:
142         alt = os.path.join(os.path.dirname(path),
143                             os.path.splitext(os.path.basename(path))[0] + '_new.xlsx')
144         wb.save(alt)
145         print(['警告'] 元数据清除时目标被占用，已写副本:', alt)
146         return alt
147     return path

```

## 2.10 B.10 季节费用拆分（season\_split.py）

固定/波动电价 × 季节的四类日费用真实计算，验证电价波动风险的季节差异。

```

1  # -*- coding: utf-8 -*-
2  """问题四季节费用拆分：固定/波动电价 × 季节 的日费用（18.0：直接聚合官方逐日费用 CSV）。
3
4  数据来源（各主脚本运行时生成，口径与官方结果完全一致）：
5      q2_daily_cost.csv   Q2 预报式两阶段随机规划（固定电价，= *）
6      q4_2_daily_cost.csv Q4-2 同口径波动电价
7      q3_daily_cost.csv   Q3 因果滚动随机 MPC（固定电价）
8      q4_3_daily_cost.csv Q4-3 同口径波动电价
9  """
10 import os, sys
11 sys.path.insert(0, os.path.dirname(__file__))
12 import numpy as np
13 import pandas as pd
14 import config as C
15
16 RES = C.RES_DIR
17
18
19 def _load(path):
20     df = pd.read_csv(os.path.join(RES, path))

```

```

21     return {pd.Timestamp(r['date']): float(r['total']) for _, r in df.iterrows()}
22
23
24 def season(d):
25     m = d.month
26     return {12: '冬', 1: '冬', 2: '冬', 3: '春', 4: '春', 5: '春',
27            6: '夏', 7: '夏', 8: '夏', 9: '秋', 10: '秋', 11: '秋'}[m]
28
29
30 SORDER = ['春', '夏', '秋', '冬']
31
32 cost_fix2 = _load('q2_daily_cost.csv')
33 cost_var2 = _load('q4_2_daily_cost.csv')
34 cost_fix3 = _load('q3_daily_cost.csv')
35 cost_var3 = _load('q4_3_daily_cost.csv')
36 dates = sorted(cost_fix2)
37
38 rows = []
39 for s in SORDER:
40     idx = [d for d in dates if season(d) == s]
41     a = np.array([cost_fix2[d] for d in idx]); b = np.array([cost_var2[d] for d in idx])
42     c = np.array([cost_fix3[d] for d in idx]); e = np.array([cost_var3[d] for d in idx])
43     rows.append(dict(seas=s, n=len(idx),
44                    q2_fix=a.mean() / 1e4, q2_var=b.mean() / 1e4,
45                    q3_fix=c.mean() / 1e4, q3_var=e.mean() / 1e4,
46                    up3=(e.mean() - c.mean()) / c.mean() * 100))
47 out = pd.DataFrame(rows)
48 print(out.round(2).to_string(index=False))
49 out.to_csv(os.path.join(RES, 'season_cost_split.csv'), index=False, encoding='utf-8-sig')
50
51 # 全年总 (2.1-12.31, 与论文口径一致)
52 do = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
53 for name, dd in [('Q2固定', cost_fix2), ('Q2波动', cost_var2),
54                 ('Q3固定', cost_fix3), ('Q3波动', cost_var3)]:
55     print(name, round(sum(dd[d] for d in do) / 1e4, 1), '万元')

```

## 2.11 B.11 季节特征分析 (seasonal\_check.py)

对负荷、光伏与电价做季节分组与统计，支撑问题分析中的季节分型与 ANOVA 检验。

```

1  # -*- coding: utf-8 -*-
2  """季节差异实证：负荷/光伏/优化结果按季节分组统计，评估'按季节分类'创新点可行性。"""
3  import os, sys, json

```

```

4 sys.path.insert(0, os.path.dirname(__file__))
5 import numpy as np
6 import pandas as pd
7 import config as C
8 import data, lp
9
10 def season(d):
11     m = d.month
12     return
13     {12:'冬',1:'冬',2:'冬',3:'春',4:'春',5:'春',6:'夏',7:'夏',8:'夏',9:'秋',10:'秋',11:'秋'}[m]
14
15 a1 = data.load_attach1()
16 dates, load, pv = data.load_attach2() # kWh/时段 (365,144)
17 dates4, p4 = data.load_attach4()
18
19 # 1) 季节负荷/光伏总量
20 day_ld = load.sum(axis=1) # kWh/日
21 day_pv = pv.sum(axis=1)
22 seas = [season(d) for d in dates]
23 df = pd.DataFrame({'seas': seas, 'ld': day_ld, 'pv': day_pv})
24 g = df.groupby('seas').agg(ld_mean=('ld', 'mean'), ld_std=('ld', 'std'),
25                             pv_mean=('pv', 'mean'),
26                             pv_std=('pv', 'std')).reindex(['春', '夏', '秋', '冬'])
27
28 print('=== 季节负荷/光伏(日 kWh) ===')
29 print(g.round(1).to_string())
30
31 # 2) 季节日内峰谷特征 (平均日负荷曲线峰值/谷值)
32 day_peak = load.max(axis=1); day_trough = load.min(axis=1)
33 df['pk'] = day_peak; df['tr'] = day_trough
34 g2 = df.groupby('seas')[['pk', 'tr']].mean().reindex(['春', '夏', '秋', '冬'])
35 print('\n=== 季节平均日峰/谷负荷(kWh/时段) ===')
36 print(g2.round(1).to_string())
37
38 # 3) 完美信息(Q2口径) 下季节购电费
39 cost_seas = {}
40 E = C.E_INIT
41 for i, d in enumerate(dates):
42     rp = lp.solve_day_plan(a1['price'], pv[i], load[i], E)
43     cost_seas.setdefault(season(d), []).append(float(np.sum(a1['price']*rp['P'])))
44     E = rp['E'][-1]
45 rows = []
46 for s in ['春', '夏', '秋', '冬']:
47     a = np.array(cost_seas[s])
48     rows.append(dict(seas=s, n=len(a), mean=a.mean(), std=a.std(),
49                     mn=a.min(), mx=a.max()))
50 g3 = pd.DataFrame(rows)
51 print('\n=== 季节日购电费分布(元) ===')

```

```

49 print(g3.round(0).to_string(index=False))
50
51 # 4) 波动电价季节均值
52 p_mean = p4.mean(axis=1)
53 df['pm'] = p_mean
54 g4 = df.groupby('seas')['pm'].mean().reindex(['春', '夏', '秋', '冬'])
55 print('\n=== 季节平均电价(元/kWh) ===')
56 print(g4.round(4).to_string())
57
58 # 5) 光伏季节日峰值时刻（负荷峰谷错位程度）
59 # 光伏正午最高、负荷晚峰，计算 负荷峰谷比
60 ratio = day_peak / np.maximum(day_trough, 1e-6)
61 df['ratio'] = ratio
62 g5 = df.groupby('seas')['ratio'].mean().reindex(['春', '夏', '秋', '冬'])
63 print('\n=== 季节日负荷峰谷比 ===')
64 print(g5.round(2).to_string())
65
66 print('\nANOVA(季节间购电费差异显著性):')
67 from scipy import stats
68 f, p = stats.f_oneway(cost_seas['春'], cost_seas['夏'], cost_seas['秋'], cost_seas['冬'])
69 print(f'F={f:.1f}, p={p:.2e} ->', '季节差异显著' if p < 0.05 else '差异不显著')

```

## 2.12 B.12 进阶图表（chart\_advanced.py）

生成箱线图、小提琴图、相关热图、帕累托前沿、雷达图、QQ 图、桑基图等分析图（图 A1–A8）。

```

1 # -*- coding: utf-8 -*-
2 """子任务3：高级图表生成（全部基于真实数据 result/附件，服务明确结论）。
3
4 生成 8 张图 → C题/figures_adv/:
5     图A1 季节购电费箱型图      （结论：冬>夏>秋>春，ANOVA p<1e-16 → 支持'按季节分类'创新点）
6     图A2 季节负荷/光伏柱+误差条 （结论：冬光伏最低、夏负荷最高 → 季节分类动因）
7     图A3 波动电价季节分布小提琴图 （结论：冬电价均值最高且右尾重 → 电价季节风险）
8     图A4 - 敏感性热力图      （结论：↑总费↓； 影响小 → 模型对 稳健）
9     图A5 帕累托前沿：计划费 vs 紧急费 （结论： 提供风险-经济折中前沿）
10    图A6 四问雷达图          （结论：随机/滚动模型以计算复杂度换可靠性与鲁棒性）
11    图A7 光伏预报误差 Q-Q 图   （结论：预报误差近似正态但右尾重 → 随机情景必要性）
12    图A8 典型日能量流向桑基图 （结论：购电结构按季节/时段显著不同）
13 """
14 import os, sys
15 sys.path.insert(0, os.path.dirname(__file__))
16 import numpy as np
17 import pandas as pd

```

```

18 import config as C
19 import data, lp, scenarios
20 C.setup_plot_style()
21 import matplotlib
22 matplotlib.use('Agg')
23 import matplotlib.pyplot as plt
24 from matplotlib import colors as mcolors
25
26 OUT = os.path.join(C.C_BASE, 'figures_adv')
27 os.makedirs(OUT, exist_ok=True)
28 CB = ['#0072B2', '#D55E00', '#009E73', '#CC79A7', '#F0E442', '#56B4E9', '#E69F00'] # 色盲友好
29
30 def season(d):
31     m = d.month
32     return
33         {12:'冬',1:'冬',2:'冬',3:'春',4:'春',5:'春',6:'夏',7:'夏',8:'夏',9:'秋',10:'秋',11:'秋'}[m]
34
35 SORDER = ['春','夏','秋','冬']
36
37 a1 = data.load_attach1(); fp = a1['price']
38 dates, load, pv = data.load_attach2()
39 dates4, p4 = data.load_attach4()
40 fc3 = data.load_attach3()
41
42 # ----- 公共：逐日完美信息购电费 -----
43 day_cost = {}
44 E = C.E_INIT
45 for i, d in enumerate(dates):
46     rp = lp.solve_day_plan(fp, pv[i], load[i], E)
47     day_cost[d] = float(np.sum(fp * rp['P']))
48     E = rp['E'][-1]
49
50 df = pd.DataFrame({'date': dates, 'seas': [season(d) for d in dates],
51                  'cost': [day_cost[d] for d in dates],
52                  'ld': load.sum(axis=1), 'pv': pv.sum(axis=1),
53                  'pm': p4.mean(axis=1)})
54
55 # ===== 图A1 季节购电费箱型图 =====
56 fig, ax = plt.subplots(figsize=(8, 4.6))
57 data_s = [df[df.seas == s]['cost'] / 1e4 for s in SORDER]
58 bp = ax.boxplot(data_s, tick_labels=SORDER, patch_artist=True, widths=0.55,
59                 medianprops=dict(color='k', lw=1.4))
60 for patch, cc in zip(bp['boxes'], CB[:4]):
61     patch.set_facecolor(cc); patch.set_alpha(0.65)
62 ax.set_ylabel('日购电费（万元）'); ax.set_xlabel('季节')
63 ax.set_title('图A1 各季节日购电费分布（完美信息口径）')
64 ax.grid(alpha=0.3, axis='y')
65 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A1_season_cost_box.png'), dpi=150);

```

```

plt.close(fig)

64
65 # ===== 图A2 季节负荷/光伏 =====
66 g = df.groupby('seas')[['ld', 'pv']].mean().reindex(SORDER) / 1e4
67 gstd = df.groupby('seas')[['ld', 'pv']].std().reindex(SORDER) / 1e4
68 x = np.arange(4); w = 0.36
69 fig, ax = plt.subplots(figsize=(8, 4.6))
70 ax.bar(x - w/2, g['ld'], w, yerr=gstd['ld'], capsize=3, color=CB[0], alpha=0.85,
       label='日负荷均值')
71 ax.bar(x + w/2, g['pv'], w, yerr=gstd['pv'], capsize=3, color=CB[1], alpha=0.85,
       label='日光伏均值')
72 ax.set_xticks(x); ax.set_xticklabels(SORDER)
73 ax.set_ylabel('日能量 (万 kWh)'); ax.set_title('图A2 各季节日负荷与光伏出力 (均值±标准差)')
74 ax.legend(); ax.grid(alpha=0.3, axis='y')
75 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A2_season_ld_pv.png'), dpi=150);
    plt.close(fig)
76
77 # ===== 图A3 季节电价小提琴图 =====
78 fig, ax = plt.subplots(figsize=(8, 4.6))
79 vp = ax.violinplot([df[df.seas == s]['pm'] for s in SORDER],
80                    positions=[0,1,2,3], widths=0.7, showmedians=True)
81 for b, cc in zip(vp['bodies'], CB[:4]):
82     b.set_facecolor(cc); b.set_alpha(0.6)
83 ax.set_xticks([0,1,2,3]); ax.set_xticklabels(SORDER)
84 ax.set_ylabel('日平均电价 (元/kWh)')
85 ax.set_title('图A3 各季节日平均电价分布 (附件4)')
86 ax.grid(alpha=0.3, axis='y')
87 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A3_price_violin.png'), dpi=150);
    plt.close(fig)
88
89 # ===== 图A4/A5: - 敏感性 (两阶段随机+CVaR, 2025-03-20) =====
90 D0 = pd.Timestamp('2025-03-20'); i0 = int(np.where(dates == D0)[0][0])
91 E0 = C.E_INIT
92 for j in range(i0):
93     rp = lp.solve_day_plan(fp, pv[j], load[j], E0); E0 = rp['E'][-1]
94 hist_lo = load[max(0, i0-9):i0]; hist_pv = pv[max(0, i0-9):i0]
95 fc = scenarios.forecast_and_scenarios(hist_lo, hist_pv, S=C.S_NUM, seed=i0+1)
96
97 LAMS = [0, 0.2, 0.5, 1, 2, 5]
98 BETS = [0.80, 0.85, 0.90, 0.95, 0.99]
99 grid = np.zeros((len(LAMS), len(BETS)))
100 plan_arr = np.zeros((len(LAMS), len(BETS)))
101 emerg_arr = np.zeros((len(LAMS), len(BETS)))
102 for ai, lam in enumerate(LAMS):
103     for bi, beta in enumerate(BETS):
104         two = lp.solve_two_stage(fp, fc['scen_ld'], fc['scen_pv'], E0, lam=lam, beta=beta)
105         settle = lp.solve_day_fixedP(fp, pv[i0], load[i0], E0, two['P'])

```



```

106     pf = float(np.sum(fp * two['P']))
107     ef = float(np.sum(5 * fp * settle['Q']))
108     plan_arr[ai, bi] = pf; emerg_arr[ai, bi] = ef
109     grid[ai, bi] = pf + ef
110     print(f'lam={lam} beta={beta} plan={pf:.0f} emerg={ef:.0f} total={pf+ef:.0f}')
111
112 # 热力图（总费用）
113 fig, ax = plt.subplots(figsize=(8.2, 5.4))
114 im = ax.imshow(grid, aspect='auto', cmap='YlOrRd')
115 ax.set_xticks(range(len(BETS))); ax.set_xticklabels([str(b) for b in BETS], fontsize=11)
116 ax.set_yticks(range(len(LAMS))); ax.set_yticklabels([str(l) for l in LAMS], fontsize=11)
117 for r in range(len(LAMS)):
118     for c in range(len(BETS)):
119         ax.text(c, r, f'{grid[r,c]/1e4:.1f}', ha='center', va='center', fontsize=11)
120 ax.set_xlabel('CVaR 置信水平 ', fontsize=12); ax.set_ylabel('风险权重 ', fontsize=12)
121 ax.set_title('图A4 - 对当日总费用（万元）的敏感性（2025-03-20）', fontsize=13)
122 cb = fig.colorbar(im, ax=ax); cb.set_label('总费用（万元）', fontsize=12)
123 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A4_heatmap_lambda_beta.png'), dpi=150);
    plt.close(fig)
124
125 # 帕累托前沿（=0.9）
126 fig, ax = plt.subplots(figsize=(8.2, 5.4))
127 ax.plot(plan_arr[:, 2]/1e4, emerg_arr[:, 2]/1e4, '-o', color=CB[0], lw=2.0)
128 for ai, lam in enumerate(LAMS):
129     px, py = plan_arr[ai, 2]/1e4, emerg_arr[ai, 2]/1e4
130     dy = (8 if ai % 2 == 0 else -14) if ai < len(LAMS) - 1 else -14
131     ax.annotate(f'={lam}', (px, py), textcoords='offset points', xytext=(10, dy),
132               fontsize=11, bbox=dict(fc='white', ec='none', alpha=0.7, pad=1))
133 ax.set_xlabel('计划购电费（万元）', fontsize=12); ax.set_ylabel('紧急购电费（万元）',
    fontsize=12)
134 ax.set_title('图A5 计划购电费—紧急购电费 帕累托前沿（=0.9, 2025-03-20）', fontsize=13)
135 ax.tick_params(labelsize=11)
136 ax.grid(alpha=0.3)
137 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A5_pareto.png'), dpi=150); plt.close(fig)
138
139 # ===== 图A6 雷达图 =====
140 fig, ax = plt.subplots(figsize=(7.4, 6.6), subplot_kw=dict(polar=True))
141 dims = ['经济性', '供电可靠性', '鲁棒性', '计算复杂度', '信息利用度']
142 # 半定量打分 0-10: 确定性LP / 两阶段随机 / 滚动MPC / 波动电价重算
143 vals = dict(LP=[8, 4, 2, 9, 3], 随机=[6, 9, 8, 4, 7], MPC=[7, 8, 9, 3, 8], 波动=[5, 8, 8, 3, 9])
144 N = len(dims); ang = np.linspace(0, 2*np.pi, N, endpoint=False).tolist(); ang += ang[:1]
145 for k, (name, v) in enumerate(vals.items()):
146     vv = v + v[:1]
147     ax.plot(ang, vv, label=name, color=CB[k], lw=1.8)
148     ax.fill(ang, vv, color=CB[k], alpha=0.12)
149 ax.set_xticks(ang[:-1]); ax.set_xticklabels(dims, fontsize=12)
150 ax.set_ylim(0, 10); ax.set_yticks([2,4,6,8,10]); ax.set_yticklabels(['2', '4', '6', '8', '10'],

```

```

        fontsize=10)
151 ax.set_title('图A6 四种模型在五维度上的对比（半定量评分）', fontsize=13)
152 ax.legend(loc='upper right', bbox_to_anchor=(1.32, 1.12), fontsize=11)
153 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A6_radar.png'), dpi=150); plt.close(fig)
154
155 # ===== 图A7 光伏预报误差 Q-Q =====
156 # 用附件3 0:00 预报 与 附件2 实际 的日总量误差（取全年）
157 day_fc = {}
158 for d in dates:
159     g0, _ = data.pv_forecast_kwh(fc3, d, 0)
160     day_fc[d] = g0.sum()
161 err = np.array([day_fc[d] - pv[i].sum() for i, d in enumerate(dates)])
162 err = err / np.maximum(np.array([pv[i].sum() for i in range(len(dates))]), 1e-6) * 100 #
    %相对误差
163 err = err[np.isfinite(err)]
164 from scipy import stats
165 fig, ax = plt.subplots(figsize=(7, 5.2))
166 stats.probplot(err, dist='norm', plot=ax)
167 ax.set_title('图A7 光伏日预报相对误差的 Q-Q 图（附件3 vs 附件2）')
168 ax.set_xlabel('理论分位数'); ax.set_ylabel('样本分位数 (%)')
169 ax.grid(alpha=0.3)
170 fig.tight_layout(); fig.savefig(os.path.join(OUT, 'A7_pv_err_qq.png'), dpi=150); plt.close(fig)
171
172 # ===== 图A8 桑基图：典型日能量流向 =====
173 try:
174     from matplotlib.sankey import Sankey
175 except Exception:
176     print('sankey unavailable'); A8 = False
177 else:
178     # 用 2025-06-21 完美信息日：电网购电 + 光伏 + 储能放电 → 负荷 + 充电 + 弃光
179     d = pd.Timestamp('2025-06-21'); i = int(np.where(dates == d)[0][0])
180     E = C.E_INIT
181     for j in range(i):
182         rp = lp.solve_day_plan(fp, pv[j], load[j], E); E = rp['E'][-1]
183     rp = lp.solve_day_plan(fp, pv[i], load[i], E)
184     P = rp['P'].sum(); G = pv[i].sum(); F = (C.ETA*rp['f']).sum() # 放电(输出侧)
185     D = load[i].sum(); Ch = rp['c'].sum(); R = rp['R'].sum()
186     # 标准两列能量流向图（自绘，标签外置零遮挡；替代 matplotlib.sankey 楔形旧图）
187     from matplotlib.path import Path
188     from matplotlib.patches import PathPatch, Rectangle
189     A8C = ['#0072B2', '#D55E00', '#009E73']
190     # 流带分解（守恒：P + (G-R-Ch) + F = D; Ch、R 记光伏）
191     assert G - R - Ch > 0, '光伏直供为负，分带方案不适用'
192     bands = [(0, 0, P), (1, 0, G - R - Ch), (1, 1, Ch), (1, 2, R), (2, 0, F)]
193     L_NAMES, R_NAMES = ['电网购电', '光伏出力', '储能放电'], ['负荷', '储能充电', '弃光']
194     L_VALS, R_VALS = [P, G, F], [D, Ch, R]
195     SRC_COLOR = {0: A8C[0], 1: A8C[1], 2: A8C[2]}

```

```

196 TP = sum(v for _, _, v in bands)
197 Y0, H, GAP = 0.12, 0.74, 0.035
198 def _stack(vals):
199     hs = [v / TP * (H - GAP * (len(vals) - 1)) for v in vals]
200     out, y = [], 1.0
201     for h in hs:
202         out.append((y, y - h)); y -= h + GAP
203     return out
204 LT, RT = _stack(L_VALS), _stack(R_VALS)
205 X0, X1, W = 0.30, 0.70, 0.028
206 fig, ax = plt.subplots(figsize=(9.2, 5.0))
207 for k, (t, b) in enumerate(LT):
208     ax.add_patch(Rectangle((X0 - W, b), W, t - b, facecolor=SRC_COLOR[k],
209                             edgecolor='black', linewidth=0.8, alpha=0.95))
210     ax.text(X0 - W - 0.02, (t + b) / 2, f'{L_NAMES[k]}\n{L_VALS[k]:.0f} kWh',
211             ha='right', va='center', fontsize=12)
212 for k, (t, b) in enumerate(RT):
213     ax.add_patch(Rectangle((X1, b), W, t - b,
214                             facecolor=[ '#444444', A8C[2], '#888888' ][k],
215                             edgecolor='black', linewidth=0.8, alpha=0.95))
216     ax.text(X1 + W + 0.02, (t + b) / 2, f'{R_NAMES[k]}\n{R_VALS[k]:.0f} kWh',
217             ha='left', va='center', fontsize=12)
218 rcur = {k: RT[k][0] for k in range(3)}
219 lcur = {k: LT[k][0] for k in range(3)}
220 for (s, t, v) in bands:
221     h = v / TP * (H - GAP * (len(R_VALS) - 1))
222     y0t, y0b = lcur[s], lcur[s] - h
223     y1t, y1b = rcur[t], rcur[t] - h
224     lcur[s] -= h; rcur[t] -= h
225     xm = (X0 + X1) / 2
226     pth = Path([(X0, y0t), (xm, y0t), (xm, y1t), (X1, y1t),
227                 (X1, y1b), (xm, y1b), (xm, y0b), (X0, y0b), (X0, y0t)],
228                [Path.MOVETO, Path.CURVE4, Path.CURVE4, Path.CURVE4,
229                 Path.LINETO, Path.CURVE4, Path.CURVE4, Path.CURVE4, Path.CLOSEPOLY])
230     ax.add_patch(PathPatch(pth, facecolor=SRC_COLOR[s], alpha=0.35, edgecolor='none'))
231 ax.set_xlim(0, 1); ax.set_ylim(0.07, 1.02); ax.axis('off')
232 fig.tight_layout()
233 for out in {OUT, os.path.join(C.C_BASE, 'paper', 'figures_adv')}:
234     os.makedirs(out, exist_ok=True)
235     fig.savefig(os.path.join(out, 'A8_sankey.png'), dpi=200)
236 plt.close(fig)
237 A8 = True
238
239 print('\n全部图表已输出至', OUT)
240 print(sorted(os.listdir(OUT)))

```

## 2.13 B.13 问题四图表 (chart\_4\_0.py)

生成风险敏感性、季节费用对比、滚动调整对比与全年费用结构图 (图 A9–A12)。

```
1  # -*- coding: utf-8 -*-
2  """4.0版优化补充图：由 results/ 下 CSV 生成 4 张图表 A9-A12，落盘 figures_adv/。
3  仅读取 results 下的 CSV，不重新求解，故可独立、低开销运行。"""
4  import os, sys
5  sys.path.insert(0, os.path.dirname(__file__))
6  import numpy as np
7  import pandas as pd
8  import config as C
9  C.setup_plot_style()
10 import matplotlib
11 matplotlib.use('Agg')
12 import matplotlib.pyplot as plt
13
14 RES = C.RES_DIR
15 FIGD = os.path.join(C.C_BASE, 'figures_adv')
16 os.makedirs(FIGD, exist_ok=True)
17 DAY_LABEL = {'2025-03-20': '03-20', '2025-06-21': '06-21',
18             '2025-09-23': '09-23', '2025-12-21': '12-21'}
19
20 # ===== A9 问题二：CVaR 风险敏感性分组柱状 (=0 vs =2, 堆叠 计划/紧急) =====
21 df2 = pd.read_csv(os.path.join(RES, 'q2_two_stage_comparison.csv'))
22 d = df2[(df2['lam'].isin([0.0, 2.0]))].copy()
23 days = ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']
24 x = np.arange(len(days)); w = 0.36
25 fig, ax = plt.subplots(figsize=(9.5, 5.2))
26 for i, lam in enumerate([0.0, 2.0]):
27     sub = d[d['lam'] == lam].set_index('day').reindex(days)
28     off = (i - 0.5) * w
29     bars_p = ax.bar(x + off, sub['plan_fee'].values / 1e4, w, label=f'={int(lam)} 计划购电费')
30     bars_e = ax.bar(x + off, sub['emerg_fee'].values / 1e4, w,
31                    bottom=sub['plan_fee'].values / 1e4, label=f'={int(lam)} 紧急购电费')
32     for b in list(bars_p) + list(bars_e):
33         ax.text(b.get_x() + b.get_width() / 2, b.get_y() + b.get_height() + 0.1,
34                f'{b.get_height():.1f}', ha='center', fontsize=10)
35 ax.set_xticks(x); ax.set_xticklabels([DAY_LABEL[k] for k in days], fontsize=11)
36 ax.set_xlabel('指定日期', fontsize=12); ax.set_ylabel('费用 (万元)', fontsize=12)
37 ax.set_title('问题二方法层：风险权重 下"计划费换可靠性"权衡 (堆叠柱)', fontsize=13)
38 ax.legend(fontsize=10); ax.grid(axis='y', ls='--', alpha=0.4)
39 ax.tick_params(labelsize=11)
40 fig.tight_layout(); fig.savefig(os.path.join(FIGD, 'A9_q2_cvar_compare.png'), dpi=150);
    plt.close(fig)
```

```

42 # ===== A10 问题四：季节日均费用 固定/波动 对比（问题三） =====
43 dfs = pd.read_csv(os.path.join(RES, 'season_cost_split.csv'))
44 seasons = list(dfs['seas'])
45 x = np.arange(len(seasons)); w = 0.36
46 fig, ax = plt.subplots(figsize=(8.5, 5.0))
47 b1 = ax.bar(x - w / 2, dfs['q3_fix'], w, label='固定电价', color='#4C72B0')
48 b2 = ax.bar(x + w / 2, dfs['q3_var'], w, label='波动电价', color='#DD8452')
49 for b in list(b1) + list(b2):
50     ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 0.03,
51             f'{b.get_height():.2f}', ha='center', fontsize=10)
52 for i, up in enumerate(dfs['up3']):
53     ax.text(x[i] + w / 2, max(dfs['q3_fix'][i], dfs['q3_var'][i]) + 0.35,
54             f'#{up:.1f}%' if up >= 0 else f'#{up:.1f}%',
55             ha='center', fontsize=10, color='#C44E52', bbox=dict(fc='white', ec='#C44E52',
56                             lw=0.6, boxstyle='round,pad=0.15'))
57 ax.set_xticks(x); ax.set_xticklabels(seasons, fontsize=11)
58 ax.set_xlabel('季节', fontsize=12); ax.set_ylabel('日均费用（万元/日）', fontsize=12)
59 ax.set_title('问题四：波动电价下季节日均费用的季节性差异（问题三）', fontsize=13)
60 ax.legend(fontsize=10); ax.grid(axis='y', ls='--', alpha=0.4)
61 ax.tick_params(labelsize=11)
62 fig.tight_layout(); fig.savefig(os.path.join(FIGD, 'A10_season_cost.png'), dpi=150);
63 plt.close(fig)
64
65 # ===== A11 问题三：仅0:00 vs 滚动 总费用对比（堆叠 电网/紧急） =====
66 df3 = pd.read_csv(os.path.join(RES,
67                                'q3_rolling_comparison.csv')).set_index('day').reindex(days)
68 x = np.arange(len(days)); w = 0.36
69 fig, ax = plt.subplots(figsize=(9.5, 5.2))
70 for i, (k, lb, c1, c2) in enumerate([('only0', '仅 0:00', '#4C72B0', '#74A7D8'),
71                                     ('roll', '0/6/12/18 滚动', '#DD8452', '#F2B08C')]):
72     off = (i - 0.5) * w
73     gr = df3[k + '_grid'].values / 1e4; em = df3[k + '_emerg'].values / 1e4
74     ax.bar(x + off, gr, w, label=f'{lb} 电网费', color=c1)
75     ax.bar(x + off, em, w, bottom=gr, label=f'{lb} 紧急购电费', color=c2)
76     for j in range(len(days)):
77         t = gr[j] + em[j]
78         ax.text(x[j] + off, t + 0.1, f'{t:.1f}', ha='center', fontsize=10)
79 ax.set_xticks(x); ax.set_xticklabels([DAY_LABEL[k] for k in days], fontsize=11)
80 ax.set_xlabel('指定日期', fontsize=12); ax.set_ylabel('费用（万元）', fontsize=12)
81 ax.set_title('问题三：引入多时刻预报前后总费用对比（堆叠柱）', fontsize=13)
82 ax.legend(fontsize=10, ncol=2); ax.grid(axis='y', ls='--', alpha=0.4)
83 ax.tick_params(labelsize=11)
84 fig.tight_layout(); fig.savefig(os.path.join(FIGD, 'A11_q3_roll_compare.png'), dpi=150);
85 plt.close(fig)
86
87 # ===== A12 全年费用结构：问题二/三/四-3（电网费 vs 紧急购电费 堆叠） =====
88 labels = ['问题二\n(完美信息)', '问题三\n(滚动MPC)', '问题四-问题三\n(波动电价)']

```

```

85 grid = [12259844.62, 13008939.33, 13591972.19]
86 emerg = [0.0, 2254548.51, 2454440.51]
87 x = np.arange(len(labels)); w = 0.5
88 fig, ax = plt.subplots(figsize=(8.5, 5.0))
89 b1 = ax.bar(x, np.array(grid) / 1e6, w, label='电网费', color='#4C72B0')
90 b2 = ax.bar(x, np.array(emerg) / 1e6, w, bottom=np.array(grid) / 1e6,
91          label='紧急购电费', color='#C44E52')
92 for j in range(3):
93     tot = (grid[j] + emerg[j]) / 1e6
94     ax.text(x[j], tot + 5, f'{tot:.0f} 万元', ha='center', fontsize=11,
95            bbox=dict(fc='white', ec='#333', lw=0.5, boxstyle='round,pad=0.2'))
96     ax.text(x[j], (grid[j] + emerg[j] / 2) / 1e6, f'{emerg[j]/1e4:.0f} 万元',
97            ha='center', fontsize=10, color='white' if emerg[j] > 1e5 else '#333')
98 ax.set_xticks(x); ax.set_xticklabels(labels, fontsize=11)
99 ax.set_ylabel('费用 (百万元)', fontsize=12); ax.set_ylim(0, 20)
100 ax.set_title('全年费用结构: 随机/滚动模型逐步逼近真实成本的对比', fontsize=13)
101 ax.legend(fontsize=10); ax.grid(axis='y', ls='--', alpha=0.4)
102 ax.tick_params(labelsize=11)
103 fig.tight_layout(); fig.savefig(os.path.join(FIGD, 'A12_annual_cost.png'), dpi=150);
104     plt.close(fig)
105
106 print('已生成: ', [f'A9_q2_cvar_compare.png', 'A10_season_cost.png',
107                   'A11_q3_roll_compare.png', 'A12_annual_cost.png'], '->', FIGD)

```

## 2.14 B.14 论文化取数 (extract\_paper\_day.py)

从求解结果提取论文正文所需的指定日数值与图数据。

```

1  # -*- coding: utf-8 -*-
2  """抓取指定日期4天在 Q2/Q3/Q4 下的详细结果，供论文表格。"""
3  import os, sys, json
4  sys.path.insert(0, os.path.dirname(__file__))
5  import numpy as np
6  import pandas as pd
7  import config as C
8  import data, lp
9  import main_q3 as M3
10
11  SLOT = [m // 10 for m in C.SPEC_MINUTES]
12  SLOT_LAB = ['10:00-10:10', '12:00-12:10', '14:00-14:10', '16:00-16:10', '18:00-18:10',
13             '20:00-20:10']
14
15  BLK = [(0, '0-4'), (4, '4-8'), (8, '8-12'), (12, '12-16'), (16, '16-20'), (20, '20-24')]
16
17  def blk(arr):
18     return [round(float(np.sum(arr[h * 6:(h + 4) * 6])), 1) for h, _ in BLK]

```

```

17
18 def price_day(*a):
19     pass
20
21 a1 = data.load_attach1(); fp = a1['price']
22 dates, load, pv = data.load_attach2()
23 fc3 = data.load_attach3()
24 dates4, p4 = data.load_attach4()
25
26 out = {}
27 for dd in C.SPEC_DAYS:
28     d = pd.Timestamp(dd); i = int(np.where(dates == d)[0][0])
29     E0 = C.E_INIT if i == 0 else None # 需追溯前一日本末
30     row = {}
31     # ---- 前日本末 (Q2) ----
32     E = C.E_INIT
33     for j in range(i):
34         rp = lp.solve_day_plan(fp, pv[j], load[j], E); E = rp['E'][-1]
35     Eprev2 = E
36     Eprev3 = C.E_INIT
37     for j in range(i):
38         r = M3._rolling(fp, load, pv, dates, fc3, j, Eprev3, 7); Eprev3 = r['E_end']
39     Eprev42 = C.E_INIT
40     for j in range(i):
41         rp = lp.solve_day_plan(p4[j], pv[j], load[j], Eprev42); Eprev42 = rp['E'][-1]
42     Eprev43 = C.E_INIT
43     for j in range(i):
44         import main_q4 as M4
45         r = M4._rolling(p4[j], load[j], pv[j], fc3, dates[j], Eprev43, 7); Eprev43 = r['E_end']
46
47     # Q2
48     r2 = lp.solve_day_plan(fp, pv[i], load[i], Eprev2)
49     # Q4-2
50     r42 = lp.solve_day_plan(p4[i], pv[i], load[i], Eprev42)
51     # Q3
52     r3 = M3._rolling(fp, load, pv, dates, fc3, i, Eprev3, 7)
53     # Q4-3
54     r43 = M4._rolling(p4[i], load[i], pv[i], fc3, dates[i], Eprev43, 7)
55
56     row['Q2'] = dict(e0=round(Eprev2,1), e24=round(r2['E'][-1],1),
57                     p_slots=[round(float(r2['P'][t]),1) for t in SLOT],
58                     c_blk=blk(r2['c']), f_blk=blk(r2['f']),
59                     cost_day=round(float(np.sum(fp*r2['P'])),1))
60     row['Q4-2'] = dict(e0=round(Eprev42,1), e24=round(r42['E'][-1],1),
61                       p_slots=[round(float(r42['P'][t]),1) for t in SLOT],
62                       c_blk=blk(r42['c']), f_blk=blk(r42['f']),
63                       cost_day=round(float(np.sum(p4[i]*r42['P'])),1))

```

```

64 row['Q3'] = dict(e0=round(Eprev3,1), e24=round(r3['E_end'],1),
65                 p_slots=[round(float(r3['P'][t]),1) for t in SLOT],
66                 a_slots=[round(float(r3['A'][t]),1) for t in SLOT],
67                 c_blk=blk(r3['c']), f_blk=blk(r3['f']),
68                 q_sum=round(float(r3['Q'].sum()),1), nq=int((r3['Q']>1e-6).sum()),
69                 grid=round(r3['grid_fee'],1), emerg=round(r3['emerg_fee'],1))
70 row['Q4-3'] = dict(e0=round(Eprev43,1), e24=round(r43['E_end'],1),
71                   p_slots=[round(float(r43['P'][t]),1) for t in SLOT],
72                   a_slots=[round(float(r43['A'][t]),1) for t in SLOT],
73                   c_blk=blk(r43['c']), f_blk=blk(r43['f']),
74                   q_sum=round(float(r43['Q'].sum()),1), nq=int((r43['Q']>1e-6).sum()),
75                   grid=round(r43['grid_fee'],1), emerg=round(r43['emerg_fee'],1))
76 out[dd] = row
77
78 print(json.dumps(out, ensure_ascii=False, indent=1))
79 print('SLOT_LAB', SLOT_LAB)

```

## 2.15 B.15 论文化取数 (extract\_paper\_nums.py)

汇总全年成本、误差等关键数值，供论文定量表述使用。

```

1  # -*- coding: utf-8 -*-
2  """抓取论文所需全部数值 → 供 tex 引用。"""
3  import os, sys, json
4  sys.path.insert(0, os.path.dirname(__file__))
5  import numpy as np
6  import pandas as pd
7  import config as C
8  import data, lp
9
10 SPEC_MIN_SLOT = [m // 10 for m in C.SPEC_MINUTES] # 时段索引: 10:00,12:00,...
11 BLK = [(0, '0-4'), (4, '4-8'), (8, '8-12'), (12, '12-16'), (16, '16-20'), (20, '20-24')]
12
13 def blk_sums(arr):
14     return [float(np.sum(arr[h0 * 6:(h0 + 4) * 6])) for h0, _ in BLK]
15
16 def q1():
17     a1 = data.load_attach1()
18     rp = lp.solve_day_plan(a1['price'], a1['pv'], a1['load'], C.E_INIT, force_end=C.E_INIT)
19     P = rp['P']; c = rp['c']; f = rp['f']
20     return dict(
21         slots=[float(P[t]) for t in SPEC_MIN_SLOT],
22         total=float(P.sum()), cost=float(np.sum(a1['price'] * P)),
23         c_blk=blk_sums(c), f_blk=blk_sums(f), E0=float(rp['E'][0]), E24=float(rp['E'][-1]))
24

```



```

25 def day_rows(price, load, pv, dates, out_dates, plan_rec):
26     out = {}
27     for dd in C.SPEC_DAYS:
28         d = pd.Timestamp(dd)
29         i = int(np.where(dates == d)[0][0])
30         pr = price[d] if d in plan_rec else None
31         r = plan_rec[d] if d in plan_rec else None
32         out[dd] = dict(idx=i)
33     return out
34
35 def compute():
36     a1 = data.load_attach1()
37     fp = a1['price']
38     dates, load, pv = data.load_attach2()
39     fc3 = data.load_attach3()
40     dates4, p4 = data.load_attach4()
41
42     # ---- Q1 ----
43     r1 = q1()
44
45     # ---- Q2 (固定电价 完美信息) 全年 + 指定4日块数据 ----
46     rec2 = {}; E = C.E_INIT
47     for i in range(len(dates)):
48         rp = lp.solve_day_plan(fp, pv[i], load[i], E)
49         rec2[dates[i]] = rp
50         E = rp['E'][-1]
51     do2 = [d for d in dates if d >= pd.Timestamp('2025-02-01')]
52     cost2 = float(sum(np.sum(fp * rec2[d]['P']) for d in do2))
53
54     # ---- Q4-2 (波动电价 完美信息) ----
55     rec42 = {}; E = C.E_INIT
56     for i in range(len(dates)):
57         pr = p4[i]
58         rp = lp.solve_day_plan(pr, pv[i], load[i], E)
59         rec42[dates[i]] = rp
60         E = rp['E'][-1]
61     cost42 = float(sum(np.sum(p4[i] * rec42[d]['P']) for i, d in enumerate(do2)))
62
63     return dict(r1=r1, cost2=cost2, cost42=cost42,
64               dates=[str(d.date()) for d in dates],
65               spec=[dict(d=str(x.date()), i=int(np.where(dates == pd.Timestamp(x))[0][0]),
66                      E2=rec2[x]['E'][-1], E42=rec42[x]['E'][-1]) for x in map(pd.Timestamp,
67                                C.SPEC_DAYS)])
68
69 d = compute()
70 print(json.dumps(d, ensure_ascii=False, indent=1))

```

## 2.16 B.16 页数校验工具 (page\_check.py)

统计编译后 PDF 页数，辅助核验”正文  $\leq 30$  页”等篇幅要求。

```
1  # -*- coding: utf-8 -*-
2  """统计 main.pdf 各章节页码分布。"""
3  import PyPDF2, re
4
5  r = PyPDF2.PdfReader(r'E:\desktop\2026建模\C题\paper\main.pdf')
6  print('总页数:', len(r.pages))
7  marks = ['摘要', '摘要', '问题重述', '问题分析', '模型假设', '符号说明',
8           '模型建立与求解', '模型检验与分析', '模型评价与推广', '参考文献',
9           '人工智能工具使用声明', '附录']
10 for i, p in enumerate(r.pages, 1):
11     t = p.extract_text() or ''
12     t1 = re.sub(r'\s+', ' ', t)[:120]
13     hits = [m for m in marks if m in t[:200]]
14     print(f'p{i:>2} | {" ".join(hits):<16} | {t1[:70]}')
```